Technical University of Crete
School of Electrical and Computer Engineering



Diploma Thesis

Augmented Reality Interface for Robotic Arm Control via Gaze-Based Interaction

Author:
Stamatios Alexiou

Committee:
Aikaterini Mania
Michail G. Lagoudakis
Nikolaos Giatrakos

A thesis submitted in fulfillment of the requirements for the degree of Electrical and Computer Engineering

Chania, May 2026

TECHNICAL UNIVERSITY OF CRETE

Electrical and Computer Engineering

Abstract

Augmented Reality Interface for Robotic Arm Control via Gaze-Based Interaction

by Stamatios ALEXIOU

Human-Robot Interfaces are pivotal in bridging the gap between humans and robots, enabling intuitive communication and collaboration. Robotic pick-and-place constitutes the core of autonomous manipulation. When conducted in cluttered or complex environments, robots must jointly reason about the selected grasp and desired placement locations to ensure successful execution. This diploma thesis presents a simulation-based augmented reality (AR) framework designed to support gaze-driven control of a robotic arm for object pick-and-place tasks. The proposed interface allows users to select the target object either through gaze dwell or blink interaction. Once an object is selected, candidate grasp poses are generated by an integrated Grasp Pose Detection (GPD) system operating on the object's point cloud, producing the top ten feasible grasp configurations. However, since GPD does not take into consideration scene complexity, a machine learning model is trained to evaluate the generated grasps based on their suitability within the given scene. For placement specification, user manipulates a virtual clone of the selected object in space using gaze fixation and locks it at the desired placement position via blinking. After, both grasp and placement poses have been acquired, this information is transmitted to the Robot Operating System (ROS). Motion Planning and trajectory generation are then performed using MoveIt, ensuring collision-free execution. The resulting trajectory is sent back to the system, where user receives real-time visual feedback of the robot performing the task or, in the case of failure, an informative log indicating the cause (e.g., unreachable target). The proposed framework was evaluated in simulated manipulation scenarios with two levels of scene complexity. The results demonstrate reliable gaze-based object selection, effective grasp execution, and efficient task completion. Overall, this work contributes to the fields of augmented reality and robotic manipulation by presenting a gaze-driven AR interface that integrates learning-based grasp selection with ROS-based motion planning.

Acknowledgements

Completing this thesis, I feel deeply indebted to a great many people who have inspired and supported me.

In particular, I extend my deepest gratitude to my supervisor, Aikaterini Mania, for her invaluable guidance, insightful comments, and considerable encouragement to complete this thesis. Her commitment to academic excellence and meticulous attention to detail have had a significant influence on this work.

My sincere thanks go to the entire SURREAL Lab, whose supportive and collaborative research environment, along with its resources and assistance, played an essential role in the development of this thesis.

Finally, on a personal level, I wish to express my heartfelt appreciation to my family and my friends Panagiotis, Ermina, Giorgos, and Nefeli, who have provided constant support and love throughout the development of the thesis and my academic journey in general.

Contents

Abstract **i**

Acknowledgements **ii**

List of Figures **vi**

List of Tables **x**

Acronyms **xi**

1 Introduction **1**
1.1 Brief Overview 1
1.2 Thesis Structure 5

2 Background/Research Overview **6**
2.1 Robotics 6
2.1.1 Brief History of Robotics 6
2.1.2 Robots in different fields 8
2.1.3 The Power of Simulation in Robotics 11
2.2 Augmented Reality 12
2.2.1 Brief History of AR 12
2.2.2 Applications of AR 14
2.3 Human-Computer Interaction (HCI) in AR 16
2.4 Human-Robot Interaction (HRI) using AR 17

3 Technological Background and Definitions **19**
3.1 Unity Game Engine 19
3.2 Magic Leap 2 (ML2) 20
3.2.1 Magic Leap SDK (MLSDK) 22
3.2.2 OpenXR 22
3.3 Robot Operating System (ROS) 23
3.3.1 Unified Robot Description Format (URDF) 23

3.3.2 MoveIt Motion Planning Framework . . . 24
3.3.3 ROS Visualization (RViz) . . . 25
3.4 Grasp Pose Detection (GPD) . . . 26
3.5 Machine Learning (ML) . . . 26

4 Augmented Reality Interaction and System Workflow 28
4.1 System Initialization and Environment Setup . . . 28
4.2 Marker-Based Scene Instantiation . . . 34
4.3 Scene Navigation and Object Selection . . . 35
4.4 Determining Placement Location . . . 38
4.5 Robot Arm Trajectory Execution . . . 40
4.6 Reset Functionality and UI Behavior . . . 41

5 Implementation 42
5.1 System Architecture . . . 43
5.2 Unity Initial Setup . . . 45
5.3 Niryo One Robotic Arm . . . 47
5.4 User Interface (UI) . . . 49
5.4.1 UI Components and Layout . . . 50
5.4.2 Interaction Methods . . . 52
5.5 Grasp Generation and Evaluation . . . 59
5.5.1 Initial Setup . . . 60
5.5.2 Point Cloud Representation . . . 61
5.5.3 Grasp Generation Implementation . . . 61
5.5.4 Machine Learning-Based Grasp Evaluation . . . 62
5.6 ROS–Unity Integration Architecture . . . 70
5.6.1 ROS Messages and Services Overview . . . 71
5.6.2 MoveIt Planning Scene and Motion Planning Service . . . 72
5.6.3 Scene-Dependent Planning Adaptations and Coordinate Transforms 76
5.6.4 Trajectory Execution, Gripper Control, and Failure Handling . . . 77
5.7 System Overall Workflow . . . 78
5.7.1 High-Level Interaction Loop . . . 78
5.7.2 Step-by-Step Communication Pipeline . . . 78

6 Evaluation & Testing 82
6.1 Machine Learning Models . . . 83
6.1.1 Model Selection . . . 83
6.1.2 Hyperparameter Optimization . . . 86
6.1.3 Decision Threshold Optimization . . . 88
6.1.4 Comparative Analysis in Grasping Trials . . . 90
6.2 Evaluation of Gaze-Based Interaction Methods . . . 92

6.2.1 Experimental Setup 92
6.2.2 Participants and Evaluation Metrics 95
6.3 Evaluation of the Whole System 101
6.3.1 Experimental Setup 101
6.3.2 Participants and Results 101

7 Conclusion & Future Work **104**
7.1 Conclusion 104
7.2 Future Work 105

Bibliography **106**

List of Figures

1.1 Application preview 3

2.1 The evolution of modern robotics 7
2.2 Example of the mobile robot GRASMOOR which was built at the University of Manchester 8
2.3 A laparoscopic robotic surgery machine. Patient-side cart of the da Vinci surgical system 9
2.4 Factory automation with industrial robots for palletizing food products like bread and toast at a bakery in Germany 10
2.5 Valkyrie (R5), a humanoid robot from NASA 10
2.6 Reality–Virtuality continuum schematic 12
2.7 Configuration of the "Sword of Damocles" 13
2.8 Value of IAR across industry 15
2.9 Examples of augmented reality and robotics research: A) collaborative programming, B) augmented arms for social robots, C) drone teleoperation, D) a drone-mounted projector, E) projected background for data physicalization, F) drone navigation for invisible areas, G) trajectory visualization, H) motion intent for pedestrian, I) a holographic avatar for telepresence robots, J) surface augmentation for shape displays 18

3.1 Unity 19
3.2 Magic Leap 2 21
3.3 ROS distributions and their compatibility with Ubuntu versions 23
3.4 MoveIt motion planning framework for ROS 24
3.5 RViz: a ROS graphical interface 25
3.6 Machine learning algorithms 27
3.7 Example of binary classification 27

4.1 Network configuration interface 29
4.2 Gesture-based interaction methods for the virtual keyboard 30
4.3 Automatic removal of input field after ROS IP address entry 30
4.4 Custom Fit application and calibration process 31

4.5 Selection of gaze-based interaction method . . . 32
4.6 Scene selection via gaze-based interaction . . . 33
4.7 Activity diagram of system initialization . . . 33
4.8 Scene instantiation process . . . 34
4.9 Interaction with the “Select” button . . . 35
4.10 Target object selection workflow for robotic manipulation . . . 36
4.11 Flowchart of scene instantiation and object selection . . . 37
4.12 Virtual clone instantiation . . . 38
4.13 Placement location selection process . . . 39
4.14 Comparison of robot arm trajectory execution results under different conditions . . . 40
4.15 Before and after pressing the “Reset” button . . . 41
4.16 User interface orientation adapting to different user positions . . . 41

5.1 System Architecture . . . 44
5.2 Configuration of the Unity project using the Magic Leap Setup Tool: (Left) before applying the setup, (Right) after applying the setup . . . 45
5.3 Modifications to the Unity Physics Settings for improved robot simulation 46
5.4 Components of the Niryo One robotic arm (without gripper) . . . 47
5.5 Niryo One robotic arm parameters: (Left) part dimensions (mm), (Right) maximum joint angles (degrees) . . . 47
5.6 Niryo One gripper: physical dimensions . . . 48
5.7 Niryo One imported in Unity . . . 48
5.8 XRI Spatial Keyboard prefab . . . 50
5.9 World-space canvas positioned above a manipulable object . . . 52
5.10 OpenXR hand joint conventions for left and right hands . . . 53
5.11 Key poses used by the OpenXR Hand Interactions Extension . . . 53
5.12 Configuration of Unity UI Button visual states . . . 54
5.13 Example of saccades and fixations, eye-movement is represented in red . 55
5.14 Application permissions configured in the Magic Leap project settings. Dangerous permissions require user confirmation at runtime . . . 58
5.15 Visualization of a target object in Unity and its corresponding point cloud used for grasp pose detection . . . 61
5.16 Selected grasps returned from GPD for a given point cloud . . . 62
5.17 Dataset structure . . . 63
5.18 Experimental scene setup for offline data collection and labeling . . . 64
5.19 Offline data collection architecture . . . 64
5.20 Data labeling process for CSV entries . . . 65
5.21 Unity Robotics Hub . . . 70
5.22 ROS Topics: Publisher and Subscriber Guide . . . 71
5.23 ROS Services: Service Client and Service Server Guide . . . 71

5.24 Obstacles included in script to be published 74
5.25 Synchronization of the simulation environment: (a) setup of the robot and obstacles within Unity (b) the resulting published planning scene in RViz via ROS 75
5.26 ROS - Unity Coordinate System Conversions 77

6.1 Detailed performance comparison for the positive class (Class 1) using Random Forest, XGBoost, LightGBM, and CatBoost models. The chart visualizes the Precision, Recall, and F1-score values as extracted from the model evaluation results, with exact values displayed above each bar . . . 85
6.2 ROC curve for the optimized XGBoost model. The AUC measures the model's ability to distinguish successful from failed grasps, with higher values indicating better class separation 87
6.3 Precision-Recall curve for the optimized XGBoost model. This curve shows the trade-off between precision and recall for different probability thresholds 87
6.4 Confusion matrix for the optimized XGBoost model on the test set. True Positives (TP) and True Negatives (TN) indicate correctly classified grasps, while False Positives (FP) and False Negatives (FN) indicate misclassifications 88
6.5 Confusion matrix for the threshold-optimized XGBoost model (threshold = 0.836) on the test set. This visualization highlights the effect of the optimized decision threshold in reducing unsafe grasp attempts 89
6.6 During the familiarization phase, participants could freely select objects and place them inside the semi-transparent blue cube, which indicates the target placement location 93
6.7 Initial instruction message presented to the participant, explaining the experiment 94
6.8 Illustration of a trial performed by a participant 94
6.9 Google Colab interface used to control the experimental flow 95
6.10 Pie charts depicting the distribution of participants by age group, gender, vision condition, and prior experience with AR/VR 95
6.11 Mean scores of UMUX-Lite questions comparing Blink and Dwell interaction methods 96
6.12 Mean NASA-TLX scores across all dimensions for Blink and Dwell interaction methods 97
6.13 Box plot of objective task completion times (in seconds) for the Blink and Dwell interaction methods across Simple and Complex scenes 98
6.14 Scatter plot illustrating the correlation between objective task completion time and subjective perceived ease (SEQ). The red trendline indicates a strong, statistically significant negative correlation 100

6.15 Subjective evaluation results for the complete system. The chart presents the Usability Metric for User Experience (UMUX-Lite) and Single Ease Question (SEQ) scores reported by participants following the pick-and-place tasks in both the simple and complex scenes 102

List of Tables

5.1 Fields stored for each grasp sample in the ML dataset. 63
5.2 Feature Overview for Grasp Success Prediction 67
5.3 Obstacle State Change Thresholds 74
5.4 Axis Conventions: Unity vs. ROS 77

6.1 Random Forest Model Evaluation Results 84
6.2 XGBoost Model Evaluation Results 84
6.3 LightGBM Model Evaluation Results 84
6.4 CatBoost Model Evaluation Results 85
6.5 Final Model Evaluation Metrics 86
6.6 Final evaluation metrics for the optimized XGBoost model using a threshold of 0.836. Emphasizing precision reduces unsafe grasp attempts while slightly decreasing recall 89
6.7 Results of 30 physical grasp trials comparing the baseline selection against the proposed ML probability ranking. The last column indicates whether the baseline and ML methods chose the same grasp 91
6.8 NASA-TLX Mean Scores for Blink and Dwell Interaction Methods 97
6.9 Average Single Ease Question (SEQ) Scores per Interaction Mode and Scene 99
6.10 Task Success Rates for Selection and Placement by Interaction Mode . . 99
6.11 Mean participant scores for the UMUX-Lite and SEQ items across the simple and complex scenes. All items were evaluated on a 7-point Likert scale 102

Acronyms

HCI	Human-Computer Interaction
HRI	Human–Robot Interaction
ROS	Robot Operating System
URDF	Unified Robot Description Format
ML	Machine Learning
ML2	Magic Leap 2
GPD	Grasp Pose Detection
AR	Augmented Reality
XR	Extended Reality
VR	Virtual Reality
MR	Mixed Reality
HMD	Head-Mounted Display
HWD	Head Worn Display
RViz	ROS Visualization
FOV	Field Of View
DOF	Degrees Of Freedom
UI	User Interface

Chapter 1

Introduction

1.1 Brief Overview

The rapid advancement of robotic manipulation systems has enabled robots to perform increasingly complex tasks in real-world environments. However, effective interaction between humans and robotic manipulators remains a significant challenge. Traditional robot control interfaces frequently rely on external monitors, handheld controllers, or complex programming procedures, requiring users to divide their attention between multiple interaction spaces. This separation introduces cognitive load, reduces situational awareness, and limits accessibility for non-expert users.

Augmented Reality (AR) provides an opportunity to rethink human–robot interaction by embedding digital information directly into the physical workspace. By presenting visual feedback aligned with real-world objects and robot actions, AR allows users to interact with robots in a spatially intuitive manner without shifting attention away from the task environment. When combined with head-mounted displays and eye-tracking technologies, AR enables interaction modalities that support hands-free operation and natural communication of user intent.

Gaze interaction is particularly promising for robotic control because it represents an implicit and continuous indication of user attention. Unlike handheld controllers or gesture-based systems, gaze-based interfaces allow users to select and manipulate objects without physical effort, making them especially suitable for assistive robotics and collaborative scenarios. Despite this potential, gaze-driven manipulation systems remain limited due to challenges in reliable object selection, grasp planning under environmental constraints, and seamless integration between user input and robotic autonomy.

Another major challenge in robotic manipulation lies in bridging high-level human intent with low-level robotic execution. While grasp pose generation algorithms can pro-

pose candidate grasps based on geometry, they often fail to account for contextual scene complexity such as nearby obstacles or cluttered environments. Consequently, improving grasp selection robustness while maintaining intuitive human interaction remains an open research problem.

Motivated by these challenges, this diploma thesis explores how augmented reality and gaze-based interaction can be combined with autonomous robotic planning to create an intuitive, hands-free framework for object manipulation tasks. The goal is to enable users to communicate high-level task intent through natural interaction, while allowing the robotic system to determine how the task should be executed safely and efficiently.

Specifically, this thesis presents a simulation-based augmented reality framework for gaze-driven control of a robotic arm, focusing on object pick-and-place tasks. The proposed system enables users to interact with a robotic manipulator through an AR interface, using gaze-based input to select objects and define placement locations. The framework aims to reduce cognitive and operational load by allowing users to communicate high-level intent while assigning low-level motion planning to the robot.

Object selection is performed through gaze-based interaction, where the user fixates on a target object and confirms selection using either a dwell-based mechanism or a blink gesture. This interaction approach eliminates the need for handheld controllers and supports hands-free operation. Once an object is selected, its corresponding point cloud is extracted and processed by an integrated Grasp Pose Detection (GPD) module. In this work, GPD is employed exclusively for the generation of 6-DOF grasp poses from point cloud data. The system produces a ranked list of candidate grasp configurations, from which the top ten are selected for further evaluation.

While GPD provides purely geometry-based grasp candidates, it does not explicitly account for scene complexity, such as nearby obstacles. To address this limitation, a machine learning–based grasp evaluation model is trained to assess the suitability of each candidate grasp within the context of the current scene. The model filters and re-ranks grasp poses by considering environmental constraints, thereby improving the likelihood of successful execution in cluttered scenarios.

For placement specification, a virtual clone of the selected object is rendered in the scene, allowing the user to manipulate its position using gaze fixation. The placement pose is then locked via a blink action, confirming the selected location.

Once both the grasp and placement poses have been defined, the relevant information is transmitted to the Robot Operating System (ROS). which computes collision-free trajectories by including all scene objects into the planning scene and satisfying the robot's

kinematic constraints.

The resulting trajectory is then returned to the AR system, where the user receives real-time visual feedback of the robot executing the task. In cases where execution is not possible (i.e, unreachable targets or planning failures) the system provides informative feedback to the user.



Figure 1.1: Application preview

The proposed framework is evaluated in a simulated environment across manipulation scenarios with two levels of scene complexity. Experimental results demonstrate reliable gaze-based object selection, effective grasp execution, and efficient task completion. Additionally, the performance of the two gaze-based selection mechanisms (dwell and blink) was compared, showing that both approaches enable accurate object selection while offering different trade-offs in speed and user experience. The learning-based grasp evaluation contributes to improved robustness in cluttered environments, while the AR interface enables intuitive and natural user interaction.

The results indicate that gaze-based AR interfaces can serve as a powerful tool for enhancing human–robot collaboration in manipulation tasks, with potential applications in assistive robotics, industrial automation, and teleoperation.

The main contributions of this thesis are summarized as follows:

• Design and implementation of a gaze-driven AR interaction framework that enables intuitive control of a robotic manipulator for object pick-and-place tasks.

• Development of a hands-free object selection mechanism based on gaze fixation, supporting both dwell-based confirmation and blink-triggered interaction.

• Integration of gaze interaction with autonomous robotic manipulation, enabling users to specify high-level task intent while assigning motion planning and execution to the robotic system.

• Incorporation of Grasp Pose Detection (GPD) for 6-DOF grasp generation combined with a machine learning–based grasp evaluation model that considers environmental context to improve grasp suitability in cluttered scenes.

• Introduction of an AR-based placement specification method through virtual object cloning, allowing users to define placement poses directly within the augmented environment.

• Establishment of seamless communication pipeline with the Robot Operating System (ROS) for collision-aware trajectory planning incorporating scene objects and robot kinematic constraints.

• Simulation-based evaluation of gaze-driven manipulation, demonstrating reliable object selection, successful grasp execution, and effective task completion across different scene complexities.

• Comparative analysis of dwell and blink gaze interactions, highlighting usability trade-offs in speed, accuracy, and user experience.

1.2 Thesis Structure

Chapter 1: Introduction
This chapter provides an overview of the thesis, including the problem statement, objectives, and motivation behind the development. It also includes a brief description of the project's context and goals.

Chapter 2: Background/Research Overview
This chapter presents an overview of Robotics and Augmented Reality (AR), discussing their historical evolution, application domains, and relevant aspects of human–computer and human–robot interaction.

Chapter 3: Technological Background and Definitions
This chapter outlines the technological background of the thesis, including the Unity engine for AR visualization and simulation, the Magic Leap 2 device, the ROS ecosystem (ROS, MoveIt, URDF, and RViz) used for robot modeling, motion planning, and visualization, and the fundamentals of the grasp pose detection.

Chapter 4: Augmented Reality Interaction and System Workflow
This chapter describes the augmented reality interaction workflow as experienced by the user, including system initialization, object selection, and placement definition.

Chapter 5: Implementation
The most detailed chapter, this section presents the technical implementation of the gaze-based augmented reality system for robotic manipulation. It covers the AR user interface and interaction techniques, the integration of the robotic arm with ROS, the generation and evaluation of candidate grasps using the GPD framework, and how user gaze, interface interactions, grasp planning, and robotic motion are connected to provide real-time feedback within the AR environment.

Chapter 6: Evaluation
This chapter presents the evaluation and testing of the proposed system. It includes the assessment of different machine learning models for grasp selection, as well as the evaluation of gaze-based interaction methods through user studies. The results provide insights into the system's performance, usability, and overall effectiveness.

Chapter 7: Conclusion and Future Work
The final chapter summarizes the main contributions and findings of the proposed system. It reflects on the development and evaluation of the augmented reality framework for gaze-based robotic manipulation and discusses potential directions for future improvements and extensions.

Chapter 2

Background/Research Overview

2.1 Robotics

Robotics has seen remarkable advancements in recent years, driven by significant progress in artificial intelligence, machine learning, and sensor technology. These advancements have enabled robots to navigate complex environments autonomously, perform intricate manipulative tasks, and interact more intuitively with humans. Autonomous navigation algorithms now allow robots to traverse dynamic spaces with real-time sensor data, while improvements in robotic arms and hands have enhanced their dexterity, making them capable of performing delicate and precise tasks essential in both industrial and medical contexts. Additionally, the focus on human-robot interaction has led to the development of robots that are more adept at understanding and responding to human commands and behaviors. This advancement fosters more seamless and effective collaboration.

2.1.1 Brief History of Robotics

The first use of the word "Robot" dates back in 1921 and it was introduced by Karel Čapek in his play Rossum's Universal Robots. The play describes mechanical men that are built to work on the factory assembly lines and that rebel against their human masters. The etymological origin of the word Robot is from the Czech word robota, which means servitude or forced labor.

The term "Robotics" was first mentioned by the Russian-born American science-fiction writer Isaac Asimov in 1942 in his short story Runabout. Asimov had a much brighter and more optimistic opinion of the robot's role in human society compared to the view of Čapek. In his short stories, he characterized the robots as helpful servants of man. Asimov defined robotics as the science that study robots. He created the Three Laws of Robotics, that state the following:

• The First Law states that a robot should not harm a person or let a person suffer damage because of their inaction.

• Second Law states that a robot must comply with all orders that a human dictates, with the proviso that occurs if these orders were in contradiction with the First Law.

• The Third Law states that a robot must take care of its own integrity, except when this protection creates a conflict with the First or Second Law.

The history of robotics is marked by several key milestones. In 1961, the first industrial robot, the Unimate invented by George Devol, was used in manufacturing. This demonstrated the potential of robots to perform repetitive tasks with high precision. In the 1970s, Stanford University's Shakey robot became the first mobile robot capable of reasoning about its actions. This development laid the groundwork for autonomous robots and AI. Robotics technology has evolved a lot over the years. Robotic arms were brought to assembly lines in the 1980s, which improved efficiency and safety in manufacturing. The 1990s had advancements in computer vision and machine learning. This enabled robots to perform more complex tasks and adapt to different environments.

The evolution of robotics from the early 19th century to the present day represents a continuous acceleration of innovation, beginning with mechanical automatons and programmable looms, progressing through the industrial robots of the mid-20th century, and culminating in today's AI-powered autonomous systems. This chronological journey reveals how each era's technological capabilities shaped the development of increasingly sophisticated machines, transforming robots from simple mechanical devices following fixed patterns to intelligent systems capable of learning, adapting, and making complex decisions in real-world environments.

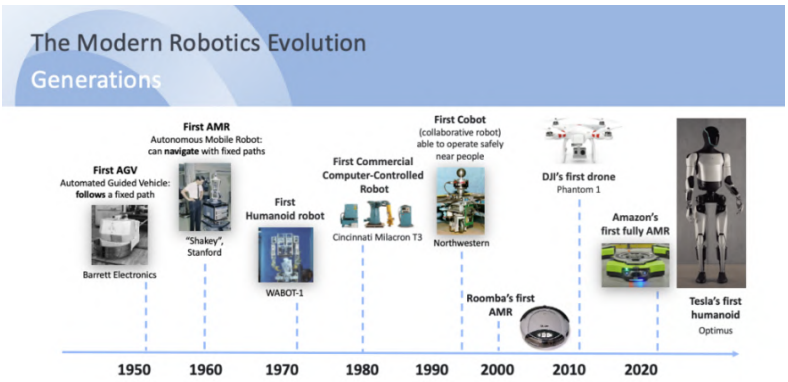


Figure 2.1: The evolution of modern robotics

2.1.2 Robots in different fields

Over the last decades, robotics evolved from fiction to reality becoming the science and technique that is involved in the design, manufacture and use of robots. Computer Science, Electrical Engineering, Mechanical Engineering and Artificial Intelligence (AI) are just some of the disciplines that are combined in robotics. The main objective of robotics is the construction of devices that perform user-defined tasks. The rapid growth of the field in scientific terms had led the development of different types of robots.

Robots are widely used across diverse fields, including healthcare, surgery, rehabilitation, telepresence, industrial automation, domestic services, collaborative and wearable systems, military and space exploration, environmental monitoring, entertainment, art, and advanced domains such as swarm, modular, and networked robotics. Robots are divided into categories: Mobile robots, service robots, industry robots and humanoid robots. The process began with industry robots that entered the world of the automotive industry at the latest in the 1970s. In the course of the years their sophisticated and continually evolving technology has formed the technological basis for other robot classes.

1. **Mobile robotics**

   Mobile robots transport or locate goods and persons or are used as technology carriers. In both cases the biggest challenge are large distances or operating ranges in environments which are not restricted or prepared. To tackle this, corresponding technological systems and methods such as autonomous navigation, mapping as well as path and motion planning are used.

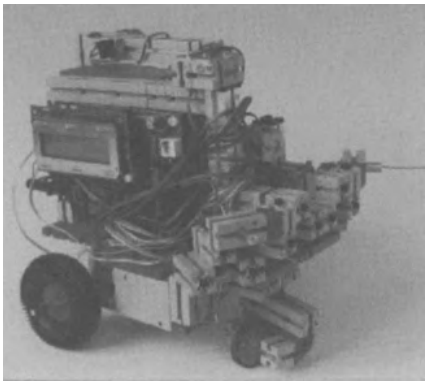


Figure 2.2: Example of the mobile robot GRASMOOR which was built at the University of Manchester

2. **Service robotics**

   The focus of service robotics lies on the performance of semi-automatic and fully automatic services for humans and facilities. Service robots are characterized by a high degree of interaction with their changeable but clearly defined environment in which they work autonomously and interact with humans.

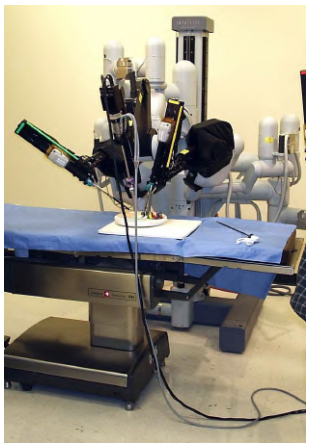


Figure 2.3: A laparoscopic robotic surgery machine. Patient-side cart of the da Vinci surgical system

3. **Industry robotics**

   Since the late 1970s, industrial robots have become an indispensable element of the manufacturing industry. No production line in the automotive industry can be imagined without them any more. Their use has contributed significantly to an increase in productivity and has turned them into an essential tool in modern industrial countries. Collaborative robots (cobots), equipped with advanced interaction capabilities, form a rapidly growing segment of industrial robotics. Their ability to cooperate among themselves, with other machines, and with humans enables a wide range of new application domains. With this collaboration focus is increasingly on the topic of "Vision". Visual perception via optical sensors gives industry robots, which are actually "blind", the capability to adjust their motion dynamically and immediately to the changed surrounding conditions and even perform challenging visual inspection tasks.

4. **Humanoid robotics**

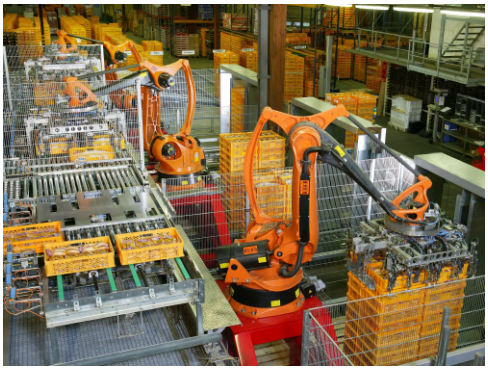


Figure 2.4: Factory automation with industrial robots for palletizing food products like bread and toast at a bakery in Germany

Humanoid robots are characterized by kinematics recreated from the human locomotor system. A big challenge is to operate multiple axes for hands, arms and legs at the same time. Walking upright is a specific task that does not occur with other robot types. Another challenge is to reproduce cognitive skills of humans. The handling effort of humanoid robots is much bigger than that of other robot types. Currently, the humanoid robots available are still far from the physical and mental performance of a human, even if developments in some functional areas are promising. It is to be expected that artificial machine intelligence will continue to develop consistently over the next years. The developments in this field will not only drastically expand the areas of application for humanoid robots but for all other robot types as well.



Figure 2.5: Valkyrie (R5), a humanoid robot from NASA

2.1.3 The Power of Simulation in Robotics

Simulation has become a fundamental tool in robotics research, supporting the design, analysis, and validation of robotic systems. It enables the study of robot structure, behavior, and performance at different levels of detail without the risks or costs associated with physical experimentation. As robotic systems grow in complexity, simulation environments provide efficient means for testing control strategies, evaluating system performance, and developing applications. Combined with visualization tools, simulation allows realistic modeling of robotic operation and is widely used in manipulator design, control development, and offline testing of robotic tasks.

In modern robotics, frameworks such as the Robot Operating System (ROS) have transformed the way simulations are conducted. ROS provides a flexible software architecture that integrates control algorithms, sensors, and actuators, while tools like Gazebo offer a physics-based simulation environment capable of modeling both the kinematics and dynamics of robots and their interactions with the surrounding environment. Visualization tools such as RViz allow for real-time observation of robot states, sensor outputs, and trajectories, while URDF (Unified Robot Description Format) files define the physical and kinematic structure of the robot, enabling consistent modeling across different tools and platforms. Together, these tools allow developers to design and test robotic behaviors safely and efficiently, experiment with control strategies, and plan complex tasks without the risks and costs associated with physical testing.

Beyond the technical capabilities, simulation provides a range of practical benefits. It reduces development time by allowing iterative testing of algorithms and hardware designs in a virtual environment, facilitates reproducible experiments, and enables the exploration of scenarios that might be too dangerous or impractical to replicate in the real world. Moreover, simulation enhances collaboration, as robot models and experiments can be shared, visualized, and analyzed across teams and locations. In essence, modern simulation environments bridge the gap between theoretical design and real-world application, accelerating innovation and ensuring that robotic systems are both robust and efficient before deployment.

In this thesis, robotics concepts are applied within a simulated manipulation framework focusing on robotic pick-and-place tasks. The work relies on robotic kinematics, grasp planning, and motion execution. Simulation enables controlled experimentation and safe evaluation of manipulation strategies without hardware constraints. A virtual robotic arm integrated with ROS is used to study task execution and trajectory planning. Thus, robotics provides the operational foundation upon which the proposed interaction framework is built.

2.2 Augmented Reality

Augmented Reality (AR) is a technology that overlays digital information onto the physical world, enabling users to interact with both real-world and virtual objects in real-time. Unlike Virtual Reality (VR), which immerses users in a completely digital environment, AR enhances the user's existing environment by adding contextual digital content. This digital content can be anything from simple 2D overlays, such as text and images, to complex 3D objects and animations that interact with real world elements. AR can be viewed as a continuum, with real-world environments at one end and completely virtual environments at the opposite end.

Different types of reality

1. **Real Environment (RE):** It is the environment in which we live and is governed by the laws of physics.
2. **Augmented Reality (AR):** Physical reality in which participants also see virtual elements.
3. **Augmented Virtuality (AV):** It is a virtual reality in which the participants also see real elements.
4. **Virtual Reality (VR):** Represents a synthetic world in which the participant is completely immersed.

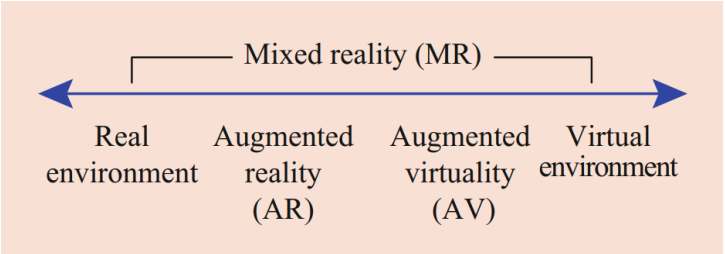


Figure 2.6: Reality–Virtuality continuum schematic

2.2.1 Brief History of AR

The first experiments to create an augmented reality occurred only in the 1957, when an American cinematographer and visionary, called Morton Heilig, described in detail his vision of a multisensory theater that could transmit images, sounds, vibrations, and smells to spectators. In 1962, he obtained the US patent and built the first prototype

of multisensory theater, together with five short films, called Sensorama, a mechanical device, very bulky, capable of projecting 3D videos on a stereoscopic color display, with stereo sounds, perfumes, wind in the hair, and vibrations, through which it simulated for the first time a motorcycle ride during which the sensory elements are activated at the right time, creating an adrenaline experience for the user. Although the Sensorama is still functional, the Heilig's project was interrupted, and it took a few more years to fully understand the potential of AR and to convince entrepreneurs, afraid of the high costs of making 3D films, to invest in this emerging technology.

It was the American researcher Ivan E. Sutherland already known for his Sketchpad application, in 1968, to create the first head mounted display system (HMD) tracked mechanically through a ceiling-mounted hardware, called The Ultimate Display, thus opening the way to the world of augmented and virtual reality. That HMD was partially see-through and consisted of two connected cathode ray tubes that projected images directly into the user's eyes allowing the superimposition of 3D images on real objects. Its structure was very similar to a medieval torture instrument, from which it had the nickname "The sword of Damocles".



Figure 2.7: Configuration of the "Sword of Damocles"

Although the concept of augmented reality (AR) was explored for over two decades, the term augmented reality became official only in the 1990s when it appeared in a document written by two Boeing engineers and researchers, Thomas P. Caudell and David W. Mizell, which described an AR system for reducing errors when creating wirings on aircraft and other aerospace industry systems. Boeing was one of the first companies to use the augmented and virtual reality as part of its commercial strategies.

In the early 2000s, AR began to make its way into the consumer market, primarily through mobile AR applications. With the proliferation of smartphones equipped with cameras and GPS, developers could create AR experiences that used the phone's display

to superimpose digital content onto the physical world. One of the earliest AR tools, ARToolKit, enabled developers to create marker-based AR applications, which could track specific visual markers and overlay digital images in real time. The introduction of Layar in 2009 marked a significant step in mobile AR, as it allowed users to view real-world environments through their smartphone's camera while receiving contextual information overlaid on the display. Around this time, the gaming industry also began exploring AR with titles like Ingress, an AR-based game that utilized real-world locations as part of the gameplay. Afterwards, there was an explosion of devices for AR and mixed reality that allowed industries to design and develop increasingly complex AR applications. Nowadays, an AR system typically consists of several key components. General hardware includes a processor, a display, sensors, and input devices. Displays can be specialized, such as head-mounted displays (HMDs), heads-up displays (HUDs), virtual retinal displays, spatial augmented reality (SAR) systems, or the screens of mobile devices. Sensors and input devices may include GPS modules, gyroscopes, accelerometers, compasses, RFID readers, wireless sensors, as well as tactile and voice recognition interfaces. Finally, the system relies on software to integrate these components, process data, and render augmented content in real time.

2.2.2 Applications of AR

Augmented reality encompasses a wide range of application domains, most of which have emerged in recent years. The areas most affected by this new technology include many industrial sectors, entertainment, training, health, forensic, and military applications, among others.

Entertainment and Gaming

One of the most well-known uses of AR is in the entertainment and gaming industries. AR allows players to interact with virtual characters and objects within the real world, creating immersive and engaging experiences. A notable example of AR in gaming is Pokémon GO (2016), which allowed players to explore their physical environment while capturing virtual creatures superimposed on real-world locations. This game popularized AR on a global scale, demonstrating the technology's potential to merge digital and physical worlds for enhanced player experiences.

Beyond gaming, AR is also being used in theme parks, interactive movies, and live events to create more engaging and immersive entertainment experiences. As AR technology continues to evolve, its applications in the entertainment industry are expected to grow, with new opportunities for interactive storytelling and personalized content.

Education

Displaying information by using virtual things that the user cannot directly detect with his own senses can enable a person to interact with the real world in ways never before possible. The position, shape, and/or other graphical features of virtual objects can be modified with interaction techniques augmented reality supports.

For example, in medical education, numerous studies have used augmented reality (AR) to provide training, plan and guide surgical procedures and teach anatomy, employing features such as AR image-guided therapy, interactive 3D anatomy models and haptic feedback.

Industry

The term IAR can be defined as the application of AR in order to support an industrial process. One of the IAR most common applications is the assistance to workers in maintenance/repair/control tasks through instructions with textual, visual, or auditory information. Figure 2.8 summarizes the most relevant industrial tasks and sectors where IAR can bring value.

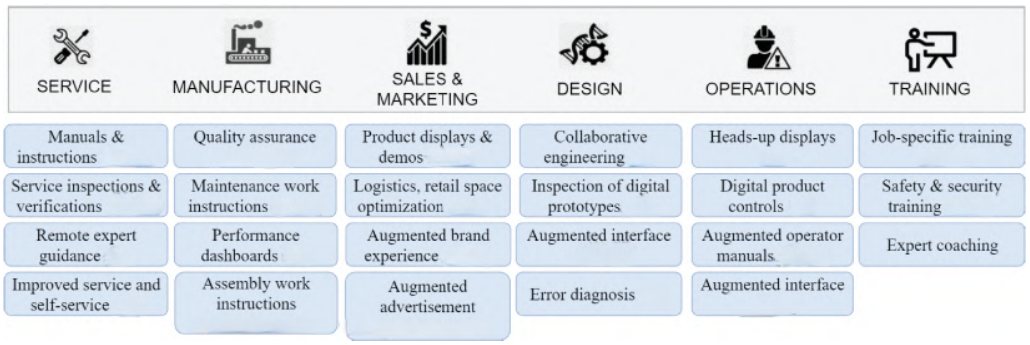


Figure 2.8: Value of IAR across industry

In this thesis, augmented reality is used as the main interface for interacting with the robotic system. The AR environment visualizes virtual objects, robot states, and interaction feedback within simulation. This allows the user to specify manipulation tasks through spatial interaction rather than traditional input devices. In this way, AR supports intuitive communication between the user and the robotic manipulation framework.

2.3 Human-Computer Interaction (HCI) in AR

The field of Human-Computer Interaction (HCI) focuses on the design and evaluation of computer technology and the interactions between users and systems. As VR and AR technologies gain prominence, the principles of HCI must be adapted to meet the demands of these immersive environments. Unlike traditional graphical user interfaces, VR and AR offer users the ability to engage with virtual objects in 3D space, challenging HCI researchers to innovate new ways to facilitate interaction.

In VR and AR, input devices play a crucial role in facilitating human-computer interactions. These devices allow users to interact with immersive environments through actions such as selection, manipulation, and navigation. The three primary interaction modalities in head-worn AR systems are hand gestures, speech, and gaze, each of which presents distinct advantages and limitations.

1. **Hand gesture**
   Gesture-based interaction allows users to select, manipulate, and control virtual objects by performing gestures such as pinching, tapping, or dragging, while maintaining situational awareness of the real environment, which is particularly important in industrial, medical, and assistive applications. By using built-in sensors, head-worn displays (HWDs) can detect and interpret hand and head gestures in real time. However, hand gesture–based interaction presents notable limitations, including sensitivity to occlusion and the potential for arm fatigue during long-term use.

2. **Speech**
   Speech-based interaction enables the user to control the device through verbal commands. Voice commands are another key method of interaction in AR systems, particularly in environments where hands-free operation is essential. The speech-based system offers better controllability, however, it requires user to remember and pronounce the verbal command correctly. It also increases user's cognitive workload, especially for complex tasks.

3. **Gaze**
   Gaze tracking is a highly intuitive method of interaction that uses the user's eye movements to control the system. Eye gaze-based interaction also requires less physical effort, and provides more natural experience than gesture or speech , which is potentially an effective channel in wearable HMD systems. However, the gaze-based system often suffers from the Midas Touch problem, where users unintentionally trigger a target with every glance. The insufficient eye tracking accuracy also degrades the user experience.

To tackle the limitations of single-modal interaction systems and enrich user experience in AR, multimodal interactive systems have become a recent research focus. These systems usually combine two modalities at a time, i.e., Gesture+Speech, Gesture+Gaze, Gaze+Speech etc.

In this thesis, human–computer interaction principles guide the design of a gaze-driven interaction interface. Gaze dwell and blink actions are employed to enable hands-free object selection and placement specification. This interaction approach is particularly suited for assistive manipulation scenarios where traditional input devices are impractical. The proposed interface evaluates how gaze-based interaction can support efficient and intuitive robotic control within augmented reality.

2.4 Human-Robot Interaction (HRI) using AR

Human-robot interaction (HRI) is a multidisciplinary field focused on the study and design of interactions and communication between humans and robots, distinct from human-computer interaction (HCI), which primarily addresses interactions between humans and computers or digital devices. Traditional robot interaction relies primarily on the robot's built-in physical and visual feedback mechanisms, such as movement, gestures, gaze cues, physical transformations, lights, or small displays. However, these modalities are inherently limited by the robot's fixed physical form, which restricts the expressiveness and adaptability of feedback. In particular, visual outputs such as lights or displays are constrained by size and cannot easily convey information related to the surrounding physical space. Augmented reality (AR) offers a promising alternative by enabling flexible and expressive visual feedback that is not bound to the robot's physical design. By presenting information directly within the user's field of view and spatial context, AR can reduce cognitive load and improve interaction efficiency. Figure 2.9 demonstrates different examples of augmented reality and robotics research.

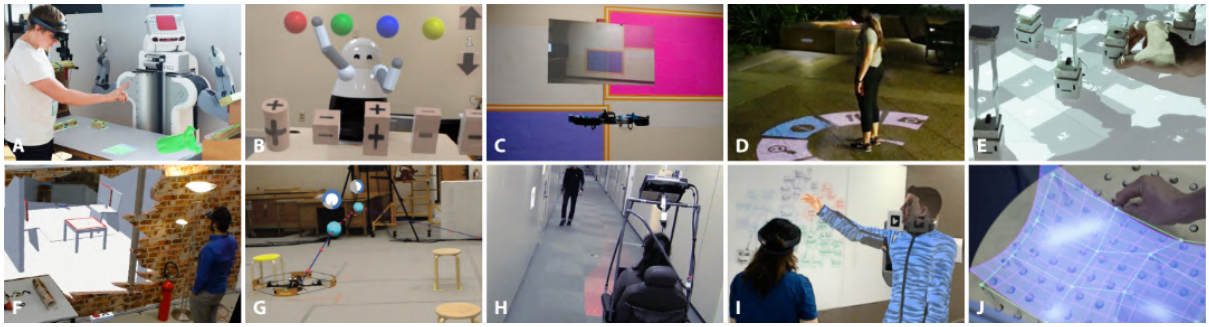


Figure 2.9: Examples of augmented reality and robotics research: A) collaborative programming, B) augmented arms for social robots, C) drone teleoperation, D) a drone-mounted projector, E) projected background for data physicalization, F) drone navigation for invisible areas, G) trajectory visualization, H) motion intent for pedestrian, I) a holographic avatar for telepresence robots, J) surface augmentation for shape displays

In this thesis, human–robot interaction is explored through a simulation-based augmented reality framework for robotic manipulation tasks. Users interact with virtual objects and a simulated robotic arm through a gaze-based interface within the AR environment. This approach allows users to intuitively select objects and define manipulation goals while observing the robot's actions in real time. By integrating augmented reality visualization with gaze-driven interaction, the system demonstrates how AR can serve as an intuitive interface for controlling robotic systems.

Chapter 3

Technological Background and Definitions

This chapter presents the technological background relevant to this thesis. It introduces the Unity game engine as the primary simulation and visualization platform and discusses the key features used for augmented reality interaction and robotic integration. The chapter further describes the Magic Leap 2 head-worn display and its Software Development Kits. Additionally, the chapter introduces the Robot Operating System (ROS) ecosystem, including MoveIt, URDF, and RViz, which are used for motion planning, robot modeling, and visualization. Finally, the foundations of grasp pose detection and learning-based grasp evaluation are outlined to provide context for the proposed manipulation pipeline.

3.1 Unity Game Engine

Unity is a popular cross-platform game engine that enables developers to create 2D, 3D, virtual reality, and augmented reality games and interactive experiences. It provides a wide range of tools and features to help developers create games, including an integrated development environment (IDE), physics engine, animation system, scripting tools, and a vast library of assets and resources.



Figure 3.1: Unity

Unity consists of several key components, including a rendering engine, which is respon-

sible for depicting visuals in real time, and a physics engine, which governs the behavior of objects according to the laws of physics.

Unity is widely used for augmented reality application development due to its real-time 3D rendering capabilities and flexible component-based architecture. In AR systems, Unity serves as the application layer responsible for visualizing virtual content, handling user interaction, and maintaining spatial alignment between digital objects and the physical environment. Its support for XR standards such as OpenXR, along with device-specific SDKs, allows developers to deploy AR applications across different hardware platforms while maintaining a consistent development workflow. These characteristics make Unity a suitable environment for prototyping and implementing interactive AR interfaces that integrate perception, input, and visualization in real time.

In this thesis, Unity was used as the primary platform for developing the AR application, including the interfaces that handle the user's gaze input and the simulation of the entire environment.

3.2 Magic Leap 2 (ML2)

ML2 is an enterprise-focused AR headset developed and manufactured by Magic Leap Inc.. Released on September 30, 2022, it serves as the successor to the company's earlier Magic Leap 1 device. ML2 represents a significant advancement in AR technology, designed primarily for professional and industrial applications rather than consumer use. The device focuses on providing a more immersive augmented reality experience with improved comfort for extended use, enhanced visual capabilities, and powerful computing performance.

ML2 features a lightweight, ergonomic design optimized for all-day comfort in professional environments. Unlike many competitors, it utilizes a split-system architecture where the computing and battery units are housed in a separate Compute Pack that connects to the headset via a thin cable. This design choice shifts weight away from the user's head, reducing strain during extended use sessions. The headset itself weighs just 260 grams, while the Compute Pack weighs 425 grams and can be attached to a pocket or worn with a strap. The device runs on Lumin OS, a custom operating system based on the Android Open Source Project (AOSP) Android 10. This foundation provides compatibility with existing Android development tools while incorporating Magic Leap's specialized features for augmented reality.



Figure 3.2: Magic Leap 2

ML2 utilizes proprietary Liquid crystal on silicon (LCoS) display technology with advanced waveguide optics developed and manufactured in-house. This technology allows digital content to be projected into the user's field of view while maintaining visibility of the real world. The device features:

- **Resolution:** 1440 × 1760 pixels per eye
- **Field of View:** 45° horizontal × 55° vertical (approximately 70° diagonal)
- **Refresh Rate:** 120 Hz
- **Color Depth:** Support for over 16.8 million colors
- **Brightness:** Adjustable from 20 to 2,000 nits

Beyond the controller, that uses optical tracking rather than the electromagnetic field tracking of its predecessor, the device supports multiple input methods such as **hand tracking** for controller-free interaction, **eye tracking** for gaze-based selection, **voice** commands and support for **Bluetooth keyboards and other peripheral devices**.

3.2.1 Magic Leap SDK (MLSDK)

Magic Leap provides the Magic Leap Software Development Kit (MLSDK), which offers device-specific tools, APIs, and services required for developing applications on the ML2 platform. The MLSDK includes support for core AR functionalities such as spatial mapping, hand and eye tracking, input handling, and access to device sensors, enabling developers to build immersive and interactive mixed reality experiences. While earlier Magic Leap applications relied primarily on proprietary MLSDK interfaces, the platform is progressively transitioning toward the OpenXR standard. This shift aims to improve cross-platform compatibility, reduce dependency on device-specific APIs, and align Magic Leap development with industry-wide XR standards, allowing applications to be more portable and future-proof.

3.2.2 OpenXR

OpenXR is an open, royalty-free standard developed by the Khronos Group that defines a unified API for building augmented, virtual, and mixed reality applications across diverse hardware platforms. It provides a common interface for core XR functionality such as tracking, rendering, and input handling, reducing the need for device-specific code and enabling developers to target multiple XR devices with the same underlying codebase. ML2 is compatible with OpenXR, and the platform's SDK support reflects a strategic transition away from MLSDK components toward an OpenXR-based workflow. As part of this evolution, Magic Leap's Unity support has shifted from the legacy Magic Leap XR plugin to the standard Unity OpenXR plugin, and many of the previously MLSDK-specific APIs are being re-exposed through OpenXR features and extensions that can be enabled in project settings. This allows developers to access device-specific capabilities within the OpenXR framework while maintaining greater cross-platform compatibility and reducing development overhead for multiple XR targets.

In this thesis, the Magic Leap 2 head-mounted device was used as the target augmented reality platform. Application development was carried out in Unity using both the MLSDK and OpenXR for different purposes. The MLSDK was employed for marker understanding, while OpenXR provided eye and hand tracking data, managed the calibration process, and handled device permissions.

3.3 Robot Operating System (ROS)

Robot Operating System, or ROS, is an open-source software framework for building scalable robotic applications. It is a collection of tools, libraries, and conventions that aim to simplify the task of making complex and robust robots. It provides a framework to create and connect components like actuators, sensors and control systems using ROS tools called topics and messages. This framework uses a distributed architecture, with the ROS computation graph at its heart. A ROS distribution is a versioned set of ROS packages. The ROS distributions are fully compatible with Ubuntu, and most of the ROS distributions are planned according to their respective Ubuntu versions. Figure 3.3 illustrates some of the most used ROS distributions and their compatibility with Ubuntu.

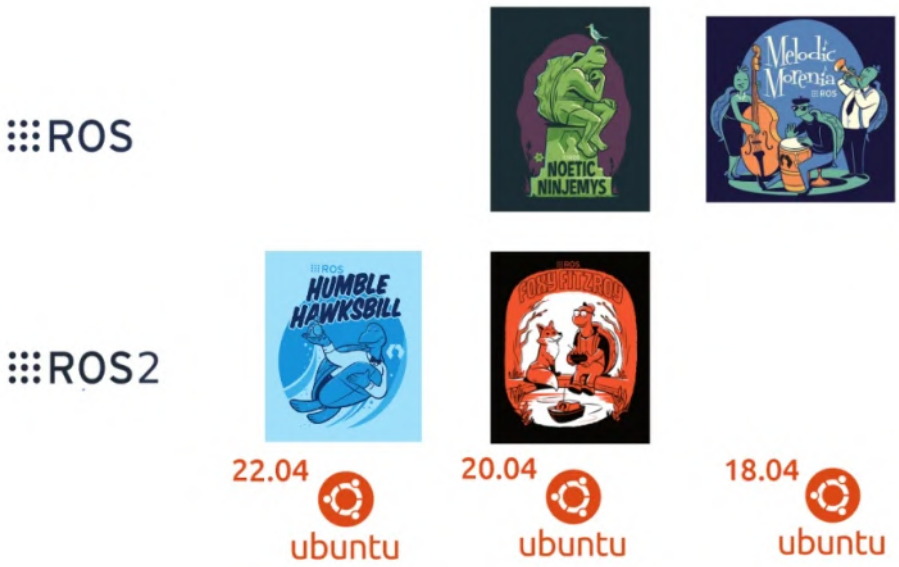


Figure 3.3: ROS distributions and their compatibility with Ubuntu versions

3.3.1 Unified Robot Description Format (URDF)

URDF, or Unified Robot Description Format, is an XML-based markup language developed as part of the Robot Operating System (ROS) ecosystem. It represents a robot model in terms of its physical structure, kinematics, inertial properties, visual appearance, collision geometry, and sensor placement. URDF files define robots as collections of links connected by joints, specifying joint limits, coordinate frames, and sensor frames, which enables accurate simulation, visualization, and motion planning. In simulation environments like Unity, URDF ensures that physical constraints and visual representations

match real-world expectations, allowing developers to test algorithms safely before deploying them on hardware. Motion planning frameworks such as MoveIt rely on URDF models for inverse and forward kinematics computations, while visualization tools like RViz render 3D robot models overlaid with sensor data to facilitate debugging and understanding of system behavior.

3.3.2 MoveIt Motion Planning Framework

MoveIt is an open source robotics manipulation platform for developing algorithms in robotics, prototyping robotic systems designs, benchmarking algorithms and commercial applications. The platform is developed and maintained in the sope of an open source project that is the result of the combined efforts of a large international community and multiple organizations. MoveIt runs on top of ROS and builds on the ROS messaging and build systems and utilizes some of the common tools in ROS like the ROS Visualizer (RViz) and the URDF. The components of MoveIt provide various functionalities for robotics manipulation and integration with ROS which makes the repository of the platform quite extensive.



Figure 3.4: MoveIt motion planning framework for ROS

MoveIt for ROS provides a comprehensive framework for robotic manipulation by integrating advanced motion planning, kinematic computation, and execution control within a unified architecture. It supports widely used sampling-based planning algorithms, such as Rapidly-exploring Random Trees (RRT) and Probabilistic Roadmaps (PRM), enabling the generation of safe and efficient trajectories for complex manipulators. The framework incorporates continuous collision checking to ensure that planned motions remain free of self-collisions and environmental obstacles. Additionally, it includes robust inverse kinematics solvers that compute feasible joint configurations for desired end-effector poses, facilitating precise task execution. MoveIt provides interfaces to ROS controllers for reliable trajectory execution on both simulated and physical robots. It also includes visualization and debugging tools that support efficient analysis and monitoring of motion plans.

3.3.3 ROS Visualization (RViz)

RViz is a 3D visualization tool within the ROS ecosystem that enables real-time monitoring of a robot's perception and behavior in a unified interface. It visualizes sensor data such as camera streams, LiDAR point clouds, and depth measurements, while also displaying robot models, joint states, and planned trajectories to track motion and interaction with the environment. RViz supports navigation and mapping tasks by presenting maps, localization outputs, and path planning data. It is widely used for debugging, algorithm validation, and system testing, allowing developers to analyze perception results and robot behavior in simulation before deployment on physical hardware.

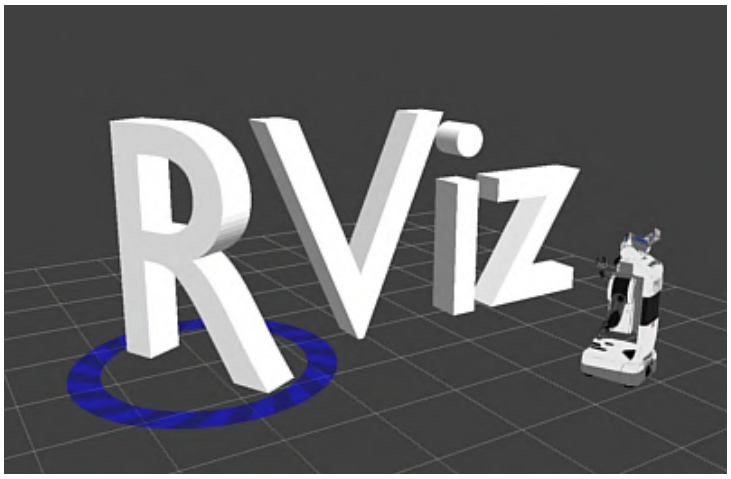


Figure 3.5: RViz: a ROS graphical interface

In this thesis, ROS Melodic running on Ubuntu 18.04 was employed for the development and control of the robotic system. The robot's URDF model, provided by Unity, was subsequently imported and utilized within the ROS environment. The integration between Unity and ROS was achieved through the ROS–TCP Connector and the URDF Importer packages, enabling consistent kinematic representation and bidirectional communication between the two platforms.

For motion planning, the MoveIt framework was utilized. MoveIt generated collision-free trajectories while continuously performing obstacle avoidance through planning scene collision checking.

RViz was used as a visualization and debugging tool to validate the system configuration. It allowed verification of the robot model, confirmation that obstacles were correctly represented in the planning scene, and inspection of the generated trajectories prior to execution.

3.4 Grasp Pose Detection (GPD)

Robotic manipulation, defined as the ability of a robot to physically interact with and modify its environment, is one of the most challenging problems in autonomous robotics. These challenges arise from the need to integrate accurate perception, effective motion planning, and precise control in dynamic and often unstructured environments. Grasping is a fundamental aspect of manipulation, enabling the robot to establish control over an object's motion for tasks such as relocation. A grasp pose specifies the position and orientation of the robot's end-effector relative to the object at the moment of contact. Grasp representations are commonly distinguished by their degrees of freedom (DOF). A 4-DOF grasp encodes the end-effector pose using a three-dimensional position (x,y,z) and a single rotational component (yaw) about a predefined approach vector, typically aligned with the direction of gravity. This configuration, often referred to as top-down grasping, simplifies grasp planning by constraining the orientation space. However, it is limited in environments where top-down access is obstructed or otherwise infeasible. In contrast, a 6-DOF grasp fully specifies both position and orientation of the end-effector, removing constraints on the approach vector. This additional flexibility enables the robot to grasp objects from arbitrary directions, making it more suitable for complex or cluttered environments. The trade-off is increased computational and perceptual complexity, as both grasp generation and motion planning must consider a larger configuration space.

In this thesis, a package called Grasp Pose Detection (GPD) was used to detect 6-DOF grasp poses (3-DOF position and 3-DOF orientation) for a 2-finger robot hand (e.g., a parallel jaw gripper) in 3D point clouds. GPD takes a point cloud as input and produces pose estimates of viable grasps as output.

3.5 Machine Learning (ML)

Machine learning is the subset of artificial intelligence (AI) focused on algorithms that can "learn" the patterns of training data and, subsequently, make accurate inferences about new data. This pattern recognition ability enables machine learning models to make decisions or predictions without explicit, hard-coded instructions. Some of the main types of machine learning algorithms are Supervised Machine Learning, Unsupervised Machine Learning, and Reinforcement Learning. Additionally, there is a more specific category called Semi-Supervised Learning and Self-Supervised Learning, which combines elements of both supervised and unsupervised learning.

In machine learning, supervised learning is a type of machine learning paradigm where an algorithm learns to map input data to a specific output based on example input-output pairs. This process involves training a statistical model using labeled data, meaning each

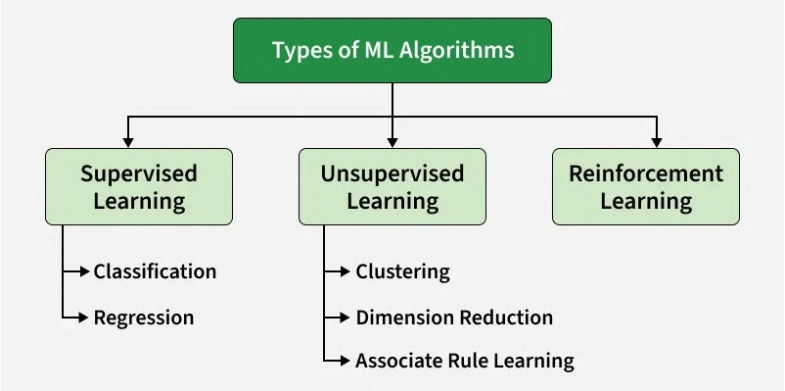


Figure 3.6: Machine learning algorithms

piece of input data is provided with the correct output. The goal of supervised learning is for the trained model to accurately predict the output for new, unseen data.

Classification and regression are fundamental supervised machine learning techniques. The former aims to assign data points to predefined categories or classes, such as distinguishing between spam and non-spam emails or between diseased and healthy patients, while the latter focuses on predicting continuous numerical values by modeling the relationships between input features and a target variable.

Binary Classification is the simplest kind of classification. In binary classification, the goal is to sort the data into two distinct categories.

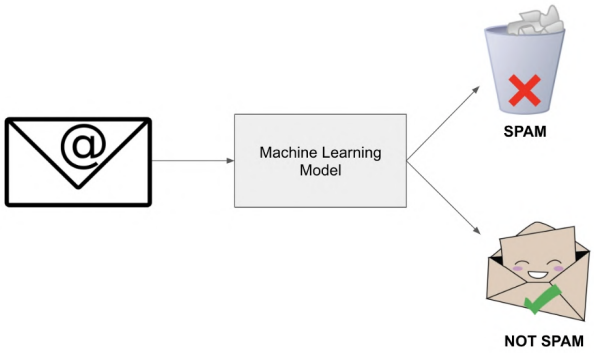


Figure 3.7: Example of binary classification

In this thesis, we employed binary classification to evaluate the feasibility of grasps generated by the GPD algorithm, determining whether each grasp can be successfully executed by the robotic arm within its operational environment.

Chapter 4

Augmented Reality Interaction and System Workflow

This chapter presents the augmented reality interfaces and the overall interaction workflow as experienced by the user. It describes how the system is initialized, how the user specifies the object to be grasped, and how the desired placement location is defined. The focus of this chapter is on the visual elements and interaction mechanisms that shape the user experience within the environment.

4.1 System Initialization and Environment Setup

Upon launching the application, the user has to insert the IP addresses of the two following systems:

1. **ROS:** This IP address is responsible for controlling the robotic arm.
2. **GPD:** This IP address performs the grasp pose detection.

The GPD package and its dependencies (e.g., Eigen) were compiled with advanced vector instruction sets such as AVX2, AVX512, and SSE4.2. However, the virtual machine's CPU did not support these instructions, resulting in runtime execution failures. To resolve this issue, GPD was deployed within a WSL (Windows Subsystem for Linux) Ubuntu 18.04 environment. Consequently, this setup operated on a different IP address than the ROS environment running on the host Ubuntu 18.04 system.

This interface was implemented to allow dynamic configuration of IP addresses, thereby eliminating the need to rebuild and redeploy the application whenever network parameters change.

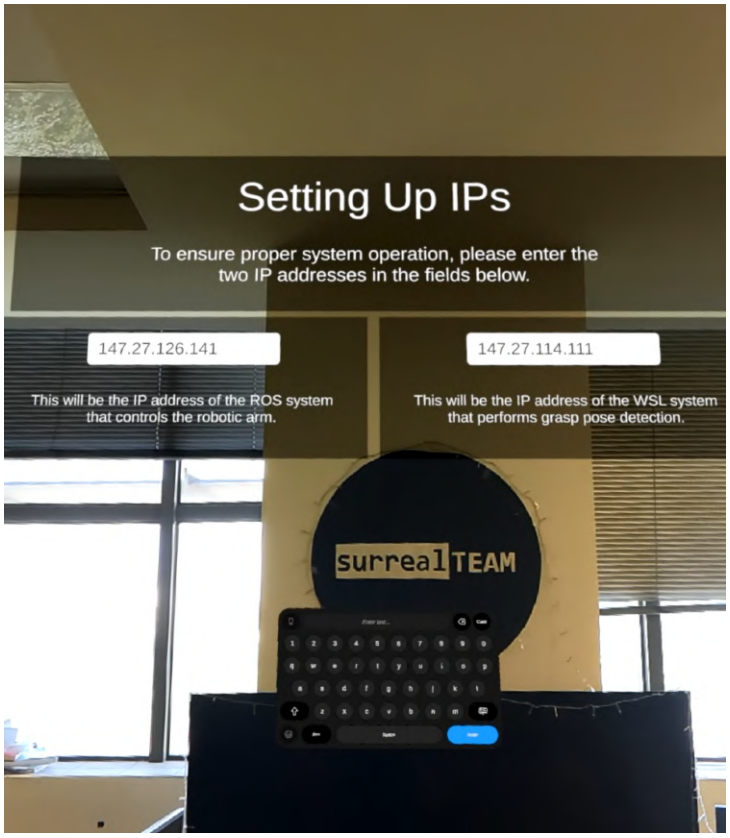


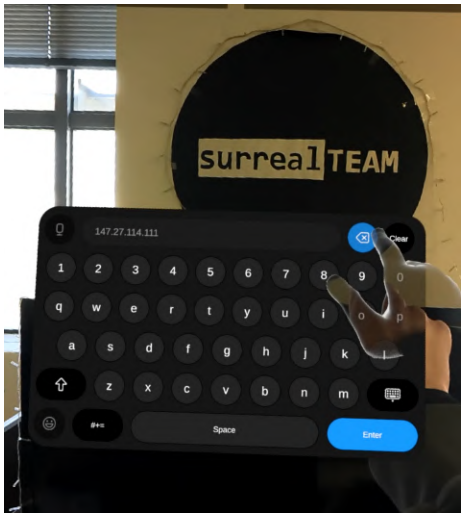
Figure 4.1: Network configuration interface

The interaction here, is via hand tracking provided by the XR Hands package which defines an API that allows us to access hand tracking data from ML2. The user can interact with two ways:

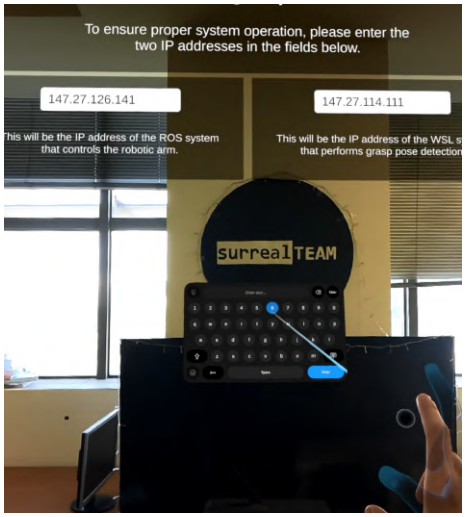
1. **Tap:** The tap gesture is used for interacting with interface elements located within a short distance from the user. It functions similarly to pressing a nearby virtual button and enables direct activation of UI components.

2. **Pinch:** The pinch gesture is used for interacting with interface elements positioned at greater distances. A visual ray is projected from the user's hand to indicate the pointing direction. The user can activate a virtual button by aligning the ray with the desired interface element and completing the pinch gesture.

In both interaction modes, the targeted UI element is highlighted in blue to provide visual selection feedback, indicating the element that will be activated upon gesture completion.



(a) Keyboard interaction using tap gesture



(b) Keyboard interaction using pinch gesture

Figure 4.2: Gesture-based interaction methods for the virtual keyboard

Upon entering a valid IP address, the corresponding input interface element is automatically removed, indicating that the address has been successfully registered. This behavior is illustrated in the screenshot below.

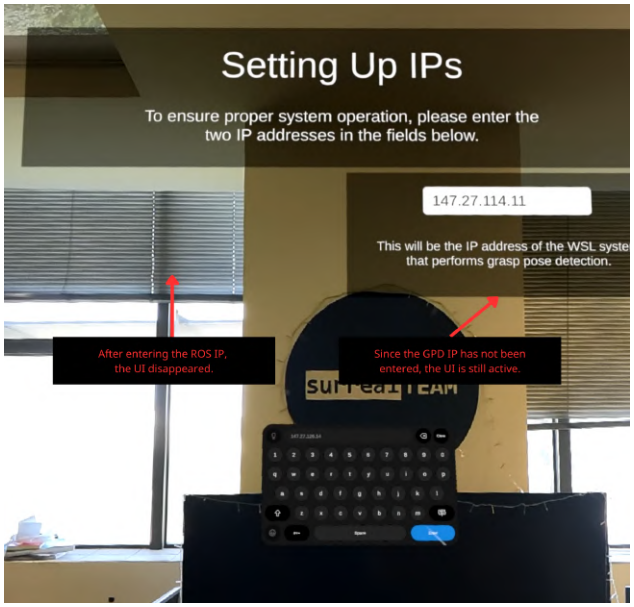


Figure 4.3: Automatic removal of input field after ROS IP address entry

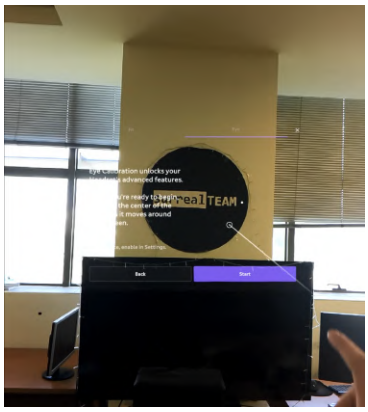
Once both IP addresses have been entered, the corresponding interface elements are removed, indicating that the system is properly configured and connected to both ROS and GPD. Subsequently, the system verifies the user's eye calibration status. The Eye Calibration API returns one of three possible states:

• Fine – Eye tracking accuracy is sufficient for precise interactions such as button or icon selection.

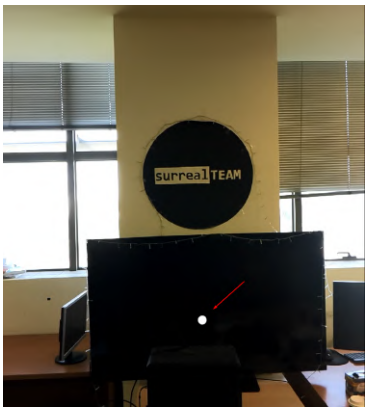
• Coarse – Eye tracking is functional for basic interactions (e.g., scrolling) but lacks the precision required for selection tasks.

• None – Eye calibration has not been performed or was not completed successfully.

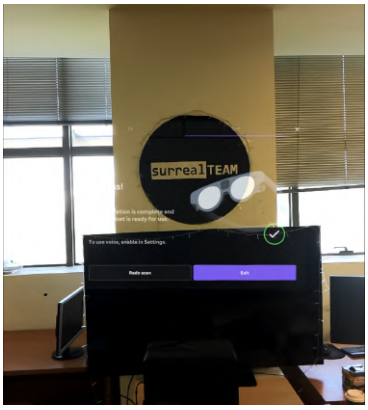
If the reported state is not Fine, the fitting intent is automatically triggered to assist the user in completing the required eye calibration and headset adjustment process.



(a) Custom Fit application launch



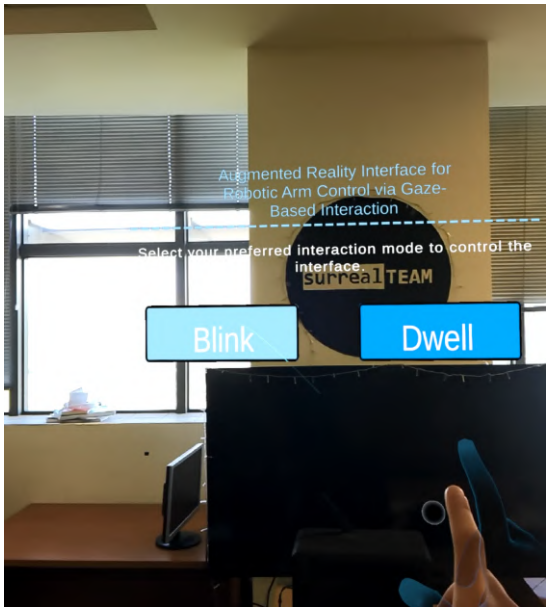
(b) Eye calibration via visual target tracking



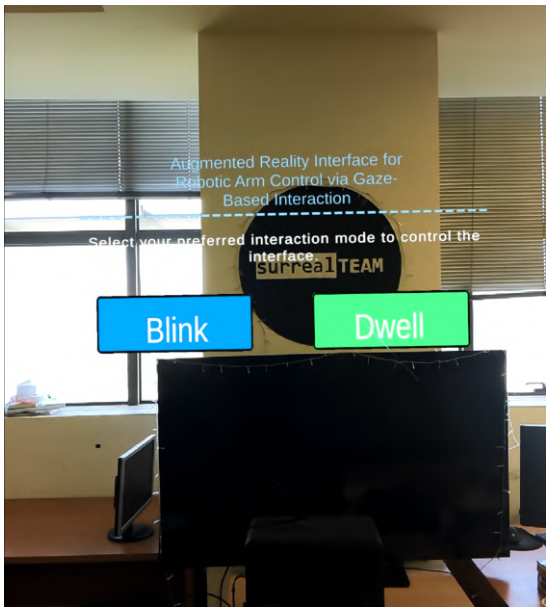
(c) Calibration process completed

Figure 4.4: Custom Fit application and calibration process

After calibration is successfully completed and the eye calibration API indicates a valid result, a different user interface is presented to allow the user to select their gaze-based interaction method. The available options are blink and dwell, and at this stage, the user still interacts using hand tracking to make the selection. The interface provides explicit visual feedback through color transitions. When a button is targeted, it changes to a lighter blue to indicate the hover state. Upon activation, the button transitions to light green, signaling successful selection.



(a) The user hovers over the "Blink" button



(b) The user pressed the "Dwell" button

Figure 4.5: Selection of gaze-based interaction method

Once the interaction method is chosen, the current interface disappears, and a new interface appears for scene difficulty selection. The options are as follows:

Simple: Consists of one self and three objects.

Complex: Composed of two selves, one table, and six items.

The user now interacts using the previously selected gaze-based method (blink or dwell) to choose the desired scene difficulty. Selection is performed by maintaining gaze on the desired button for 0.8 seconds, either by dwelling or continuously blinking. Blinking can be performed with either the left or right eye. When the user's gaze is over a button, it changes to a lighter blue color. During the selection process, a green progress line appears along the button's border, starting after 0.3 seconds to avoid flickering caused by unintentional blinks or brief dwells, and visually indicates the activation progress until the button is fully selected.

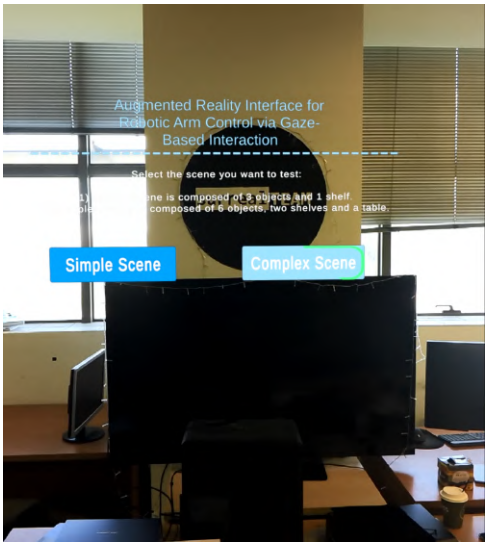


Figure 4.6: Scene selection via gaze-based interaction

Once the user has selected their preferred scene, the application is fully configured and ready to proceed with spawning the scene. The activity diagram of the overall system initialization is presented below.

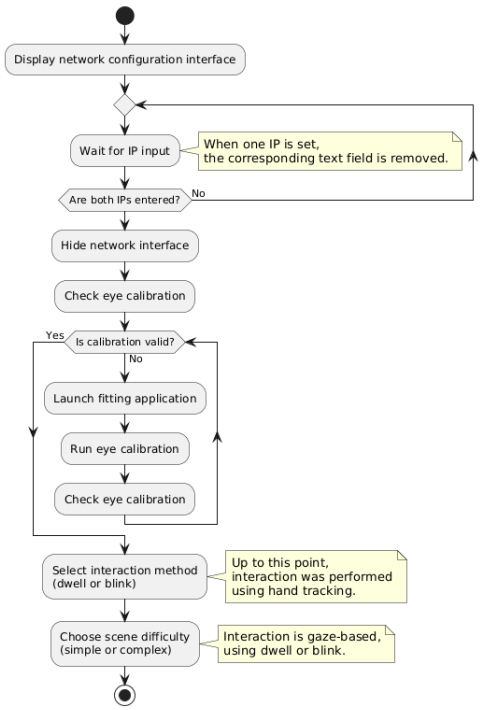


Figure 4.7: Activity diagram of system initialization

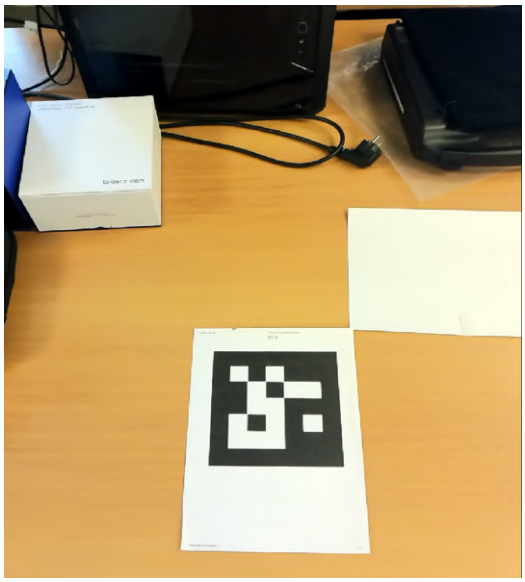
4.2 Marker-Based Scene Instantiation

Once the system has been fully configured, the selected scene can be instantiated through the marker-tracking mechanism.

Marker-based AR relies on predefined visual markers to anchor digital content within the physical environment. These markers are high-contrast patterns that can be reliably detected by an AR system using a device's camera. Once recognized, a marker serves as a spatial reference point for accurately aligning virtual elements with the real world.

The process involves real-time image capture and continuous analysis using computer vision algorithms to detect predefined markers. After detection, the system estimates the marker's pose allowing the virtual content to be rendered with correct scale, rotation, and perspective. As a result, digital objects remain consistently aligned with the physical marker, creating a stable and coherent augmented reality experience.

In this thesis, an ArUco marker was generated using an online ArUco marker generator to anchor and place the virtual scene on the physical table surface. Following system initialization, the user directs the device's camera towards the marker, after which the virtual scene is automatically detected and rendered directly above it in real time.



(a) The user directs the device's camera towards the marker



(b) Selected scene is instantiated above the marker

Figure 4.8: Scene instantiation process

4.3 Scene Navigation and Object Selection

After successful instantiation, the user can observe the virtual scene anchored to the physical table surface. When the user directs their gaze towards a virtual object, a user interface (UI) element appears above that object, providing the option to select it.

Interaction is gaze-based, in accordance with the selection method defined during the system initialization process. When the user shifts their gaze to a different object, the previously displayed UI element disappears and a new one appears above the currently observed object.

The activation of the "Select" button is performed using the same interaction mechanism that was employed to choose the scene complexity during the initialization phase.



Figure 4.9: Interaction with the "Select" button

When the "Select" button is activated, a secondary user interface appears, presenting two options: "Grab" and "Cancel." The Grab option finalizes the user's selection and indicates that the chosen object is the target for manipulation by the robotic arm.

Conversely, selecting Cancel aborts the current action and returns the user to the initial interaction phase, allowing them to continue exploring the scene and select a different object for manipulation.



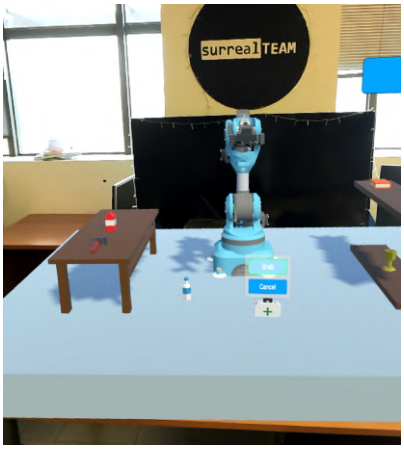
(a) User selects the blue bottle



(b) User cancels the current selection



(c) User selects the first-aid kit



(d) Confirmation of the selected object

Figure 4.10: Target object selection workflow for robotic manipulation

In the illustrated scenario, the user's interaction flow can be described as follows:

1. The user initially fixates on the blue bottle, indicating the intention to select it as the target object for manipulation by the robotic arm.
2. Before confirming the action, the user reconsiders and cancels the pending selection, thereby aborting the manipulation command.
3. The user subsequently shifts their attention to another object, namely the first aid kit, thereby redefining the intended target for manipulation.
4. Finally, the user confirms and locks the selection by activating the "Grab" command, which issues the corresponding execution request to the robotic arm.

This sequence demonstrates the system's support for dynamic intent revision, enabling the user to safely modify or cancel selections prior to command confirmation.

The flowchart below illustrates the overall workflow for scene instantiation and object selection.

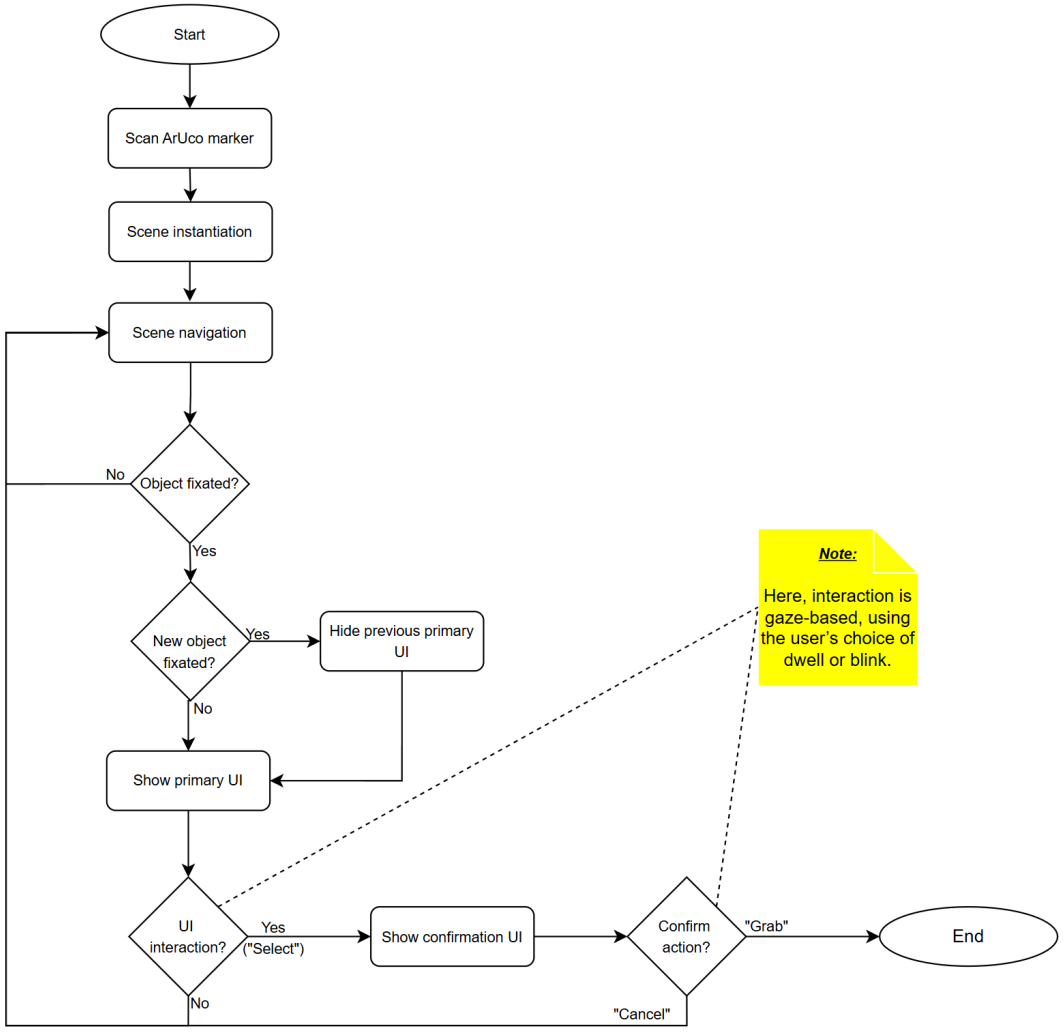


Figure 4.11: Flowchart of scene instantiation and object selection

4.4 Determining Placement Location

After selecting an object for manipulation, the user proceeds to define the desired placement location. At this stage, a virtual clone of the selected object appears within the scene. This clone represents a preview of where the robot arm will place the object.

The virtual clone follows the user's gaze, allowing the user to intuitively explore potential placement positions by simply looking at different areas within the environment. As the user shifts their visual focus, the clone moves accordingly, continuously updating its position to reflect the current gaze direction.

Initially, the clone is displayed in orange, indicating that its position is not yet confirmed. In this state, the object remains in a preview mode and continues to follow the user's gaze. The orange color indicates that the placement is still adjustable and has not been finalized.

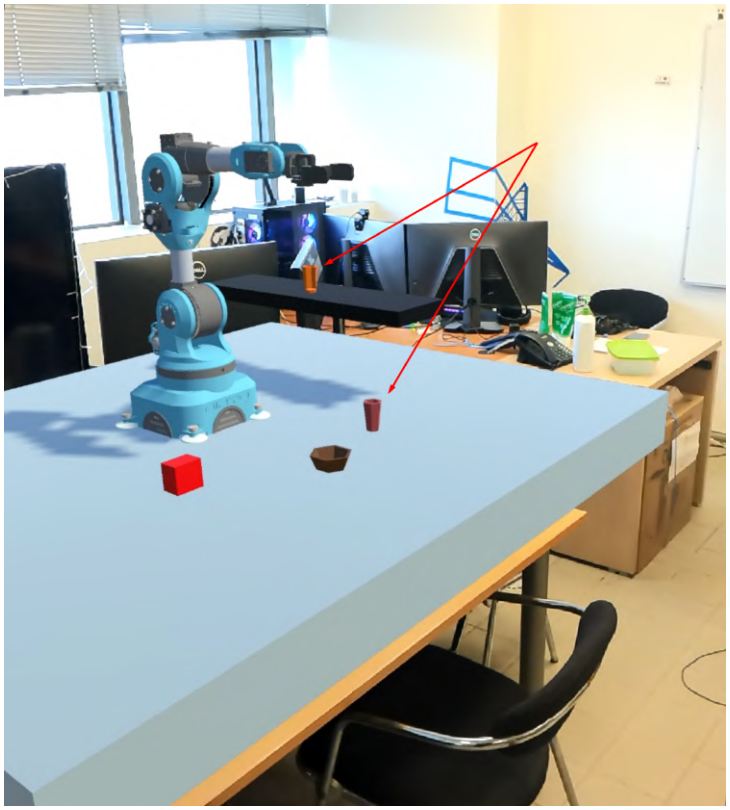
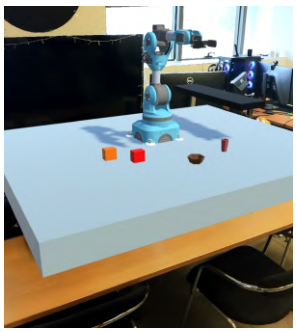


Figure 4.12: Virtual clone instantiation

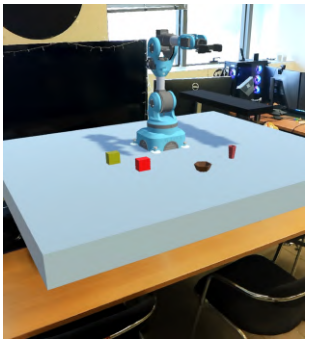
The placement position can be locked by maintaining a continuous blink for 0.8 seconds. During this process, the user receives real-time visual feedback: the virtual clone gradually transitions in color from orange to green, indicating that the locking action is being recognized. Once the clone has fully transitioned to green, the placement location is considered locked, and a confirmation user interface appears above the clone.

To confirm the selected location, the user interacts using the gaze-based input method defined during scene initialization (i.e., blink-based or dwell-based interaction). This additional confirmation step ensures that the placement is intentional.

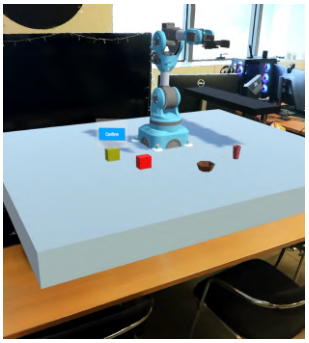
If the user decides to modify the selected location, they can unlock the placement using the same interaction method employed for locking. During the unlocking process, the clone transitions from green back to its initial orange color, visually indicating that the placement is no longer fixed and can be repositioned.



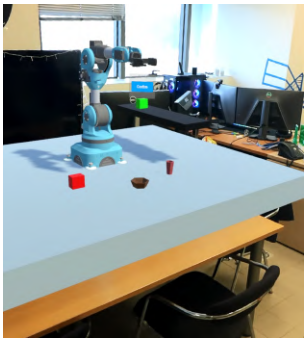
(a) Virtual cube clone instantiation



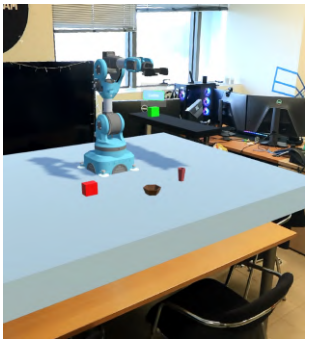
(b) Placement locking



(c) Placement unlocking



(d) New placement selection



(e) Final placement confirmation

Figure 4.13: Placement location selection process

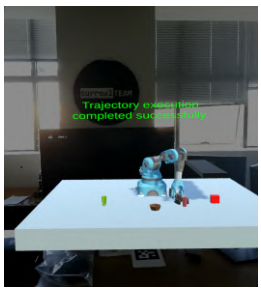
4.5 Robot Arm Trajectory Execution

Once both the target object and the target location have been determined by the user, the robot arm is capable of executing the trajectory. Trajectory execution is enhanced with an additional user interface that logs the results. The output can be categorized into four types:

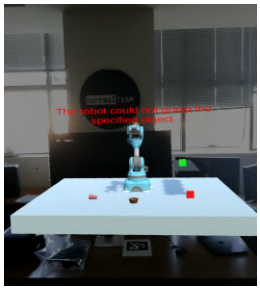
1. **Planning:** Indicates that the trajectory is currently being planned.
2. **Success:** Indicates that the trajectory execution was completed successfully.
3. **Failure in grasping:** Indicates that the robot was unable to grasp the specified object.
4. **Failure in placement:** Indicates that the robot was unable to reach the target location.



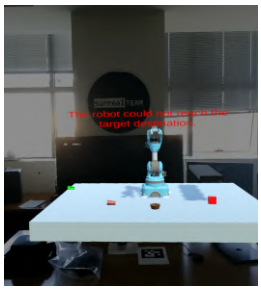
(a) Trajectory planning in progress



(b) Trajectory successfully executed



(c) Failed to reach the target object



(d) Failed to reach the target location

Figure 4.14: Comparison of robot arm trajectory execution results under different conditions

4.6 Reset Functionality and UI Behavior

To prevent situations in which objects become unreachable by the robotic arm, an additional user interface mechanism was implemented. This interface allows the user to reset objects to their predefined initial positions, thereby restoring the workspace to a valid and reachable configuration.

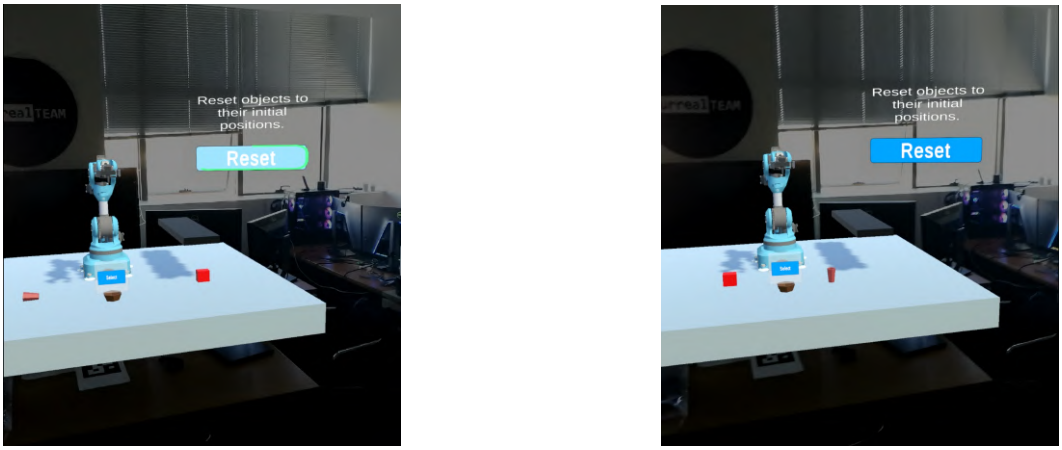


Figure 4.15: Before and after pressing the "Reset" button

Moreover, all user interface elements dynamically adjust their orientation to continuously face the user. While their spatial position remains fixed in the environment, their rotation is updated in real time to maintain visibility and ensure consistent usability as the user moves.



Figure 4.16: User interface orientation adapting to different user positions

Chapter 5

Implementation

This chapter presents the technical implementation of the augmented reality system introduced in the previous chapter. While Chapter 4 described the system from the user's perspective, focusing on interaction workflow and task execution, the present chapter examines the internal architecture, development methodology, and software integration that enabled the proposed gaze-based robotic manipulation framework.

The system was implemented in Unity, targeting the Magic Leap 2 platform, and combines augmented reality interaction, robotic control, machine learning–based grasp evaluation, and real-time communication within a unified framework. The objective of this chapter is to explain how user intentions captured through gaze interaction are translated into executable robotic actions.

The chapter begins with the design and implementation of the augmented reality user interface, describing how spatial UI elements, interaction techniques, and visual feedback mechanisms were developed in Unity. Early prototypes used hand-tracking interactions on the Magic Leap 2 device, but the system later shifted to gaze-based control—the main interaction modality of the proposed framework. The user interface was designed to support gaze selection and hands-free operation, introducing visual cues that assist users in targeting objects and confirming actions.

Following the interface implementation, the integration of the robotic arm is presented. The robotic model was imported using URDF descriptions and connected to the ROS framework, enabling bidirectional communication between the AR application and the robotic manipulation pipeline. This section discusses coordinate transformations, message exchange, motion command generation, and synchronization between the virtual representation and the robotic execution environment.

A dedicated section describes grasp generation using the Grasp Pose Detection (GPD) framework. GPD processes object point clouds to produce candidate grasp poses, forming the basis for autonomous manipulation. Since grasp feasibility depends on both local geometry and scene context, a machine learning module was integrated to evaluate and rank generated grasps according to environmental constraints and task suitability.

Finally, the chapter then explains how user intent, grasp planning, motion planning, and visualization are interconnected through a complete pipeline. This includes communication with motion planning tools, trajectory validation, and real-time feedback visualization within the AR environment.

Through this technical analysis, the chapter provides a comprehensive understanding of how augmented reality, robotics, and machine learning technologies were combined to implement an intuitive gaze-driven interface for robotic arm manipulation.

5.1 System Architecture

The system is designed to enable intuitive augmented reality (AR) interaction for robotic manipulation tasks by integrating gaze-based object selection, intelligent grasp planning, and robotic control. The architecture consists of several interconnected components that collaboratively manage virtual scene interaction, generate manipulation strategies, and execute robotic actions through the AR interface.

Overview of the Architecture

As shown in Figure 5.1, the architecture is centered around the interaction between the AR application running on the Magic Leap 2 device, the Task Planning System, and the Robotics Control System. These subsystems exchange information continuously to support virtual object interaction, grasp planning, and real-time robot execution.

• **Augmented Reality System:** The AR system provides a Unity-based interface running on the Magic Leap 2 device, responsible for gaze interaction and scene visualization. Through the AR interface, users interact with virtual objects rendered within the augmented environment. Object models and scene elements are visualized in the AR space, allowing the user to select targets using gaze-based interaction. Once a virtual object is selected, its corresponding object point cloud representation is transmitted to the task planning module for grasp pose generation (Step 1).

• **Task Planning System:** The task planning subsystem performs grasp pose generation using point cloud processing techniques. From the received object representation,

multiple candidate grasp configurations are produced (Step 2). These candidates are evaluated by a machine learning–based grasp evaluation network that predicts grasp success. The best grasp pose found, represented by position and orientation parameters, is then returned to the AR system (Step 3–4).

• **Robotics Control System:** The robotics control subsystem manages motion planning and execution through ROS-based robot control. Following grasp pose generation, the user selects the desired placement position within the AR interface. The AR system then transmits both the pick pose and the user-defined place pose to the robotic arm (Step 5). The robotic system performs motion planning and executes the manipulation task accordingly. During execution, trajectory feedback is continuously transmitted back to the AR interface (Step 6), enabling real-time visualization and monitoring of the robot's motion.

The diagram below illustrates the overall system architecture and the flow of information between the AR interface, grasp planning framework, and robotic control module.

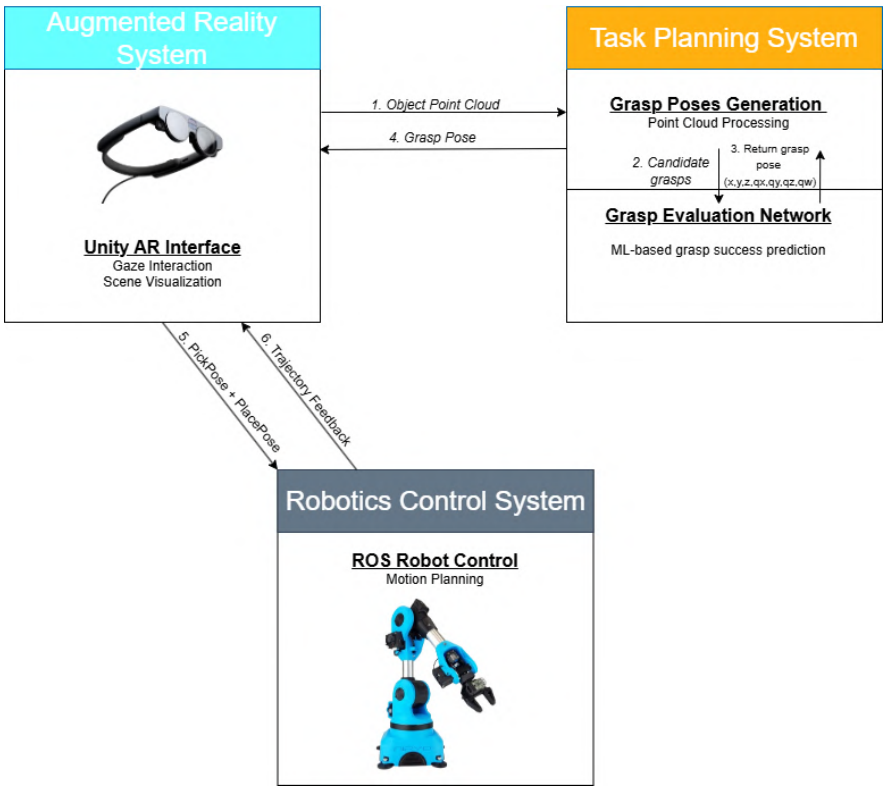


Figure 5.1: System Architecture

This architecture establishes a closed interaction loop between user interaction, intelligent task planning, and robotic execution, enabling gaze-driven manipulation within an augmented reality environment.

5.2 Unity Initial Setup

To prepare the Unity project for building and deploying OpenXR applications on the Magic Leap 2, the Magic Leap Setup Tool was first downloaded from the Unity Asset Store. This setup tool streamlines the configuration process by guiding the user through all necessary project settings. The setup begins by configuring the Magic Leap SDK. Using the *Locate SDK* option, the folder containing the latest version of the Magic Leap Unity SDK is selected. Once the SDK path is specified, the *Apply All* button can be used to automatically adjust all required project settings, ensuring compatibility with Magic Leap 2 development. After confirming all prompts and restarting the Unity editor, the project is fully configured and ready for Magic Leap 2 development.

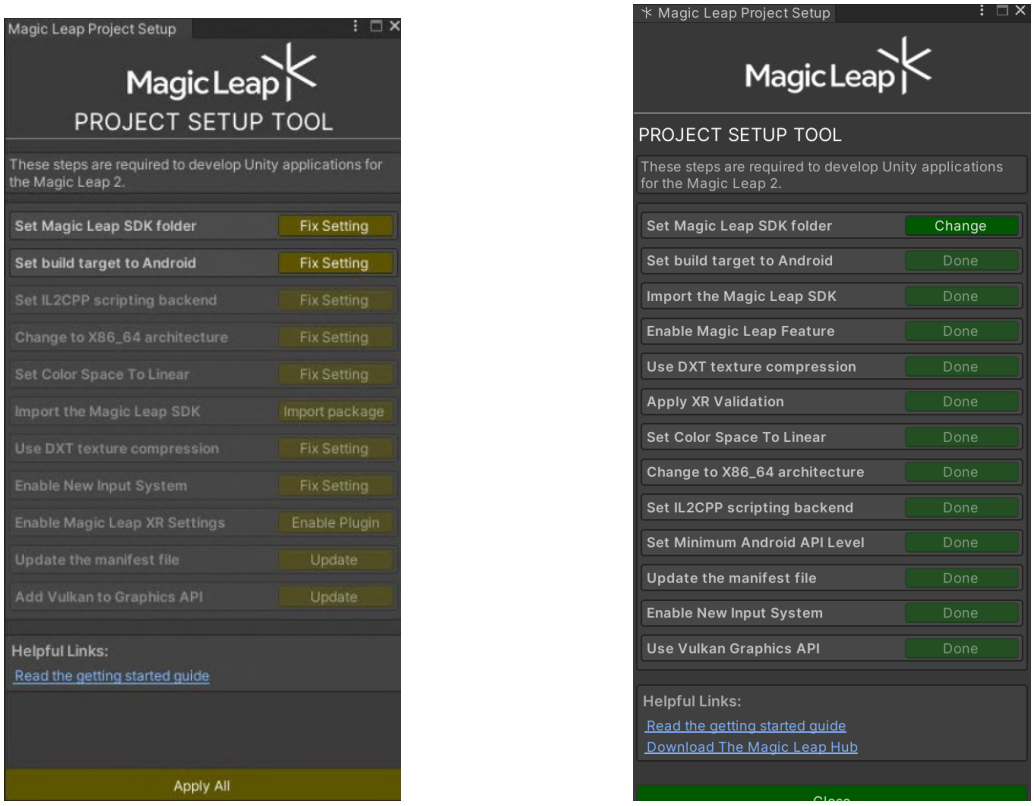


Figure 5.2: Configuration of the Unity project using the Magic Leap Setup Tool: (Left) before applying the setup, (Right) after applying the setup

To enable communication with ROS, the *ROS-TCP Connector* package was added to the project. This package allows Unity to send and receive messages to and from ROS, enabling seamless integration of robotic control within the Unity environment. Additionally, the *URDF Importer* was included to import robots defined in the Unified Robot Description Format (URDF) into a Unity scene. URDF files describe a robot's structure, including geometry, visual meshes, kinematic chains, and dynamic properties. The URDF Importer parses these files and generates corresponding Unity objects using PhysX 4.0 articulation bodies, preserving the robot's physical and kinematic characteristics for accurate simulation and interaction.

Unity provides the URDF file for the Niryo One robot along with essential ROS configuration files for basic motion planning, which greatly facilitated the integration between Unity and ROS and enabled rapid setup and testing of robotic interactions within the virtual environment. Furthermore, the physics solver type was set to *Temporal Gauss-Seidel* to prevent erratic joint behavior caused by the default solver, and the *Default Contact Offset* was reduced to 10^{-5} to ensure precise collision detection.

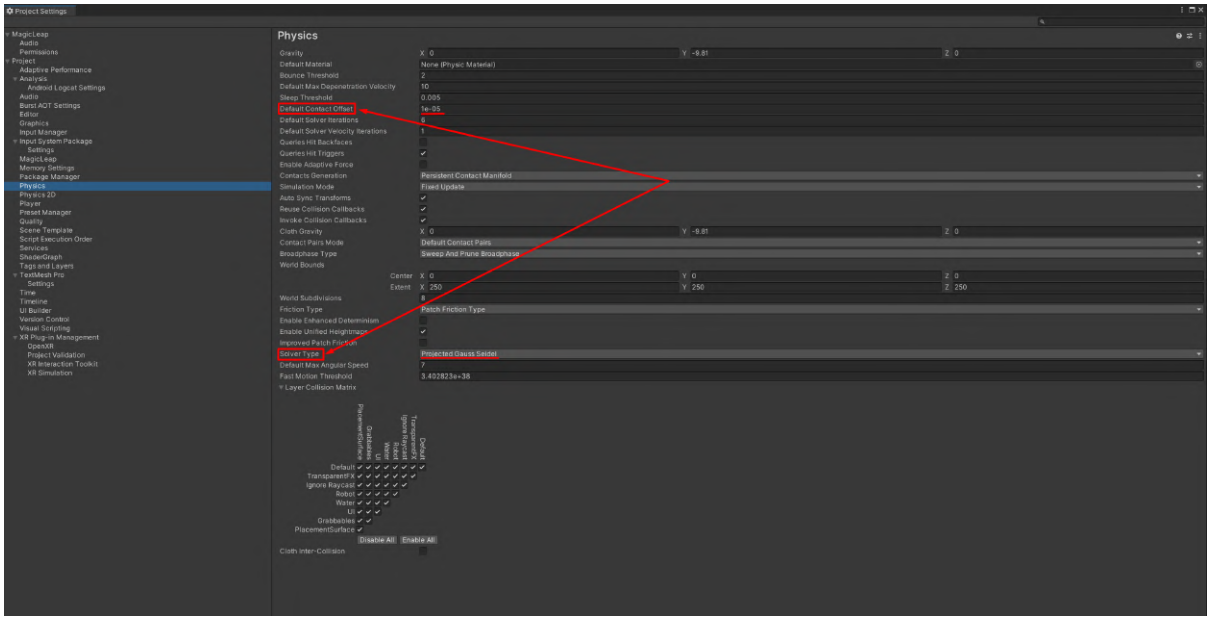


Figure 5.3: Modifications to the Unity Physics Settings for improved robot simulation

5.3 Niryo One Robotic Arm

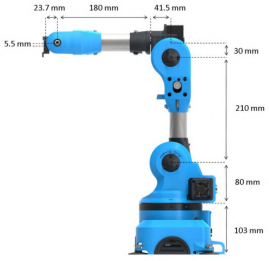
Niryo One is a 6-axis robotic arm, providing six degrees of freedom. The robot can be decomposed into seven main parts, as shown in Figure 5.4.

1 - Base	Orange
2 - Shoulder	Yellow
3 - Arm	Green
4 - Elbow	Turquoise
5 - Forearm	Blue
6 - Wrist	Purple
7 - Hand	Red



Figure 5.4: Components of the Niryo One robotic arm (without gripper)

Figure 5.5 illustrates the physical dimensions of each part and the maximum allowable rotation angles for each joint.



	Min	Max
J1	-175°	175°
J2	-90°	36.7°
J3	-80°	90°
J4	-175°	175°
J5	-100°	110°
J6	-147.5°	147.5°

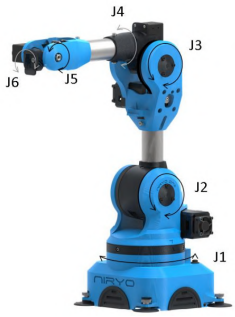


Figure 5.5: Niryo One robotic arm parameters: (Left) part dimensions (mm), (Right) maximum joint angles (degrees)

In our setup, a gripper is attached to the Niryo One to enable manipulation tasks. Its specifications are shown in Figure 5.6.

Weight	70 g
Length (gripper closed)	80 mm
Max opening width	27 mm



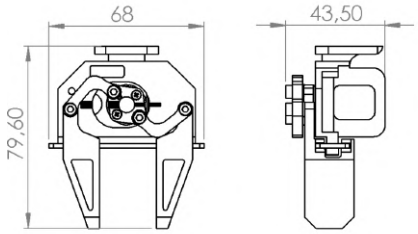


Figure 5.6: Niryo One gripper: physical dimensions

All these information is gathered and structured within the URDF file of the robot and was imported into the Unity Scene as shown below.

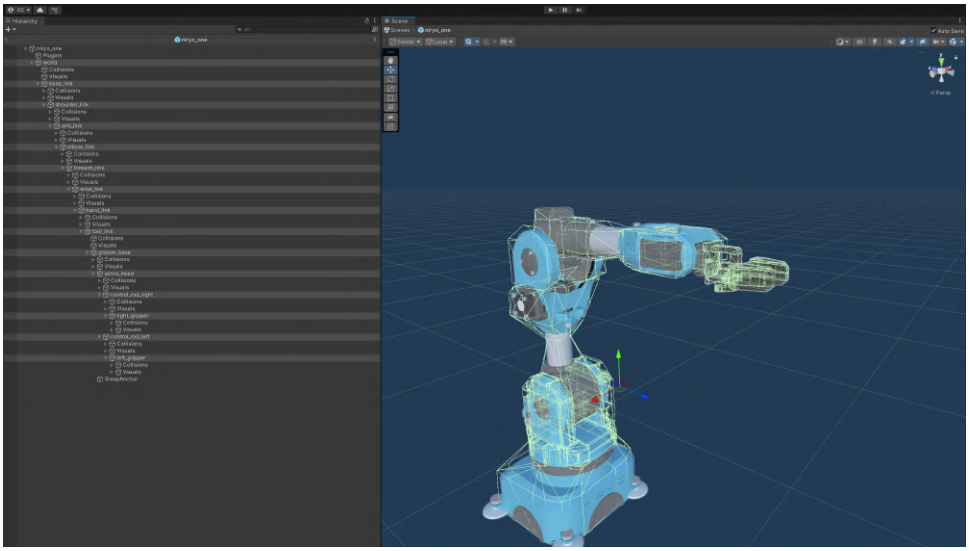


Figure 5.7: Niryo One imported in Unity

5.4 User Interface (UI)

The design of the user interface (UI) for the augmented reality application was guided by the need to ensure system reusability, configurability, and reliable interaction within a gaze-based control environment. The primary design objective was to provide users with flexible system configuration while maintaining interaction safety and minimizing unintended actions. Key design considerations included:

• **System Reusability and Configuration:** The UI was structured to allow dynamic system configuration without modifying source code. Users can define network parameters (e.g., IP addresses), select operational scenes, and select interaction modules directly through the interface. This approach allows the application to be reused without having to rebuild the app each time.

• **Prevention of Misinteractions in Gaze-Based AR:** Because gaze interaction can easily trigger unintended inputs, additional UI safeguards were introduced. Confirmation panels, interaction delays, and secondary UI layers were designed to reduce accidental selections. Critical commands require explicit user validation, ensuring safe operation when controlling physical or simulated robotic systems.

• **Clarity and Cognitive Load Reduction:** To avoid overwhelming the user's field of view, supplementary UI elements were displayed only when required. Context-aware panels and minimal persistent overlays were used to preserve environmental awareness while maintaining access to essential controls.

Early testing revealed that direct gaze-based interaction without structured UI separation increased the likelihood of accidental activations. Additionally, interface elements were distributed across multiple spatial locations, requiring users to frequently redirect their visual attention within the augmented environment. This increased cognitive load and disrupted interaction continuity, negatively impacting usability and task efficiency.

Auxiliary configuration menus and interaction control mechanisms were introduced to group related controls within consistent UI regions instead of distributing panels across the scene. By limiting unnecessary gaze movements and organizing interactions more consistently, the final design reduced cognitive effort, improved interaction clarity, and enhanced overall system reliability.

5.4.1 UI Components and Layout

While Chapter 4 provides a detailed presentation of the user interfaces, this section describes their implementation, operational behavior, and the interaction workflow associated with UI button interactions.

For the configuration of network IP addresses, the XR Interaction Toolkit was imported in order to utilize the Spatial Keyboard prefab for immersive text input. The World Keyboard (Spatial Keyboard), provided by Unity's XR Interaction Toolkit (version 2.6.3), enables text entry within extended reality environments through a fully interactive world-space virtual keyboard. Positioned relative to the XR Origin or the user's viewpoint, the keyboard supports natural interaction using controller rays, direct hand interaction, or other XR interaction modalities.

Key interactions are performed through selectable virtual keys that trigger predefined commands such as character insertion, Shift and Caps Lock toggling, Backspace and Delete operations, text clearing, submission via the Enter key, and switching between alternative layouts (e.g., symbol keyboards) defined through the `XRKeyboardConfig` layout system. The `XRKeyboardDisplay` component establishes the connection between the spatial keyboard and UI input elements, such as `TMP_InputField` components, enabling real-time synchronization of typed text, enforcement of character limits, and optional behaviors including automatic clearing upon keyboard activation or submission.

The keyboard architecture follows a modular and extensible design through the use of `KeyFunction` ScriptableObjects, which encapsulate individual key behaviors and allow custom functionality without modifying the core keyboard logic. Integrated with the XR Interaction Toolkit interaction framework, the system operates seamlessly with ray-based and direct interactors, providing consistent feedback within three-dimensional user interfaces.

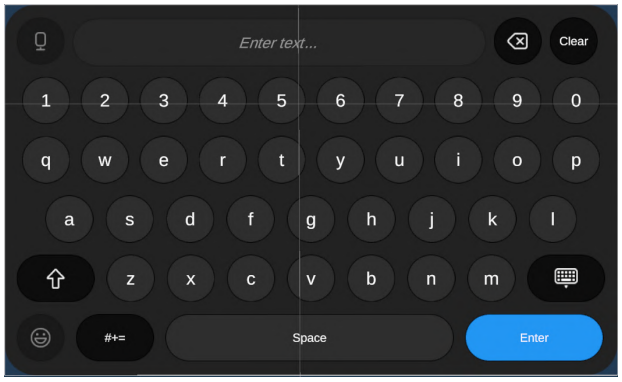


Figure 5.8: XRI Spatial Keyboard prefab

The input fields are configured to receive text input from the world-space keyboard positioned relative to the user interface panels. A character limit of 15 is enforced, corresponding to the maximum length of a valid IPv4 address (ranging from 0.0.0.0 to 255.255.255.255). The configuration procedure follows a guided workflow in which setup panels are progressively completed and subsequently hidden, allowing the user to advance through the initialization process in a structured manner. After all required connection parameters are provided, the application automatically transitions from the configuration interface to the calibration stage by disabling the setup canvas and activating the calibration environment, a process specifically handled on the Magic Leap device. To support gaze-based interaction and scene selection, interface panels are instantiated directly in front of the user by referencing the device's camera position and projecting a fixed offset along its forward direction. The panels continuously rotate to face the user, adjusting their horizontal orientation based on the relative position of the camera while maintaining a fixed vertical alignment to prevent tilting.

Next, the system implements canvases that appear above manipulable objects within the XR environment. While these canvases maintain the same orientation logic to face the user, their position must adapt dynamically to the location and rotation of the associated object. To achieve this, the `MeshRenderer` component of each object is accessed to determine the vertical extent of the object by computing the maximum y-coordinate of its bounding box. The panel position is then calculated relative to the object's center, combining its x and z coordinates with the adjusted y-coordinate to place the canvas just above the object. Specifically, the panel is positioned at:

$$\begin{aligned} x &= \text{targetObject.position.x}, \\ y &= \text{objectTopY} + 0.5 \times \text{panelHeight}, \\ z &= \text{targetObject.position.z} \end{aligned}$$

The x, y, and z coordinates refer to the center of the panel. Figure 5.9 illustrates the variables involved and the method used for this calculation. The same logic is applied to cloned instances of virtual objects: whenever a clone is instantiated or moved within the scene, its associated canvas is automatically repositioned above it and oriented to face the user, ensuring consistent visibility and interaction for both original and duplicated objects.

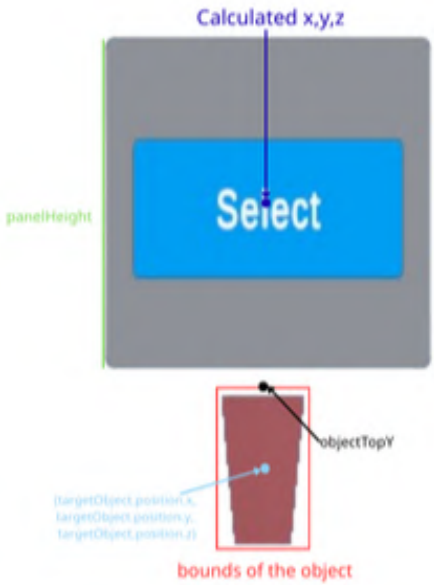


Figure 5.9: World-space canvas positioned above a manipulable object

5.4.2 Interaction Methods

Interaction within the application is supported through both hand tracking and gaze tracking. Hand tracking is primarily employed during the initial setup phase, including the configuration of network IP addresses and the selection of gaze-based interaction as the preferred input method. Hand tracking functionality is implemented using the Unity XR Hands package in conjunction with the XR Interaction Toolkit, providing natural, controller-free manipulation of interface elements. Gaze-based interaction is implemented by accessing eye-tracking data through OpenXR, enabling the user to select objects and interface components using their gaze within the immersive environment.

5.4.2.1 Hand Tracking

During the initial setup phase, hand tracking was enabled and configured to allow intuitive, controller-free interaction with the application's user interface. This was accomplished by integrating Unity's `XR Hands` package together with the `OpenXR Hand Interaction Profile`. The Hand Interaction Profile exposes a standardized device layout within the Unity input system, allowing the application to interpret hand pose and gesture data from the device's hand tracking subsystem. To activate this functionality, the Hand Interaction Profile and the XR Hand Tracking subsystem were enabled in the project's OpenXR settings, and the required `HAND_TRACKING` permission was granted through the Magic Leap project configuration.

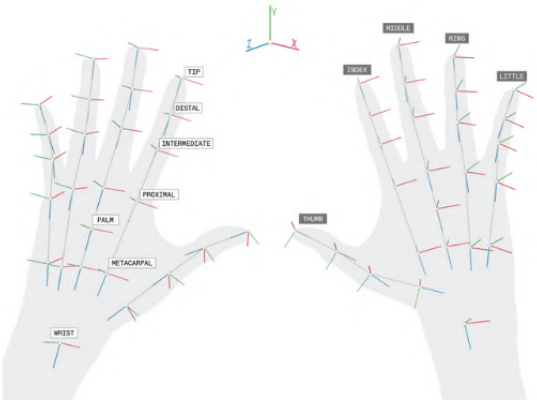
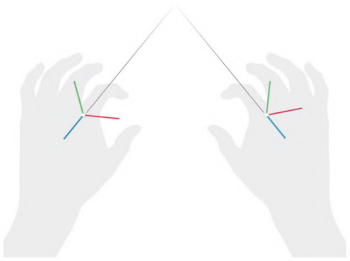
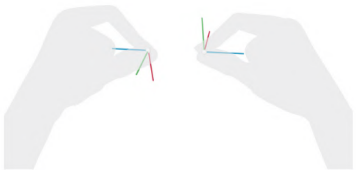


Figure 5.10: OpenXR hand joint conventions for left and right hands

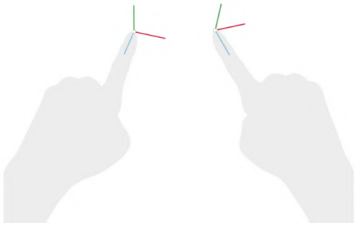
Three primary gestures are supported for interaction: **aim**, **pinch**, and **poke**, and the user can select their preferred method according to personal comfort. The *aim* and *pinch* gestures are suitable for interactions at larger distances, allowing the user to point toward or select interface elements remotely, while the *poke* gesture is used for close-range interactions, such as confirming selections or activating UI panels within immediate reach.



(a) OpenXR aim key pose for left and right hands



(b) OpenXR pinch key pose for left and right hands



(c) OpenXR poke key pose for left and right hands

Figure 5.11: Key poses used by the OpenXR Hand Interactions Extension

To provide clear visual feedback during hand-based interaction, Unity's UI Button component was configured using the Color Tint transition mode. Each button supports five states: Normal, Highlighted, Pressed, Selected, and Disabled. When a hand ray or poke gesture hovers over a button, the Highlighted state indicates focus, while a pinch or poke action triggers the Pressed state to confirm selection. A short fade duration (0.1 s) was applied to ensure responsive state transitions. This visual feedback improves clarity and reduces ambiguity during controller-free interaction.

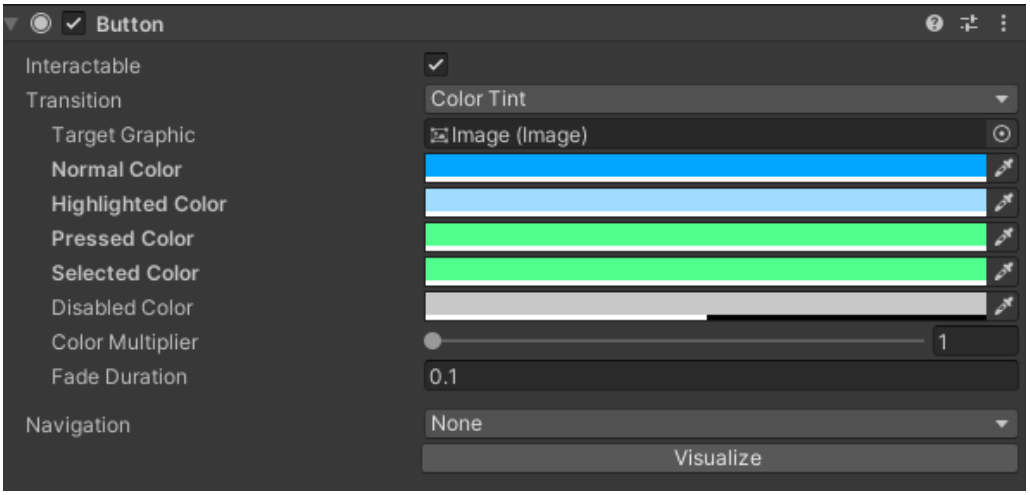


Figure 5.12: Configuration of Unity UI Button visual states

5.4.2.2 Eye Tracking

The Magic Leap 2 headset provides integrated eye-tracking capabilities through dedicated APIs that allow applications to access detailed information about the user's visual attention and eye behavior. These APIs enable estimation of where the user is looking in 3D space and support gaze-based interaction techniques.

The Magic Leap Eye Tracker API exposes multiple data streams describing the state and behavior of the user's eyes, including:

- **Gaze Direction:** Determines the direction of the user's visual focus in 3D space.
- **Pupil Size:** Measures the diameter of each pupil.
- **Eye Openness:** Indicates whether the eyes are open, partially closed, or blinking.
- **Gaze Behavior:** Detects eye movement patterns such as saccades and fixations.

Human eye movements mainly consist of rapid transitions and stable viewing periods:

- **Saccades:** Fast, ballistic eye movements that rapidly shift the gaze from one point to another. During a saccade, visual perception is significantly reduced, meaning the user is not intentionally observing objects.
- **Fixations:** Short periods during which the eyes remain relatively stable and focused on a specific point. Visual information processing primarily occurs during fixations.

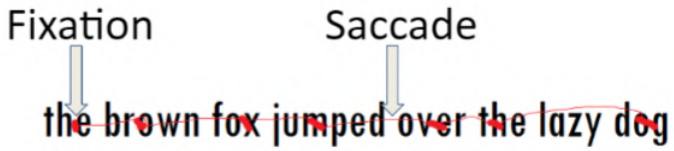


Figure 5.13: Example of saccades and fixations, eye-movement is represented in red

In gaze-based interaction systems, only fixations are considered meaningful indicators of user intent. Saccades represent transitional movements rather than deliberate attention; therefore, interaction techniques rely on fixation data to avoid unintended selections and improve interaction stability.

The *eye fixation point* represents the 3D position where the user is currently looking. It is computed as the intersection of the visual axis vectors originating from the left and right eyes. A virtual *gaze ray* is derived from the head pose toward this fixation point and is used throughout the system for interaction and spatial targeting.

Within Unity, eye-tracking functionality is accessed through the `MagicLeapEyeTrackerFeature`, an experimental OpenXR extension designed for Magic Leap 2 devices. This feature provides real-time access to gaze pose information, pupil measurements, and behavioral eye metrics required for immersive interaction.

The eye-tracking data is organized into several categories:

- **Geometric Data:** Eye openness and eye position relative to the head.
- **Pupil Data:** Pupil diameter measurements for each eye (requires the `PUPIL_SIZE` permission).
- **Gaze Behavior Data:** Information about gaze events such as fixations and saccades.
- **Static Data:** Hardware-related parameters, such as eye camera resolution.
- **Pose Data:** Spatial pose information including gaze origin and gaze direction for both eyes.

Before accessing eye-tracking data, the Unity project must be configured to use Magic Leap OpenXR with the `MagicLeapEyeTrackerFeature` enabled in the XR Plugin Management settings.

UI Gaze Based Interaction

Gaze-based interaction allows users to select and manipulate virtual elements without requiring handheld controllers. Two interaction techniques are implemented: **Dwell** and **Blink**. Both rely on the previously defined eye fixation point and gaze ray as the primary interaction reference. All gaze interactions employ temporal validation combined with progressive visual feedback to prevent unintended activation.

- **Dwell Interaction:** Dwell interaction is based on sustained visual attention. When the user fixates on an interface element for a predefined duration, the system interprets this as an intentional selection. The gaze ray performs continuous ray-casting to detect intersections with interactive UI elements. When the gaze remains on the same element, a dwell timer accumulates time. After a short interaction delay, a progressive visual indicator appears, informing the user that selection is in progress. If the fixation duration exceeds a predefined threshold, the element is automatically selected. Moving the gaze away immediately resets the timer and visual feedback.
- **Blink Interaction:** Blink interaction provides an explicit confirmation mechanism based on voluntary eye closure. While observing an interface element, the system monitors eye openness independently for each eye. A blink gesture is detected when either eye crosses a predefined openness threshold. Maintaining this closure beyond the interaction delay activates the progressive confirmation indicator. If the closure duration reaches the required threshold, the corresponding action is executed. Reopening the eye before completion cancels the interaction and resets feedback.

Both interaction techniques follow a shared workflow:

- The eye fixation point determines the observed interface element.
- The element is highlighted to provide immediate feedback.
- Temporal validation confirms user intent through:
  - sustained fixation for **Dwell**, or
  - voluntary single-eye closure for **Blink**.
- After validation, the corresponding action is executed.

Gaze Based Placement Determination

After an object has been selected, a gaze-driven placement phase allows the user to position the object within the environment before robotic manipulation. The placement interaction is designed as a two-stage workflow consisting of continuous gaze-based positioning followed by placement confirmation.

Instead of directly manipulating the original object, the system generates a **clone** acting as a visual placement preview.

- The original object state remains unchanged during placement.
- Users receive immediate spatial feedback before committing to an action.
- The preview can be repositioned safely multiple times.

The clone inherits the position, rotation, and world scale of the selected object and is displayed as a semi-transparent visualization to distinguish it from real objects.

Placement is controlled using the gaze ray. The procedure operates as follows:

1. A ray is projected along the gaze direction into the environment.
2. The ray intersects valid placement surfaces defined in the virtual environment.
3. The intersection determines the desired placement position.
4. The object is positioned slightly above the surface using an offset derived from its geometric bounds, ensuring natural resting without surface penetration.

To prevent jitter caused by small eye movements, the clone motion is smoothly interpolated toward the target pose, producing stable placement behavior. While placement is active, the clone continuously follows the gaze target.

Placement locking is triggered using the previously defined voluntary blink gesture. Maintaining eye closure for a specified duration toggles the placement state:

- **Unlocked State:** the clone follows the gaze and can be repositioned freely.
- **Locked State:** the clone remains fixed in space.

During locking, the clone gradually changes color to provide progressive visual feedback. Releasing the blink before completion cancels the action. After locking the placement, a confirmation interface becomes visible near the object. Confirmation reuses the previously described dwell and blink interaction techniques applied to confirmation UI elements.

Permission Management

Accessing sensitive device capabilities such as eye tracking, pupil size measurement, and the eye camera on the Magic Leap 2 requires explicit user consent, as these are classified as *dangerous* permissions by the Android-based platform. During application startup, the system requests permission from the user for these features. If the user grants access, the corresponding eye-tracking sensors and interaction modules are enabled. This ensures that the system can provide accurate gaze-based interaction while respecting user permissions.

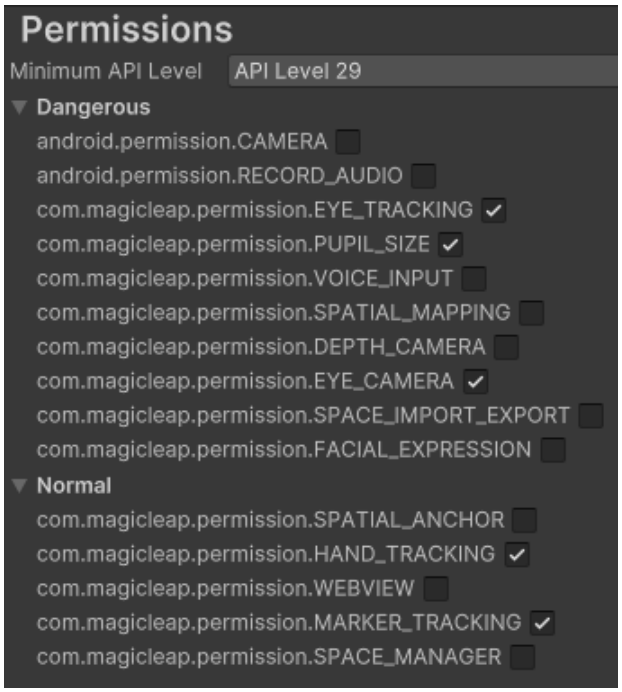


Figure 5.14: Application permissions configured in the Magic Leap project settings. Dangerous permissions require user confirmation at runtime

5.5 Grasp Generation and Evaluation

Once gaze-based object selection identifies the target, the next step is determining how the robotic system should physically grasp the object. To address this, we rely on the grasp pose detection framework proposed by Andreas ten Pas, Marcus Gualtieri, Kate Saenko, and Robert Platt (2017) in "Grasp Pose Detection in Point Clouds", which formulates grasping as a direct detection problem operating on raw 3D geometry rather than requiring object recognition or CAD models.

The grasp pose detection method works as follows. Given a 3D point cloud representing an object or the surrounding scene, the algorithm first samples points on the object surfaces to capture relevant geometric information. For each sampled point, local geometric features are estimated, including surface normals, which indicate the direction perpendicular to the surface at that point, and curvature directions, which describe how the surface bends. These local features provide critical information for determining how a gripper can approach and securely hold the object.

Using this geometric information, a virtual parallel-jaw gripper is aligned with each sampled point to generate multiple candidate grasp poses in six degrees of freedom (6-DOF). A 6-DOF grasp specifies both the position and orientation of the gripper, allowing the robot to approach the object from any angle. Each candidate grasp is first evaluated for basic geometric feasibility, including collision avoidance with the object, and ensuring that there are sufficient points within the gripper's closing region to make a stable grasp possible.

Next, each candidate grasp is represented as a small local map of the space around the gripper. This map describes which areas are occupied by the object, the orientation of nearby surfaces, and which parts of the object the gripper can reach. By providing this structured information, the neural network can effectively evaluate the quality and feasibility of each grasp. The structured representation is then input to a convolutional neural network (CNN) trained on force-closure simulations. A force-closure grasp is one in which the gripper can securely hold the object and resist external forces or torques from any direction, ensuring a stable grasp. The CNN predicts the likelihood that each candidate grasp will achieve force closure under real-world conditions. Finally, the highest-scoring grasps are selected and returned as viable six-degree-of-freedom (6-DOF) poses, ready for execution by the robotic manipulator. By integrating local geometric information, systematic candidate generation, and learned evaluation, this approach enables reliable grasping of previously unseen objects.

5.5.1 Initial Setup

The Grasp Pose Detection (GPD) system was set up and configured in a WSL Ubuntu 18.04 environment, providing a Linux-based platform for running the point cloud processing and neural network evaluation components. The setup involved preparing configuration files that define the gripper geometry, workspace parameters, point cloud preprocessing options, candidate grasp generation settings, and grasp selection criteria.

Key configuration parameters included the geometry of the gripper, such as finger width, hand depth, hand height, and initial bite, which determine the physical constraints of candidate grasps. The workspace dimensions and camera position were specified to ensure that point clouds captured from the environment were correctly aligned with the robot base frame.

Grasp candidate generation parameters defined the number of points sampled from the point cloud, the number of gripper orientations and finger placements evaluated, and additional criteria for producing feasible grasps. Filtering options were applied to remove candidates that did not satisfy geometric constraints or approach directions. In particular, grasps with approach directions from below the object were discarded to avoid collisions with the supporting surface. Clustering and scoring procedures were then used to select a final set of high-confidence grasps, retaining up to 20 candidates for execution.

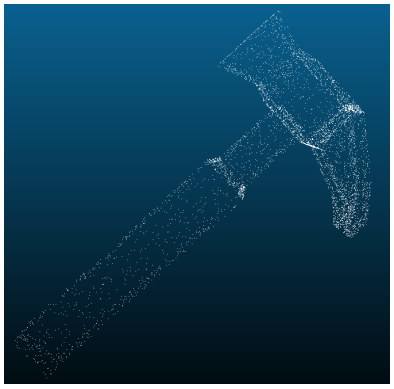
To improve runtime performance, the number of CPU processing threads used by the GPD system was increased to seven, enabling parallel evaluation of grasp candidates and significantly reducing grasp computation time. Visualization parameters were available to inspect intermediate results, such as surface normals, sampled points, and candidate grasps, but were typically disabled during normal operation to maintain faster execution.

5.5.2 Point Cloud Representation

In our implementation, each object selected via gaze is converted into a point cloud by sampling points from its mesh surface. Surface normals are optionally included to capture local geometric orientation. The sampled points are then transformed into the robot's base frame using ROS-compatible coordinates, ensuring alignment between the augmented reality scene and the manipulator's workspace. This representation encodes the object's shape directly, allowing grasp planning to operate without relying on predefined object models.



(a) Hammer object in Unity



(b) Hammer point cloud from Unity

Figure 5.15: Visualization of a target object in Unity and its corresponding point cloud used for grasp pose detection

5.5.3 Grasp Generation Implementation

After the point cloud is generated, it is transmitted to the grasp planning module through a TCP-based communication interface. The sampled points are serialized using the Point Cloud Data (PCD) format and sent together with scene metadata describing the target object pose and surrounding obstacles.

After the point cloud is received, the Grasp Pose Detection (GPD) module is executed to generate candidate grasp configurations directly from the observed geometry. GPD evaluates multiple potential grasp locations by aligning a virtual parallel-jaw gripper with local surface regions and estimating the likelihood of successful force-closure grasps. Each detected grasp is represented as a six-degree-of-freedom (6-DOF) pose consisting of a 3D position and an orientation expressed as a quaternion.

Since GPD typically produces many valid grasp hypotheses, the candidates are sorted according to their predicted grasp quality score. Rather than immediately selecting a single grasp, the system stores the top 10 highest-scoring candidates in a CSV file for further examination. This design choice is motivated by the fact that grasp scoring is based primarily on local geometry; therefore, the highest-scoring grasp is not always the most practical choice when considering the full scene context or execution constraints.

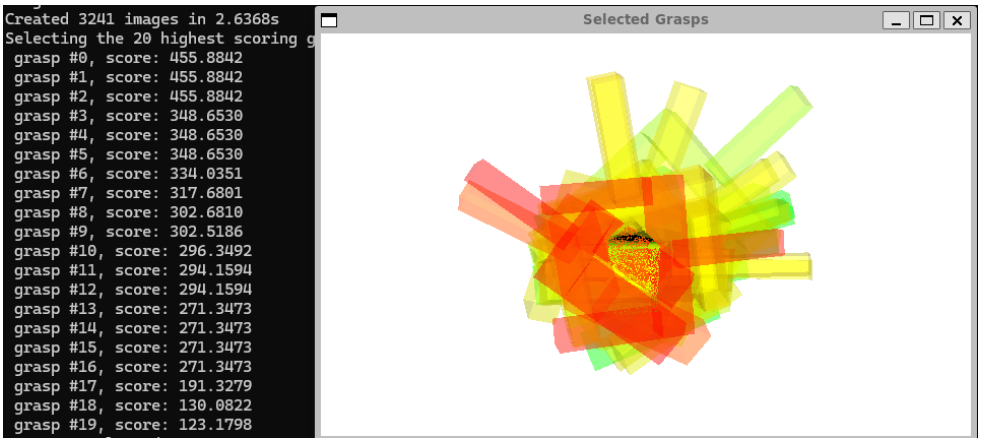


Figure 5.16: Selected grasps returned from GPD for a given point cloud

5.5.4 Machine Learning-Based Grasp Evaluation

Although the Grasp Pose Detection (GPD) framework assigns a geometric quality score to each candidate grasp, these scores rely mainly on local surface geometry and do not explicitly consider global scene constraints or execution feasibility. To improve grasp selection reliability, a machine learning (ML) module was introduced to evaluate candidate grasps using both geometric and contextual scene information.

5.5.4.1 Dataset Logging

A TCP-based grasp server was developed to automatically generate a dataset during grasping trials. The complete pipeline operates as follows:

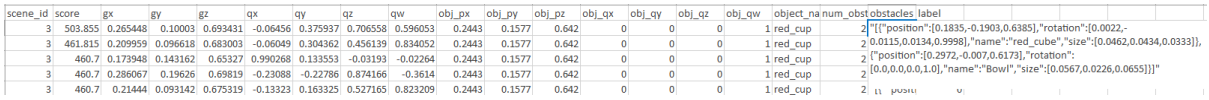
1. **Scene Transmission:** Unity transmits scene metadata and the sampled object point cloud to the grasp server.
2. **Grasp Generation:** The received point cloud is processed by the GPD framework, producing multiple grasp candidates with associated scores and 6-DOF poses.
3. **Dataset Construction:** The top $N = 10$ grasps are appended to a dataset used for ML training and inference.

Each grasp candidate is stored together with structured scene metadata describing the manipulation context. Table 5.1 summarizes the recorded features.

Category	Stored Information
Scene Information	Scene ID
Grasp Candidate	GPD score, position (x, y, z), orientation quaternion (q_x, q_y, q_z, q_w)
Target Object	Object name, position, orientation quaternion
Environment Context	Number of obstacles
Obstacle Metadata	Serialized list containing obstacle name, pose and size

Table 5.1: Fields stored for each grasp sample in the ML dataset.

This representation associates each grasp with both local geometric quality and global scene context, enabling supervised learning of execution success beyond purely geometric grasp metrics.



scene_id	score	gx	gy	gz	qx	qy	qz	qw	obj_px	obj_py	obj_pz	obj_qx	obj_qy	obj_qz	obj_qw	object_na	num_obst	obstacles	label
3	503.855	0.265448	0.10003	0.693431	-0.06456	0.375937	0.706558	0.596053	0.2443	0.1577	0.642	0	0	0	1	red_cup	2	"[{"position":[0.1835,-0.1903,0.6385],"rotation":[0.0022,-	
3	461.815	0.209959	0.096618	0.683003	-0.06049	0.304362	0.456139	0.834052	0.2443	0.1577	0.642	0	0	0	1	red_cup	2	0.0115,0.0134,0.9998],"name":"red_cube","size":[0.0462,0.0434,0.0333]},	
3	460.7	0.173948	0.143162	0.65327	0.990268	0.133553	-0.03193	-0.02264	0.2443	0.1577	0.642	0	0	0	1	red_cup	2	{"position":[0.2972,-0.007,0.6173],"rotation":	
3	460.7	0.286067	0.19626	0.69819	-0.23088	-0.22786	0.874166	-0.3614	0.2443	0.1577	0.642	0	0	0	1	red_cup	2	[0.0,0.0,0.0,1.0],"name":"Bowl","size":[0.0567,0.0226,0.0655]]"	
3	460.7	0.21444	0.093142	0.675319	-0.13323	0.163325	0.527165	0.823209	0.2443	0.1577	0.642	0	0	0	1	red_cup	2		

Figure 5.17: Dataset structure

5.5.4.2 Scene Design and Ground Truth Labeling

Synthetic scenes were constructed without AR-specific elements, focusing purely on object placement and surrounding obstacles. Each candidate grasp was executed using ROS. Grasp outcomes were programmatically labeled as:

- **Successful:** The robot securely lifted and held the object without collisions or dropping.
- **Unsuccessful:** The grasp failed due to dropping, collision, or inability to close the gripper properly.

These labels, together with the corresponding point clouds and grasp metadata, formed the supervised dataset for training the ML model.

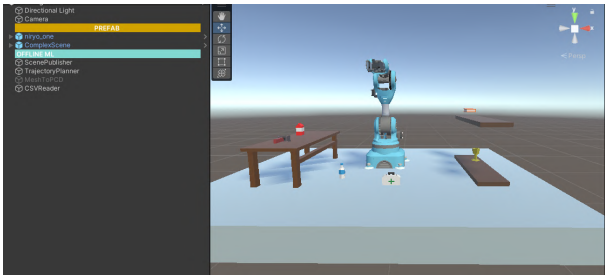


Figure 5.18: Experimental scene setup for offline data collection and labeling

5.5.4.3 Data Collection and Labeling

The dataset used for training the grasp evaluation model was generated through repeated interaction with the simulated manipulation environment. Data collection was primarily performed by manually creating diverse scene configurations in Unity. Objects and obstacles were repositioned, reoriented, and rearranged to produce variations in occlusions, grasp accessibility, and spatial constraints. For each configuration, the system automatically sampled object surface points, generated a point cloud representation, and transmitted the scene metadata to the grasp server.

In addition to offline scene construction, further data were collected during online testing of the complete application pipeline. While evaluating system performance, grasp requests generated during normal operation were also logged and appended to the dataset, allowing realistic execution conditions to be captured.

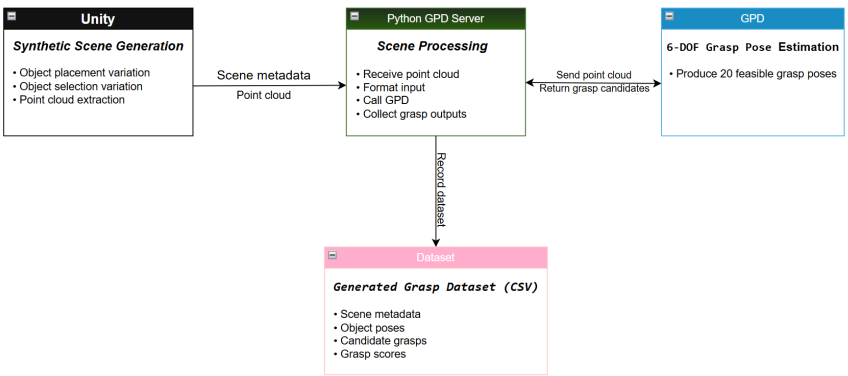


Figure 5.19: Offline data collection architecture

Dataset labeling was performed using real execution feedback rather than heuristic assumptions. Each recorded grasp candidate was transmitted to the robotic control framework through ROS, where a motion trajectory was generated and executed.

After trajectory execution, the grasp outcome was automatically evaluated and assigned a binary label:

- **1** — Successful grasp execution (stable object acquisition).
- **0** — Failed grasp execution (collision, slip, or unreachable pose).

The success label was appended to the corresponding dataset row, transforming the collected samples into supervised learning data. This execution-driven labeling strategy allows the model to learn grasp success based on real manipulation outcomes rather than purely geometric metrics.

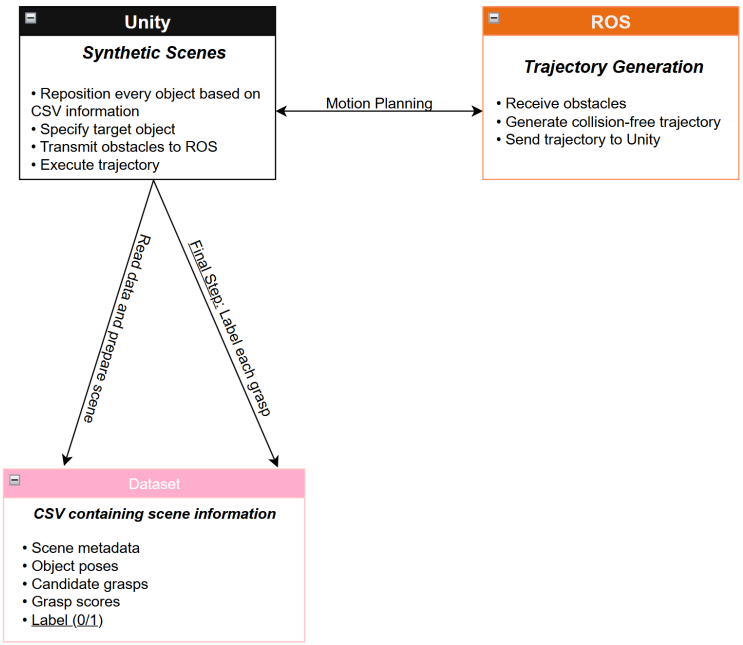


Figure 5.20: Data labeling process for CSV entries

5.5.4.4 ML Training and Model Optimization

Following feature extraction, a supervised learning pipeline was implemented to train a grasp success classifier capable of predicting whether a candidate grasp would successfully execute in the manipulation environment. The objective of the model is to learn execution feasibility by jointly considering grasp geometry, object pose, and surrounding scene constraints.

The expanded dataset was first preprocessed by removing non-numeric identifiers (such as object names) while preserving all geometric and contextual features. The resulting feature matrix $\mathbf{X}$ contained grasp pose information, object pose parameters, obstacle descriptors, the number of obstacles in the scene, and derived geometric features, while the target vector $\mathbf{y}$ represented the binary grasp success label.

To enable fixed-dimensional supervised learning on variable-complexity manipulation scenes, obstacle information was transformed into a structured representation. For each grasp candidate, the Euclidean distance between the grasp position and all scene obstacles was computed, and the $K = 3$ closest obstacles were selected. Each obstacle was encoded using its 3D position (3), orientation quaternion (4), and bounding box dimensions (3), resulting in 10 features per obstacle and 30 obstacle-related features per sample. Zero-padding was applied when fewer than three obstacles were present to maintain consistent dimensionality.

In addition to raw geometric descriptors, several derived spatial interaction features were introduced:

- Euclidean distance between grasp pose and object pose (`dist_grasp_obj`),
- Angular error between gripper and object orientation (`rot_diff_grasp_obj`),
- Gripper–obstacle distances for the three nearest obstacles (`dist_gripper_obs1-3`),
- Object–obstacle distances for the three nearest obstacles (`dist_obj_obs1-3`).

These features explicitly encode spatial proximity, orientation alignment, and local collision risk, which are critical factors in determining grasp feasibility. The choice of $K = 3$ was empirically selected to balance local scene expressiveness and feature dimensionality.

Category	Feature Names	Count
Identifier	Scene ID	1
Score Feature	GPD score	1
Grasp Pose	Position: gx, gy, gz	3
	Orientation: qx, qy, qz, qw	4
Object Pose	Position: obj_px, obj_py, obj_pz	3
	Orientation: obj_qx, obj_qy, obj_qz, obj_qw	4
Number of Obstacles	num_obstacles	1
K-Nearest Obstacles ($K = 3$)	Obstacle 1: px, py, pz; qx, qy, qz, qw; sx, sy, sz	10
	Obstacle 2: px, py, pz; qx, qy, qz, qw; sx, sy, sz	10
	Obstacle 3: px, py, pz; qx, qy, qz, qw; sx, sy, sz	10
Derived Geometric Features	Grasp–object distance: dist_grasp_obj	1
	Rotational difference: rot_diff_grasp_obj	1
	Grasp–obstacle distances: dist_gripper_obs1-3	3
	Object–obstacle distances: dist_obj_obs1-3	3
Total		**55**

Table 5.2: Feature Overview for Grasp Success Prediction

To ensure reliable evaluation and prevent data leakage, the dataset was divided using a two-stage stratified splitting procedure.

First, 20% of the dataset was reserved as an independent test set. The remaining 80% of the data was then further divided into training and validation subsets, where 20% of the remaining samples were allocated for validation. This resulted in the following effective proportions:

- **Training set:** 64% of the total dataset
- **Validation set:** 16% of the total dataset
- **Test set:** 20% of the total dataset

Stratification preserved the ratio of successful and unsuccessful grasps across all splits. Since grasp datasets typically exhibit class imbalance (fewer successful grasps than failures), a class-weight compensation factor was computed as:

$$\text{scale_pos_weight} = \frac{N_{\text{negative}}}{N_{\text{positive}}}$$

This weighting biases the learning algorithm toward correctly identifying successful grasps.

A gradient-boosted decision tree classifier was selected due to its strong performance on structured tabular data and its ability to model nonlinear relationships between spatial features. The implementation was based on the `XGBoost` framework, which provides efficient histogram-based tree construction and scalable parallel training.

Gradient boosting iteratively constructs an ensemble of decision trees, where each new tree learns to correct the residual errors of previous trees. This approach is particularly suitable for grasp evaluation because grasp success depends on complex interactions between geometry, orientation, and obstacle proximity.

Model hyperparameters were optimized using the `Optuna` automatic hyperparameter search framework. Instead of manually selecting parameters, Optuna performs adaptive sampling within predefined search ranges, maximizing validation performance.

The following parameters were optimized:

- Maximum tree depth,
- Learning rate,
- Number of boosting estimators,
- Subsample ratio,
- Feature sampling ratio,
- Minimum child weight,
- Split regularization parameter (γ).

Each trial trained an independent model and evaluated its performance on the validation dataset using the F1-score of the positive class:

$$F_1 = 2 \cdot \frac{\text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}$$

The F1-score was selected as the optimization objective because it provides a balanced measure between precision and recall. In the context of grasp prediction, this ensures that the model both correctly identifies successful grasps and avoids incorrectly predicting grasp success when a grasp would fail. During optimization, the positive class (successful grasp) was treated as the target class using `pos_label=1`.

The optimization process executed 500 trials, allowing Optuna to progressively explore the hyperparameter space and converge toward parameter configurations that improved grasp success prediction performance.

After hyperparameter optimization, a final classifier was trained using the best parameter configuration on the combined training and validation data. Training employed histogram-based tree construction to improve computational efficiency while maintaining predictive accuracy.

The model outputs a probability estimate:

$$P(\text{success} \mid \mathbf{x})$$

representing the likelihood that a grasp candidate will succeed during execution.

The optimized classifier was evaluated on the held-out test dataset using standard classification metrics, including precision, recall, F1-score, and confusion matrix analysis. These metrics quantify the model's ability to distinguish executable grasps from unstable ones under previously unseen scene configurations.

The final trained model was serialized using the `joblib` framework and stored as:

`grasp_success_xgboost_optuna.pkl`

5.5.4.5 Final Grasp Prediction and Transmission

During online operation, the trained XGBoost classifier is deployed as part of the grasp server pipeline. After Unity transmits the selected object and scene information, the Grasp Pose Detection (GPD) module generates multiple candidate grasp poses ranked by geometric quality. These candidates are appended to the dataset and immediately processed by the trained model, which evaluates each grasp and predicts its probability of successful execution.

The grasp with the highest predicted success probability is selected as the final execution candidate. The corresponding six-degree-of-freedom (6-DOF) pose, consisting of the gripper position (x, y, z) and orientation quaternion (q_x, q_y, q_z, q_w), is then transmitted back to Unity through the TCP communication interface.

This procedure enables real-time grasp selection that combines geometric grasp detection with learned experience from previous executions, allowing the system to adaptively choose grasps that are more likely to succeed in complex manipulation scenarios.

5.6 ROS–Unity Integration Architecture

The robotic motion planning pipeline was implemented using the *Robot Operating System (ROS)* in combination with the *MoveIt* motion planning framework. The initial system setup was based on the Pick-and-Place tutorial provided in the *Unity Robotics Hub* repository by Unity Technologies. Specifically, the tutorial available at the official GitHub repository served as a baseline for establishing ROS–Unity communication, configuring the MoveIt planning environment, and deploying a basic pick-and-place workflow.

While the tutorial provided the foundational infrastructure, the final system architecture was significantly extended and modified to support scene-dependent planning logic, dynamic obstacle synchronization, and integration with the proposed machine learning-based grasp evaluation module.



Figure 5.21: Unity Robotics Hub

Communication between Unity and ROS was implemented using the ROS–TCP Connector. Unity acts as the high-level orchestration layer, while ROS performs motion planning and collision-aware trajectory generation. The overall data flow is summarized as follows:

1. Unity transmits the current joint configuration, pick pose, place pose, and scene identifier.

2. ROS updates the MoveIt Planning Scene with collision objects.

3. A custom ROS service computes a multi-stage pick-and-place trajectory.

4. The generated trajectories are returned to Unity for execution.

This architecture ensures a clear separation between high-level perception and decision-making in Unity and low-level motion planning in ROS.

5.6.1 ROS Messages and Services Overview

In ROS, communication between nodes is primarily handled using *messages* and *services*.

A **message** is a data structure used to transmit information between nodes, often published on a topic and subscribed to by multiple nodes. For example, collision objects representing obstacles in the robot's environment are transmitted using the standard `CollisionObject` message.

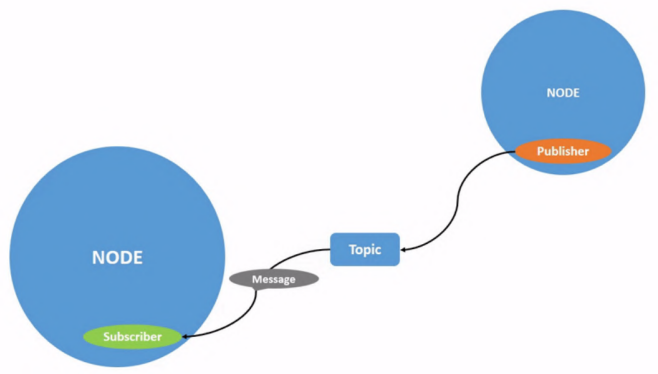


Figure 5.22: ROS Topics: Publisher and Subscriber Guide

A **service**, in contrast, implements a request-response interaction. One node sends a request, another node performs a computation or action, and returns a response. In the proposed framework, a custom service named `MoverService` is used to plan complete pick-and-place trajectories, allowing Unity to request motion plans from ROS and receive structured trajectories in return.

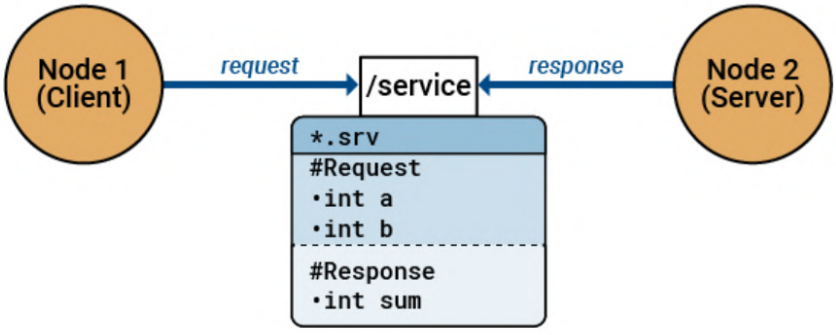


Figure 5.23: ROS Services: Service Client and Service Server Guide

5.6.2 MoveIt Planning Scene and Motion Planning Service

The *MoveIt Planning Scene* is a central representation of the robot and its environment used for motion planning. It maintains the current state of the robot, collision objects, and environmental constraints. All motion planning requests take into account this scene to ensure collision-free trajectories. In this work, the Planning Scene is dynamically updated from Unity using ROS messages, allowing real-time adaptation to changes in the environment, such as moving obstacles or target objects.

To support flexible and scene-aware motion planning, a custom ROS service node was implemented. The service, `niryo_moveit`, is based on the `moveit_commander` Python API and exposes a planning interface through a custom `MoverService` definition.

The `MoverService.srv` is designed to encapsulate all necessary parameters for a complete pick-and-place operation, while providing detailed feedback upon completion or failure.

The **Service Request** provides the planner with the robot's initial state and target objectives:

- `joints_input`: A custom `NiryoMoveitJoints` message containing the starting joint configuration (`float64[6] joints`), as well as the target `pick_pose` and `place_pose`.
- `pick_pose` & `place_pose`: Standard `geometry_msgs/Pose` messages defining the exact Cartesian targets.
- `scene_id`: An `int32` identifier used to trigger scene-dependent planning adaptations, such as the multi-candidate pre-grasp strategy.

The **Service Response** returns the computed motion plans alongside diagnostic information:

- `trajectories`: An array of `moveit_msgs/RobotTrajectory` objects corresponding to the sequential stages of the pick-and-place task.
- `status`: A string indicating the overall result (e.g., "Success", or "Failed").
- `failed_stage`: A string specifying exactly where the planner failed (e.g., "Pre-Grasp", "Grasp", "PickUp", "Place", or "ReturnHome"), allowing the Unity client to handle errors gracefully.

When a request is received, the planning pipeline is structured into sequential stages:

1. **Pre-Grasp (Approach)** – planning a collision-free approach pose.
2. **Grasp (Descent)** – moving from pre-grasp to the exact pick pose.
3. **Pick-Up (Lift)** – lifting the object to a safe offset.
4. **Place** – planning motion to the target placement pose.
5. **Return to Initial Configuration** – restoring the starting joint state.

For each stage, the start state of the robot is explicitly defined using the current joint configuration, ensuring deterministic planning behavior. The planner configuration includes:

- Enabling replanning.
- Multiple planning attempts.
- A fixed planning time window.
- `RRTConnectkConfigDefault` planner selection.

Each planned trajectory is validated to ensure that non-empty joint trajectories are generated before proceeding to the next stage.

Accurate motion planning requires a consistent representation of the environment in the MoveIt Planning Scene. To achieve this, a Unity-based publisher script continuously synchronizes obstacle information with ROS.

Collision objects are published to the `/collision_object` topic using the standard `CollisionObject` message type. The implementation supports both primitive and mesh-based representations:

- Box-like geometries are transmitted as `SolidPrimitive` objects.
- Arbitrary meshes are converted to ROS `Mesh` messages.

To reduce communication overhead and improve real-time performance, obstacle updates are transmitted only when significant changes in position, orientation, or scale exceed predefined thresholds. This prevents unnecessary Planning Scene updates while maintaining synchronization accuracy.

Attribute	Threshold Variable	Value	Condition for Update
Position	`POS_THRESHOLD`	0.001 m	Euclidean distance between current and previous local position exceeds threshold.
Orientation	`ROT_THRESHOLD`	0.5°	Angular difference between current and previous local rotation exceeds threshold.
Scale/Size	`SIZE_THRESHOLD`	0.001 m	Change in the calculated bounding box dimensions exceeds threshold.

Table 5.3: Obstacle State Change Thresholds

When an object is selected as the manipulation target, it is temporarily removed from the collision environment in ROS to prevent the planner from treating it as a static obstacle. Upon completion of the grasp phase or target change, the object is reinserted into the Planning Scene.

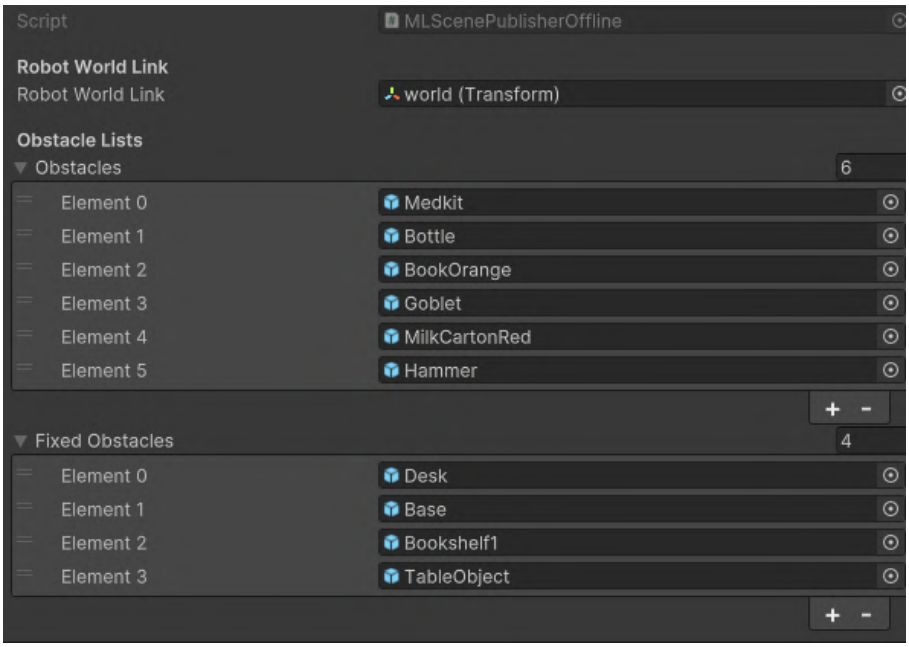
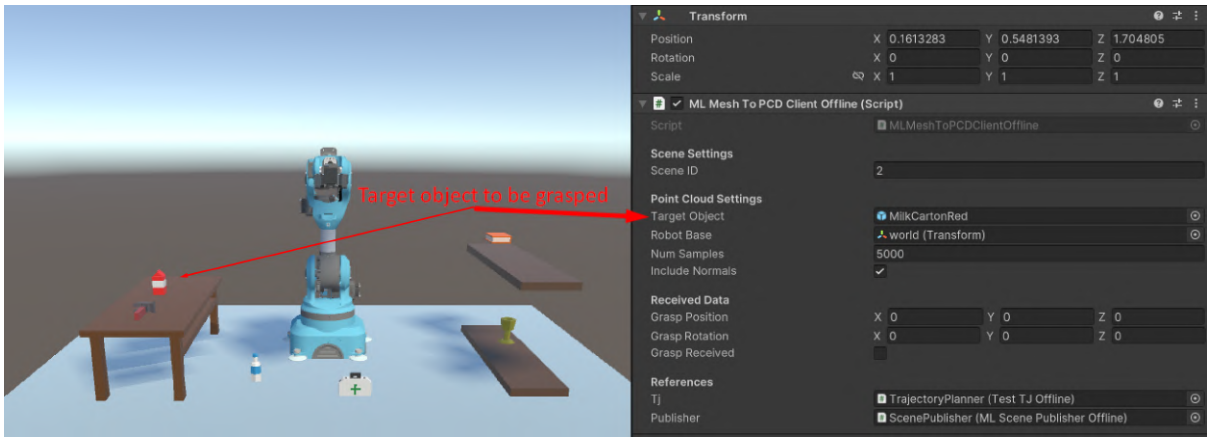
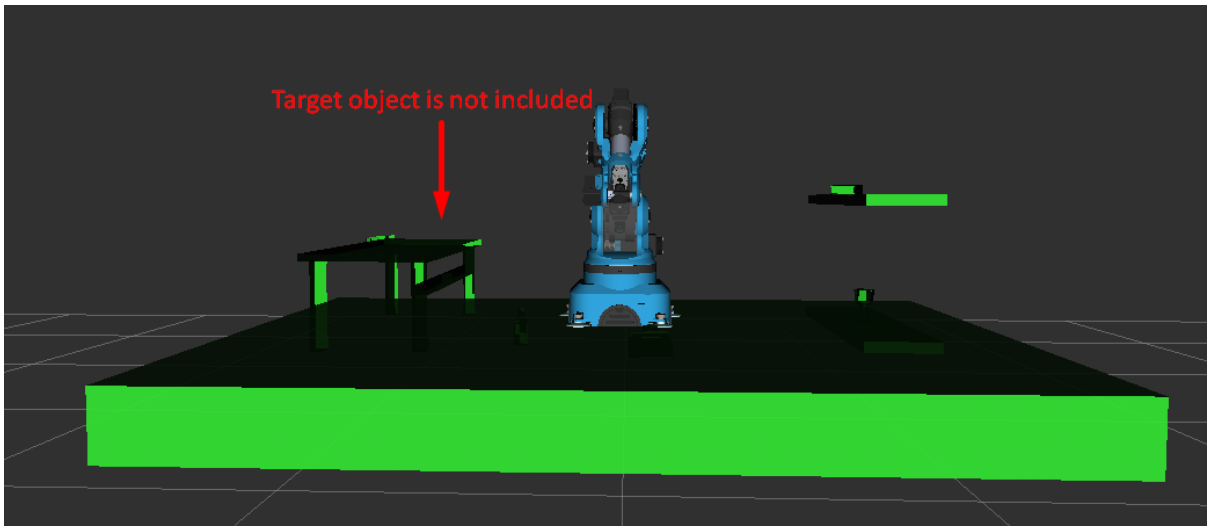


Figure 5.24: Obstacles included in script to be published



(a) Unity Editor view showing the Niryo One robot, target object (red), and all the scene obstacles



(b) Corresponding RViz visualization showing the synchronized MoveIt Planning Scene, with Unity obstacles rendered as green

Figure 5.25: Synchronization of the simulation environment: (a) setup of the robot and obstacles within Unity (b) the resulting published planning scene in RViz via ROS

5.6.3 Scene-Dependent Planning Adaptations and Coordinate Transforms

Beyond the baseline tutorial, several adaptive planning strategies were introduced.

5.6.3.1 Multi-Candidate Pre-Grasp Strategy

Instead of a single pre-grasp offset, multiple approach poses are generated dynamically based on the Unity scene identifier. Candidates are evaluated sequentially until a feasible trajectory is found, increasing robustness in constrained environments where a single approach vector risks collision. The pre-grasp candidates are defined as follows:

- **Primary Candidate (Vertical):** A standard offset of +0.05m along the Z-axis, applied to all grasp targets.
- **Secondary Candidate (Horizontal):** An offset of -0.05m along the Y-axis. This is triggered dynamically for objects located deep inside constrained environments.
- **Third Candidate (Diagonal):** A combined offset of +0.05m in Z and -0.05m in Y, triggered under the same spatial constraints as the secondary candidate.

5.6.3.2 Coordinate Frame Transformations

Unity and ROS use different coordinate system conventions. Unity operates in a left-handed coordinate system, whereas ROS follows a right-handed convention.

All obstacle poses are therefore transformed relative to the robot's world reference frame before transmission. This conversion pipeline involves two primary steps:

- **Reference Frame Transformation:** Unity world coordinates are first transformed into the robot's local base frame using standard Unity spatial math (e.g., `InverseTransformPoint` and inverse quaternion multiplication).
- **Coordinate System Conversion:** The resulting local poses are then re-mapped to the Forward-Left-Up (FLU) coordinate representation. This is handled cleanly by leveraging the `.To<FLU>()` extension method, which automatically applies the axis and rotation conversions detailed in Table 5.4.

This clear separation of spatial transformation and axis conversion ensures geometric consistency between the perception, planning, and execution layers.

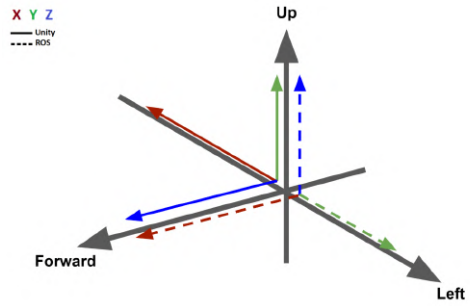


Figure 5.26: ROS - Unity Coordinate System Conversions

	Unity	ROS
Forward	Z	X
Right	X	-Y
Up	Y	Z
Rotation	clockwise	counter-clockwise

Table 5.4: Axis Conventions: Unity vs. ROS

5.6.4 Trajectory Execution, Gripper Control, and Failure Handling

While the robotic arm's high-level movements (approach, lift, place) are planned in ROS, the execution of these trajectories and the control of the end-effector are handled entirely within Unity.

When the custom ROS service returns the sequence of planned trajectories, Unity iterates through the joint states, smoothly applying them to the robot's `ArticulationBody` components. Interleaved within this execution sequence is a decoupled, physics-aware gripper control mechanism:

- **Adaptive Grasping:** At the end of the descent trajectory, an iterative coroutine incrementally closes the gripper fingers. A custom script monitors local collisions. Once both fingers detect contact, the closing motion halts.
- **Object Release:** Upon reaching the place pose, the gripper opens.

If the ROS planner fails at any stage prior to execution, the system relies on structured failure handling:

- The service returns a failure status alongside the exact failed stage identifier.
- The execution loop is aborted to prevent partial or unsafe motions.
- Any temporarily removed obstacles are restored to maintain scene integrity, and the UI resets gracefully.

5.7 System Overall Workflow

This section summarizes the complete end-to-end workflow of the implemented system, describing how gaze input, grasp generation, machine learning evaluation, motion planning, and execution are integrated into a unified closed-loop pipeline.

5.7.1 High-Level Interaction Loop

The final system operates as a sequential but tightly interconnected process composed of six major phases:

1. **Gaze-Based Target Selection (Unity / Magic Leap 2)**
2. **Point Cloud Extraction and Transmission (Unity → GPD Server)**
3. **Grasp Generation and ML Evaluation (GPD + XGBoost → Unity)**
4. **Motion Planning Request (Unity → ROS / MoveIt)**
5. **Motion Planning (ROS / MoveIt)**
6. **Trajectory Execution and Feedback (ROS → Unity)**

Each phase communicates through structured TCP or ROS-based interfaces, ensuring modular separation between perception, decision-making, and execution layers.

5.7.2 Step-by-Step Communication Pipeline

1. Gaze-Based Target Selection

The workflow begins on the **Magic Leap 2** device, where the user selects an object using gaze interaction inside Unity.

- A ray is cast from the user's gaze origin.
- Upon dwell/blink confirmation, the target object is selected.
- The object's pose and mesh information are retrieved.

At this stage, Unity acts as the perception and orchestration layer.

2. Point Cloud Extraction and Transmission

Once an object is selected:

- The mesh surface is sampled to generate a point cloud.
- Surface normals are computed.
- Points are transformed from Unity world coordinates into the robot base frame.
- The data are serialized in PCD format.

The point cloud, along with scene metadata (object pose, obstacle descriptions, scene ID), is transmitted via TCP to the grasp server running in WSL Ubuntu 18.04.

3. Grasp Generation and Machine Learning Evaluation

Inside the grasp server:

1. The point cloud is processed by the GPD framework introduced in "Grasp Pose Detection in Point Clouds".
2. Multiple 6-DOF candidate grasps are generated.
3. Candidates are geometrically filtered and ranked by GPD score.
4. The top $N = 10$ grasps are passed to the trained XGBoost classifier.

The classifier evaluates each grasp and the grasp with the highest predicted execution probability is selected as the final pick pose.

The chosen 6-DOF pose is then transmitted back to Unity through TCP.

4. Motion Planning Request

After receiving the final grasp pose:

- The user selects the desired place location in AR.
- Unity converts both pick and place poses into ROS FLU coordinates.
- Unity updates the Planning Scene by publishing collision objects.

Unity then sends a service request via the ROS–TCP Connector to the custom `MoverService` running in ROS.

The request contains:

- Current joint configuration
- Pick pose
- Place pose
- Scene identifier

5. Motion Planning

Within ROS:

- The MoveIt Planning Scene is updated.
- The multi-stage pick-and-place planner executes:
  1. Pre-Grasp
  2. Grasp
  3. Lift
  4. Place
  5. Return Home
- Multi-candidate pre-grasp offsets are tested if required.
- Each stage validates trajectory feasibility.

If planning succeeds:

- A sequence of `RobotTrajectory` messages is returned.

If planning fails:

- The failed stage identifier is returned.
- Unity aborts execution and resets the UI.

6. Trajectory Execution and Feedback

Upon successful planning:

- Unity sequentially executes the joint trajectories.
- Joint targets are applied to `ArticulationBody` components.
- The gripper closes using collision-aware incremental control.

During execution:

- The selected object is temporarily removed from the Planning Scene.
- After placement, it is reinserted.
- Real-time visual feedback is displayed in AR.

The final system architecture can be conceptually divided into three functional layers:

- **Interaction Layer (Unity / Magic Leap 2)** Gaze detection, visualization, UI, and user intent capture.
- **Grasp Decision Layer (GPD + XGBoost)** Geometry-based grasp generation and learned execution feasibility prediction.
- **Execution Layer (ROS / MoveIt)** Collision-aware motion planning and robotic trajectory generation.

Although the proposed framework integrates grasp generation, machine learning-based evaluation, and motion execution into a unified pipeline, it does not perform online adaptive learning during deployment. Instead, the system records the generated grasp candidates together with their corresponding scene metadata for offline analysis and possible future retraining.

This design choice ensures stable runtime behavior while maintaining consistent inference performance. At the same time, it allows the dataset to grow over time. Any improvement in grasp selection performance is therefore achieved through offline retraining.

Chapter 6

Evaluation & Testing

This chapter presents the evaluation and testing procedures conducted to assess the performance and usability of the proposed system. Initially, different machine learning models that were explored for grasp selection are discussed, along with their respective performance results. Based on this evaluation, a model was selected and further fine-tuned in order to improve selection accuracy and overall system performance.

Following the model selection, additional techniques were investigated to achieve a better balance between selection accuracy and system responsiveness. These approaches were experimentally evaluated and compared against both the fine-tuned model and the baseline implementation to determine their effectiveness within the application.

Furthermore, the chapter evaluates the proposed system through two distinct user studies. First, an evaluation of the interaction methods used throughout the gaze-based control is presented, specifically comparing blink-based and dwell-time mechanisms. Second, an assessment of the complete gaze-controlled robotic manipulation pipeline is conducted. These evaluations include quantitative measurements, such as task completion time and success rates, as well as user-centered assessments using standardized usability questionnaires, including the Usability Metric for User Experience-Lite (UMUX-Lite), the Single Ease Question (SEQ), and the NASA Task Load Index (NASA-TLX). These metrics provide insights into user workload, efficiency, and overall system usability.

The results presented in this chapter aim to provide a comprehensive assessment of both the technical performance of the detection system and the usability of the gaze-based interaction methods.

6.1 Machine Learning Models

This section presents the machine learning approaches explored to predict successful robotic grasps. The objective is to identify a model that provides accurate and reliable predictions while minimizing unsafe grasp attempts. Initially, multiple candidate models were evaluated on a tabular dataset representing grasp features, using standard classification metrics including accuracy, precision, recall, and F1-score. Following model selection, further improvements were pursued through hyperparameter optimization and decision threshold tuning to enhance both predictive performance and operational safety. The following subsections detail the evaluation process, optimization strategies, and the resulting performance of the selected model.

6.1.1 Model Selection

Since the dataset used in this study consists of tabular data, four machine learning models were evaluated: `Random Forest`, `XGBoost`, `LightGBM`, and `CatBoost`. These models were selected due to their strong performance on tabular classification tasks.

All models were trained and evaluated using the same dataset in order to ensure a fair comparison. Their performance was assessed using four standard evaluation metrics: accuracy, precision, recall, and F1-score.

These metrics are calculated based on the following classification outcomes:

- **True Positive (TP):** The model correctly predicts a positive instance.
- **True Negative (TN):** The model correctly predicts a negative instance.
- **False Positive (FP):** The model incorrectly predicts a positive instance when it is actually negative.
- **False Negative (FN):** The model incorrectly predicts a negative instance when it is actually positive.

Using these values, the evaluation metrics are defined as follows:

- **Accuracy**: Measures the proportion of correctly classified instances among all predictions.
$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$
- **Precision**: Measures how many of the predicted positive instances are actually positive.
$$Precision = \frac{TP}{TP + FP}$$

- **Recall**: Measures how many of the actual positive instances were correctly identified by the model.

$$Recall = \frac{TP}{TP + FN}$$

- **F1-score**: The harmonic mean of precision and recall, providing a balance between the two metrics.

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

By comparing these metrics across the four models, the most suitable model for the task was identified.

Class	Precision	Recall	F1-score	Support
0	0.804	0.757	0.780	325
1	0.794	0.835	0.814	364
Accuracy			0.798	689
Macro Avg	0.799	0.796	0.797	689
Weighted Avg	0.799	0.798	0.798	689

Table 6.1: Random Forest Model Evaluation Results

Class	Precision	Recall	F1-score	Support
0	0.818	0.763	0.790	325
1	0.801	0.849	0.824	364
Accuracy			0.808	689
Macro Avg	0.809	0.806	0.807	689
Weighted Avg	0.809	0.808	0.808	689

Table 6.2: XGBoost Model Evaluation Results

Class	Precision	Recall	F1-score	Support
0	0.810	0.735	0.771	325
1	0.782	0.846	0.813	364
Accuracy			0.794	689
Macro Avg	0.796	0.791	0.792	689
Weighted Avg	0.795	0.794	0.793	689

Table 6.3: LightGBM Model Evaluation Results

Class	Precision	Recall	F1-score	Support
0	0.819	0.726	0.770	325
1	0.778	0.857	0.816	364
Accuracy			0.795	689
Macro Avg	0.799	0.792	0.793	689
Weighted Avg	0.798	0.795	0.794	689

Table 6.4: CatBoost Model Evaluation Results

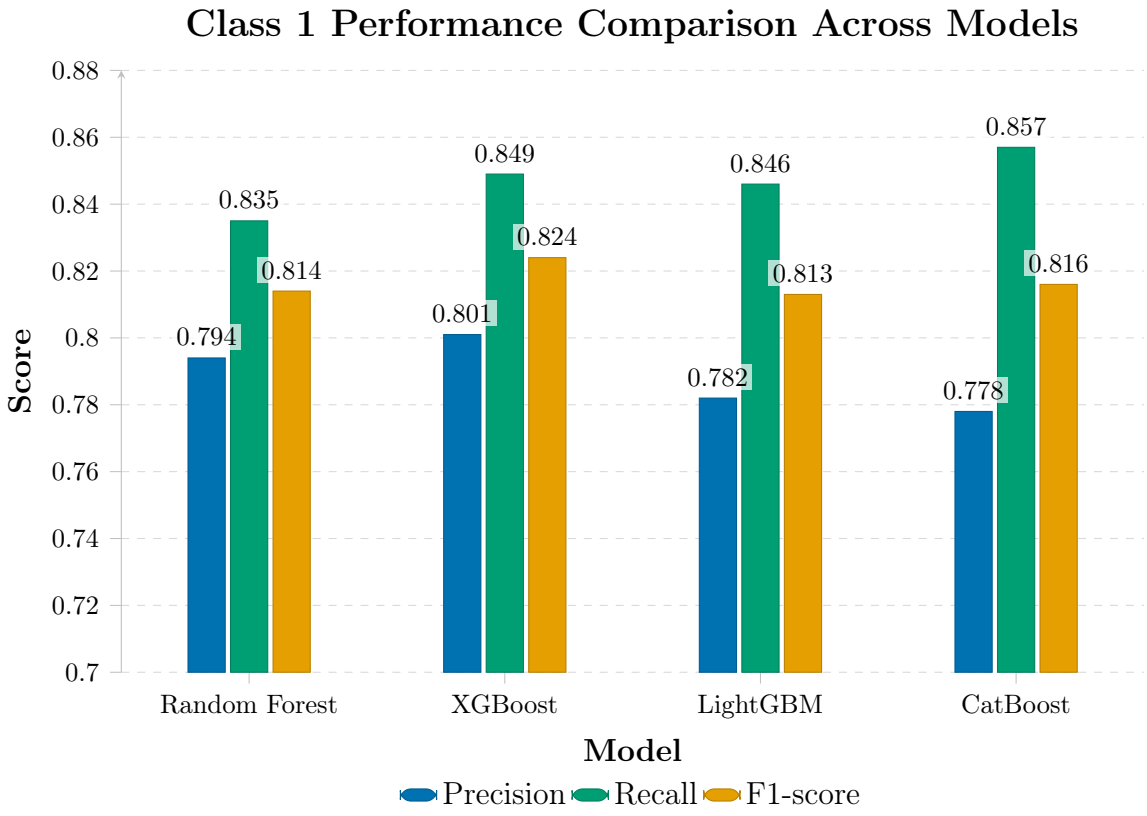


Figure 6.1: Detailed performance comparison for the positive class (Class 1) using Random Forest, XGBoost, LightGBM, and CatBoost models. The chart visualizes the Precision, Recall, and F1-score values as extracted from the model evaluation results, with exact values displayed above each bar

Based on the evaluation results summarized above, all four models achieved reasonably high accuracy and F1-scores, establishing a strong predictive baseline. After a comprehensive evaluation of `Random Forest`, `XGBoost`, `LightGBM`, and `CatBoost`, `XGBoost` emerges as the most effective model for this classification task. It achieved the highest

overall accuracy at 80.8%, marginally outperforming `Random Forest` (79.8%), `CatBoost` (79.5%), and `LightGBM` (79.4%). Furthermore, `XGBoost` demonstrated superior predictive balance across both classes, yielding the highest F1-scores for Class 0 (0.790) and Class 1 (0.824), alongside the highest macro and weighted averages. Given its consistent superiority in precision, F1-score, and overall accuracy, `XGBoost` is the best model choice for this dataset.

6.1.2 Hyperparameter Optimization

To improve the performance of the classification model, hyperparameter tuning was performed using Optuna. Optuna is an automatic hyperparameter optimization framework that efficiently searches the parameter space to identify the optimal configuration for a machine learning model. It uses adaptive sampling strategies and pruning techniques to reduce unnecessary computations and focus on promising parameter combinations.

During the optimization process, 500 trials were executed where different hyperparameter configurations were evaluated. The objective function returned the F1-score of the model on the validation dataset:

```
f1_score(y_valid, preds, pos_label=1)
```

The F1-score was selected as the optimization metric because it provides a balanced evaluation of precision and recall, ensuring that the model performs well in both correctly identifying positive samples and avoiding false positive predictions. The parameter `pos_label=1` ensures that the evaluation focuses on the positive class, which represents the target class of interest in the classification task.

Class	Precision	Recall	F1-score	Support
0	0.854	0.754	0.801	325
1	0.801	0.885	0.841	364
Accuracy			0.823	689
Macro Avg	0.827	0.819	0.821	689
Weighted Avg	0.826	0.823	0.822	689

Table 6.5: Final Model Evaluation Metrics

The performance of the baseline XGBoost model and the optimized model obtained through Optuna hyperparameter tuning is presented in Tables 6.2 and 6.5. The optimized model achieved a slight improvement in overall performance, with accuracy increasing from 0.808 to 0.823 and the F1-score for the positive class improving from 0.824 to 0.841. Recall for successful grasps also increased from 0.849 to 0.885.

However, the overall improvement is relatively modest. This is likely due to the limited size of the dataset (3446 samples), which restricts the potential gains from extensive hyperparameter optimization. Nevertheless, the optimized model provides slightly more reliable predictions while maintaining balanced performance across both classes.

To further evaluate the performance of the optimized XGBoost model, three complementary figures were generated: the ROC curve, the precision-recall curve, and the confusion matrix.

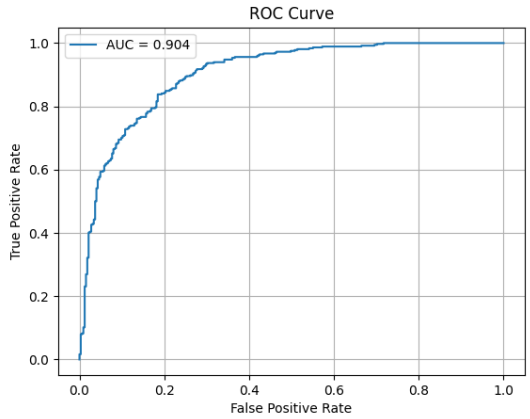


Figure 6.2: ROC curve for the optimized XGBoost model. The AUC measures the model's ability to distinguish successful from failed grasps, with higher values indicating better class separation

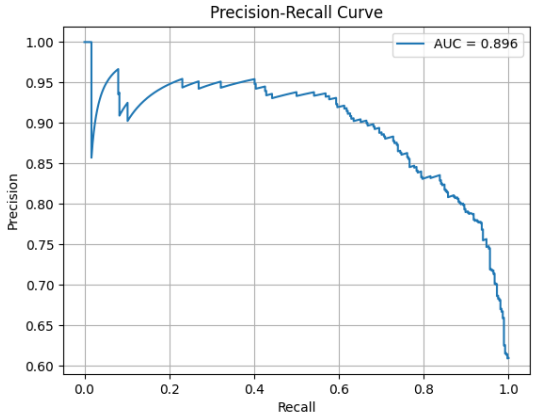


Figure 6.3: Precision-Recall curve for the optimized XGBoost model. This curve shows the trade-off between precision and recall for different probability thresholds

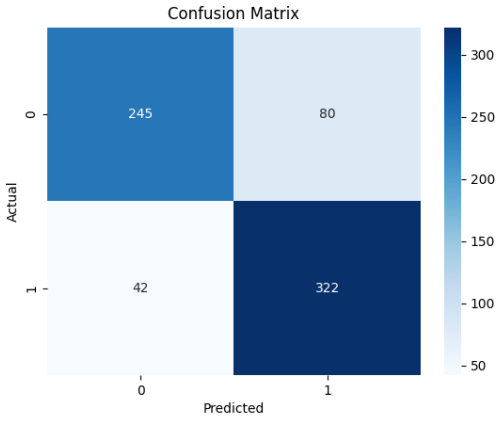


Figure 6.4: Confusion matrix for the optimized XGBoost model on the test set. True Positives (TP) and True Negatives (TN) indicate correctly classified grasps, while False Positives (FP) and False Negatives (FN) indicate misclassifications

6.1.3 Decision Threshold Optimization

In grasp prediction, false positives (FP) are particularly costly: a grasp incorrectly predicted as feasible may lead the robot to attempt a failed or unsafe maneuver, potentially causing collisions, dropped objects, or damage to the robot. In contrast, false negatives (FN) merely result in missed grasp opportunities, which are less critical.

To account for this imbalance, the decision threshold was optimized rather than using the default 0.5. The threshold was selected by maximizing the F_β score with $\beta = 0.5$, emphasizing precision over recall:

$$F_\beta = (1 + \beta^2)\frac{\text{Precision} \cdot \text{Recall}}{\beta^2 \cdot \text{Precision} + \text{Recall}}.$$

Candidate thresholds between 0 and 1 were evaluated on the test predictions, and the threshold yielding the highest $F_{0.5}$ score (0.836) was chosen as the operational boundary. Table 6.6 summarizes the model performance with this optimized threshold.

Class	Precision	Recall	F1-score	Support
0	0.746	0.892	0.812	325
1	0.883	0.728	0.798	364
Accuracy			0.806	689
Macro Avg	0.814	0.810	0.805	689
Weighted Avg	0.818	0.806	0.805	689

Table 6.6: Final evaluation metrics for the optimized XGBoost model using a threshold of 0.836. Emphasizing precision reduces unsafe grasp attempts while slightly decreasing recall

Shifting the threshold from 0.5 to 0.836 makes the model more conservative before classifying a grasp as feasible. Precision for successful grasps (Class 1) increased to 0.883, indicating that predicted successful grasps are highly reliable. As a trade-off, recall for Class 1 dropped to 0.728, meaning some viable grasps are intentionally rejected to minimize unsafe attempts.

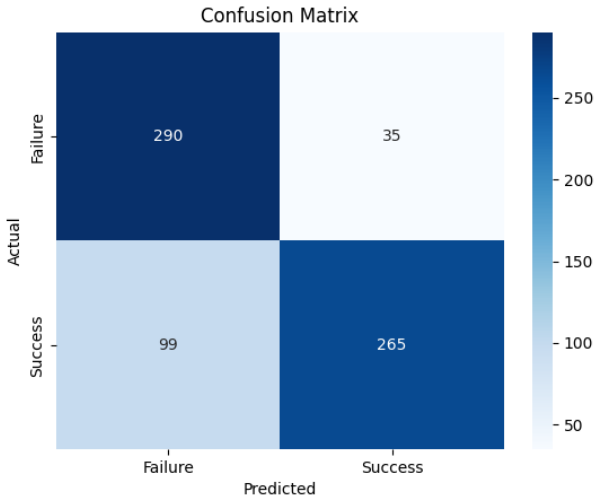


Figure 6.5: Confusion matrix for the threshold-optimized XGBoost model (threshold = 0.836) on the test set. This visualization highlights the effect of the optimized decision threshold in reducing unsafe grasp attempts

Comparing the confusion matrices of the baseline model (threshold = 0.5, Figure 6.4) and the threshold-optimized model (Figure 6.5) highlights the practical benefits of this approach. By shifting the decision boundary, false positives decreased significantly from 80 to 35, effectively preventing 45 potentially catastrophic grasp attempts. As an expected trade-off for this conservative, safety-first behavior, false negatives increased from 42 to

99. Consequently, while true positives decreased from 322 to 265, the system demonstrated an improved capacity to reliably reject unsafe grasps, aligning perfectly with the strict safety requirements of physical robotic manipulation.

The original motivation for exploring an optimized decision threshold was to investigate whether a more conservative boundary could effectively filter out low-confidence grasps compared to the default 0.5 threshold. In practice, however, the system is required to consistently provide an actionable command to the robotic arm. To ensure continuous operation, grasp selection relies on choosing the candidate with the highest predicted probability of success (i.e., the `argmax` of the model output), rather than applying a strict binary filter. Because shifting the decision threshold only affects the final classification and does not alter the underlying probability distribution, the relative ranking of the candidates remains identical. Consequently, whether the default 0.5 or the optimized 0.836 threshold is applied conceptually, the `argmax` operation consistently selects the exact same grasp. Therefore, while optimizing the threshold provides a valuable theoretical framework for future conditional safety checks, implementing a shifted decision boundary offers no practical difference for the current selection pipeline.

6.1.4 Comparative Analysis in Grasping Trials

To evaluate the real-world performance of the proposed selection pipeline, a series of 30 physical robotic grasping trials were conducted. The trials were divided into two categories to test robustness: 10 trials in simple scene, and 20 trials in complex, cluttered scene requiring precise collision avoidance. Each scene was evaluated using two distinct grasp selection methods:

- **Baseline (Without ML):** The system selects the grasp candidate with the highest default geometric or heuristic score generated by the baseline grasp generator (i.e., GPD).
- **Proposed (With ML):** The system selects the grasp candidate with the highest predicted probability of success (i.e., the `argmax`) output by the trained machine learning model.

Scene Type	Trial	Baseline (Without ML)	Proposed (With ML)	Same Grasp?
Simple	1	Success	Success	No
	2	Success	Success	No
	3	Fail	Success	No
	4	Fail	Success	No
	5	Success	Success	No
	6	Fail	Success	No
	7	Fail	Success	No
	8	Success	Success	Yes
	9	Success	Success	No
	10	Success	Success	No
Simple Total		6 / 10 (60%)	10 / 10 (100%)	
Complex	11	Success	Success	No
	12	Fail	Success	No
	13	Fail	Success	No
	14	Fail	Fail	No
	15	Success	Success	No
	16	Fail	Success	No
	17	Fail	Fail	Yes
	18	Success	Success	No
	19	Fail	Success	No
	20	Fail	Fail	Yes
	21	Success	Success	Yes
	22	Fail	Success	No
	23	Fail	Fail	No
	24	Success	Success	Yes
	25	Fail	Success	No
	26	Fail	Success	No
	27	Success	Success	No
	28	Fail	Success	No
	29	Fail	Success	No
	30	Fail	Success	No
Complex Total		5 / 20 (35%)	16 / 20 (80%)	
Overall Total Success Rate:		**11 / 30 (36.6%)**	**26 / 30 (86.6%)**	

Table 6.7: Results of 30 physical grasp trials comparing the baseline selection against the proposed ML probability ranking. The last column indicates whether the baseline and ML methods chose the same grasp

The results in Table 6.7 show a clear improvement when using the proposed ML–based grasp ranking compared to the baseline method. In the simple scene, the ML approach achieved a 100% success rate (10/10) compared to 60% (6/10) for the baseline, while in the complex scene the baseline dropped to 35% (5/20) whereas the ML method reached 80% (16/20). Additionally, the two approaches often selected different grasp candidates, even when both succeeded or failed. This suggests that the ML model evaluates grasp configurations differently and can identify alternative grasps that lead to more successful manipulations.

6.2 Evaluation of Gaze-Based Interaction Methods

6.2.1 Experimental Setup

To evaluate the gaze-based interaction techniques, a user study was conducted comparing two methods for UI button activation: blink-based and dwell-time. The evaluation focused on usability, efficiency, and perceived workload during interaction tasks, such as object selection and placement confirmation, within an augmented reality environment.

Participants performed a series of pick-and-place tasks using the AR interface. The objective of each task was to select a target object using gaze interaction and place it at a designated location within the scene. The robotic arm was not included during the evaluation; instead, the study focused exclusively on the interaction mechanisms used for object selection and placement.

Each participant completed a total of 20 trials, consisting of 10 trials with the blink-based interaction method and 10 trials with the dwell-time interaction method. The same set of task scenarios was used for both interaction techniques to allow direct comparison of performance across methods. However, the order of trials was randomized to reduce learning effects and ordering bias. In addition, the order in which participants used the two interaction techniques was counterbalanced across participants so that some participants began with blink-based interaction while others started with dwell-time interaction.

During the trials, participants interacted with two predefined experimental scenes developed for the study: a simple scene and a complex scene. These scenes were designed to represent different levels of visual and spatial difficulty. The simple scene contained fewer surrounding objects and minimal visual clutter, allowing participants to easily focus on the target object. In contrast, the complex scene included additional surrounding objects and visual elements, increasing the level of visual interference and making gaze-based object targeting more challenging. Across trials, different virtual objects were used as targets to introduce variation in object shape, size, and spatial placement. This design ensured that participants interacted with a variety of targets while maintaining consistent experimental conditions across both interaction techniques.

During each trial, the system recorded quantitative performance metrics. The primary metric was task completion time, defined as the time required for the participant to successfully select the target object and place it at the specified location. In addition to completion time, the system also recorded whether the task was successfully completed in order to capture potential selection errors or interaction failures. Immediately after each trial, participants were asked to answer the **Single Ease Question (SEQ)**, which measures perceived task difficulty on a seven-point scale.

To complement the per-task subjective ratings, overall usability and perceived workload were assessed using standardized questionnaires. After completing the set of trials for each interaction method, participants completed the **UMUX-Lite (Usability Metric for User Experience – Lite)** questionnaire to evaluate perceived usability of the interaction technique. Additionally, participants completed the **NASA Task Load Index (NASA-TLX)** to measure perceived workload across multiple dimensions, including mental demand, effort, and frustration.

In addition to these standardized instruments, participants were asked to provide qualitative feedback regarding their experience with the two gaze-based interaction methods. Specifically, participants were asked to rate how the interaction speed of each method (blink-based and dwell-time selection) felt during use. Finally, participants were asked to indicate which interaction technique they preferred overall.

Together, the collected metrics enable a comprehensive comparison of the two gaze-based interaction techniques in terms of efficiency, usability, and user workload within the augmented reality interface.

Before the formal evaluation, participants were allowed to interact freely with both selection methods in order to familiarize themselves with the system. Once they indicated readiness, the experimental procedure commenced.

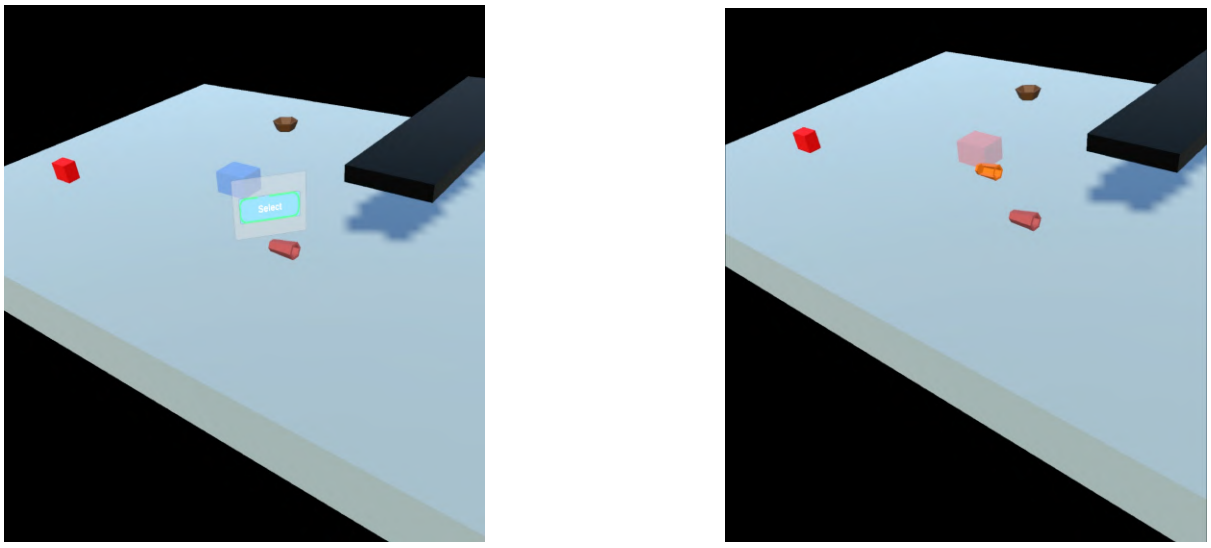


Figure 6.6: During the familiarization phase, participants could freely select objects and place them inside the semi-transparent blue cube, which indicates the target placement location

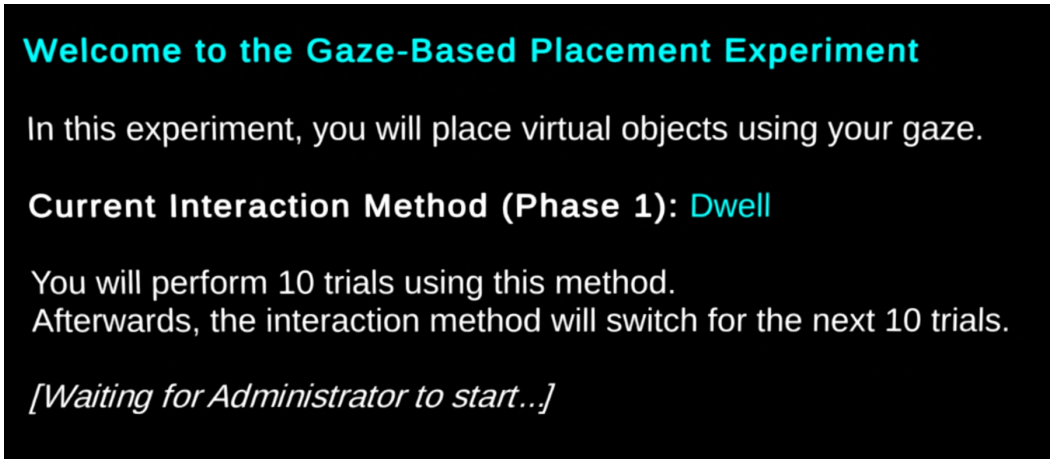


Figure 6.7: Initial instruction message presented to the participant, explaining the experiment

Prior to each trial, participants were informed about the upcoming task and the target object to be selected. Once they confirmed their readiness, the trial began.

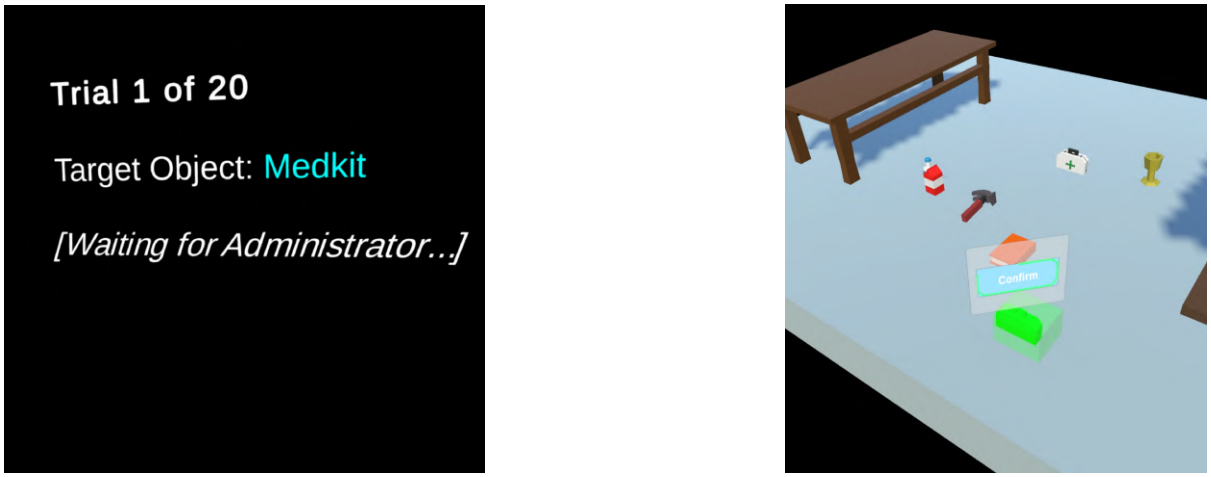


Figure 6.8: Illustration of a trial performed by a participant

In order to control the sequence of trials and systematically record participant responses to the **Single Ease Question (SEQ)**, an external Python-based system was developed using Google Colab. This system communicated with the AR application via UDP, enabling real-time coordination of trial progression and reliable collection of subjective ratings after each task.

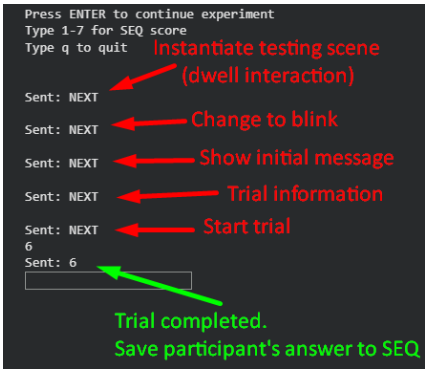


Figure 6.9: Google Colab interface used to control the experimental flow

6.2.2 Participants and Evaluation Metrics

6.2.2.1 Participants

A total of *12* participants took part in the study. Participants were recruited from the university community and had varying levels of experience with augmented reality systems. All participants reported normal or corrected-to-normal vision, which was required for reliable gaze tracking with the Magic Leap 2 device. Participation was voluntary and all participants completed the full set of experimental trials.

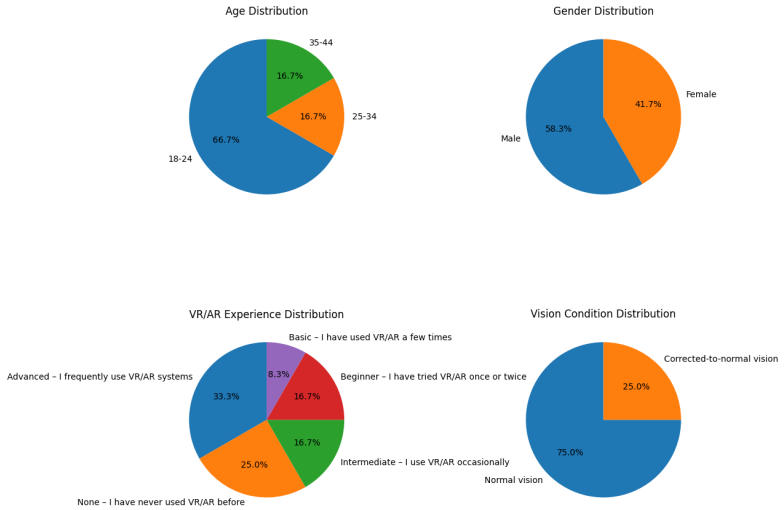


Figure 6.10: Pie charts depicting the distribution of participants by age group, gender, vision condition, and prior experience with AR/VR

6.2.2.2 Evaluation Metrics

For the evaluation of this experiment, both subjective questionnaires (**UMUX-Lite, NASA-TLX, SEQ**) and objective system logs (completion time, success rates) were analyzed to compare the Blink and Dwell interaction techniques.

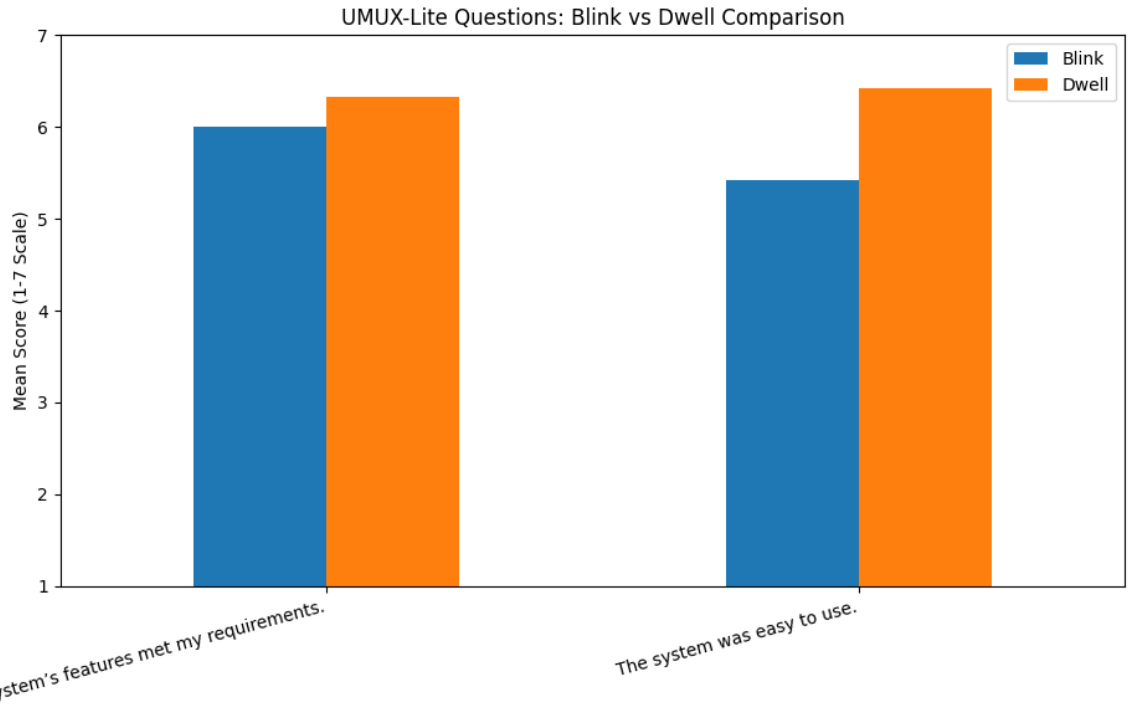


Figure 6.11: Mean scores of UMUX-Lite questions comparing Blink and Dwell interaction methods

Results from the **UMUX-Lite** questionnaire reveal that while both methods were effective, the Dwell technique was perceived as more usable. Both interaction techniques scored highly regarding the system's features meeting user requirements (Dwell: $\mathbf{M = 6.33}$; Blink: $\mathbf{M = 6.00}$). However, a more notable difference was observed in perceived ease of use, where participants rated Dwell ($\mathbf{M = 6.42}$) much higher than Blink ($\mathbf{M = 5.42}$), suggesting it provided a more natural interaction.

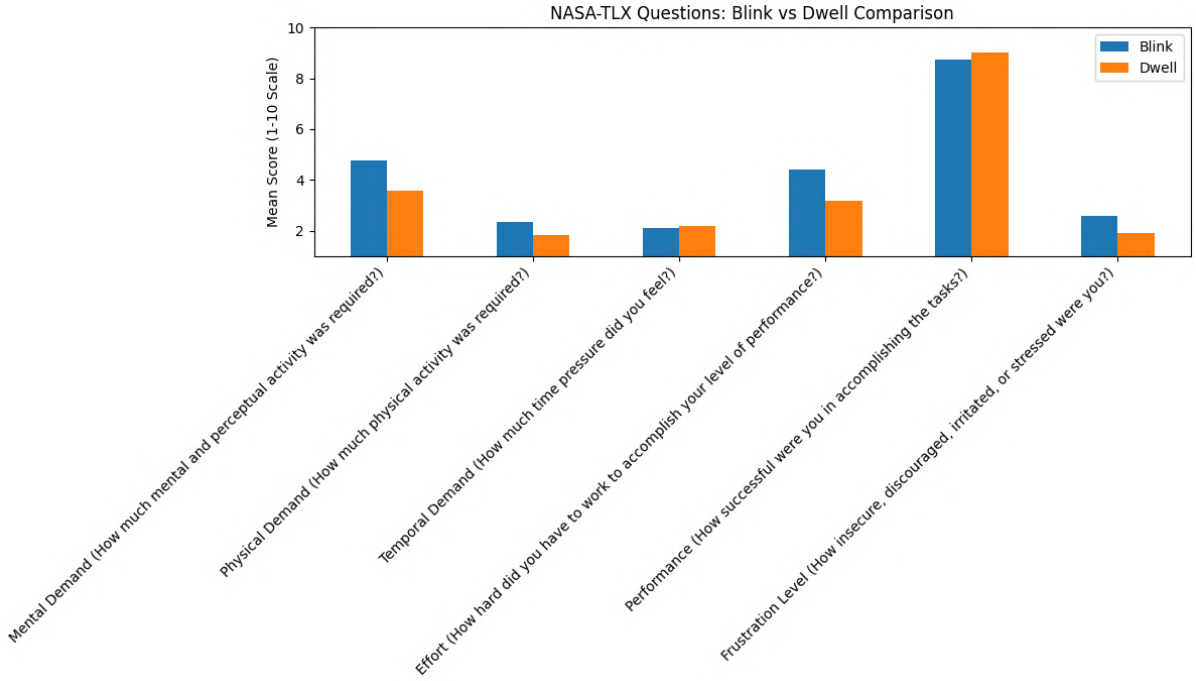


Figure 6.12: Mean NASA-TLX scores across all dimensions for Blink and Dwell interaction methods

NASA-TLX Subscale	Blink Mean	Dwell Mean
Mental Demand	4.75	3.58
Physical Demand	2.33	1.83
Temporal Demand	2.08	2.17
Effort	4.42	3.17
Performance	8.75	9.00
Frustration Level	2.58	1.92

Table 6.8: NASA-TLX Mean Scores for Blink and Dwell Interaction Methods

The **NASA-TLX** results (Table 6.8) support these findings, indicating that Dwell generally imposed a lower cognitive and physical load. The most notable differences were observed in the Mental Demand and Effort subscales. Blink required noticeably higher mental demand ($\mathbf{M = 4.75}$) and effort ($\mathbf{M = 4.42}$) than Dwell ($\mathbf{M = 3.58}$ and $\mathbf{M = 3.17}$, respectively), likely due to the conscious concentration needed to execute intentional blinks. Consequently, participants reported higher frustration levels when using the Blink method ($\mathbf{M = 2.58}$ vs. $\mathbf{M = 1.92}$).

Regarding task speed, participants evaluated the perceived speed of both techniques on a 7-point scale (where 4 represents a balanced, optimal speed). Interestingly, even though the programmed activation time was identical for both methods, users perceived Dwell's speed to be near the ideal midpoint (**M = 4.67**), whereas Blink was perceived as noticeably slower (**M = 3.17**). This inconsistency suggests that the cognitive load and error recovery associated with blinking distorted users' perception of time. Consequently, an overwhelming majority of participants (**11 out of 12**) selected Dwell as their overall preferred interaction method.

While the questionnaire data established a clear user preference for the Dwell method based on perceived workload and ease of use, it is crucial to verify these subjective experiences against the actual system logs. To determine whether the perceived speed of the interactions aligned with reality, the primary objective metric evaluated was task completion time.

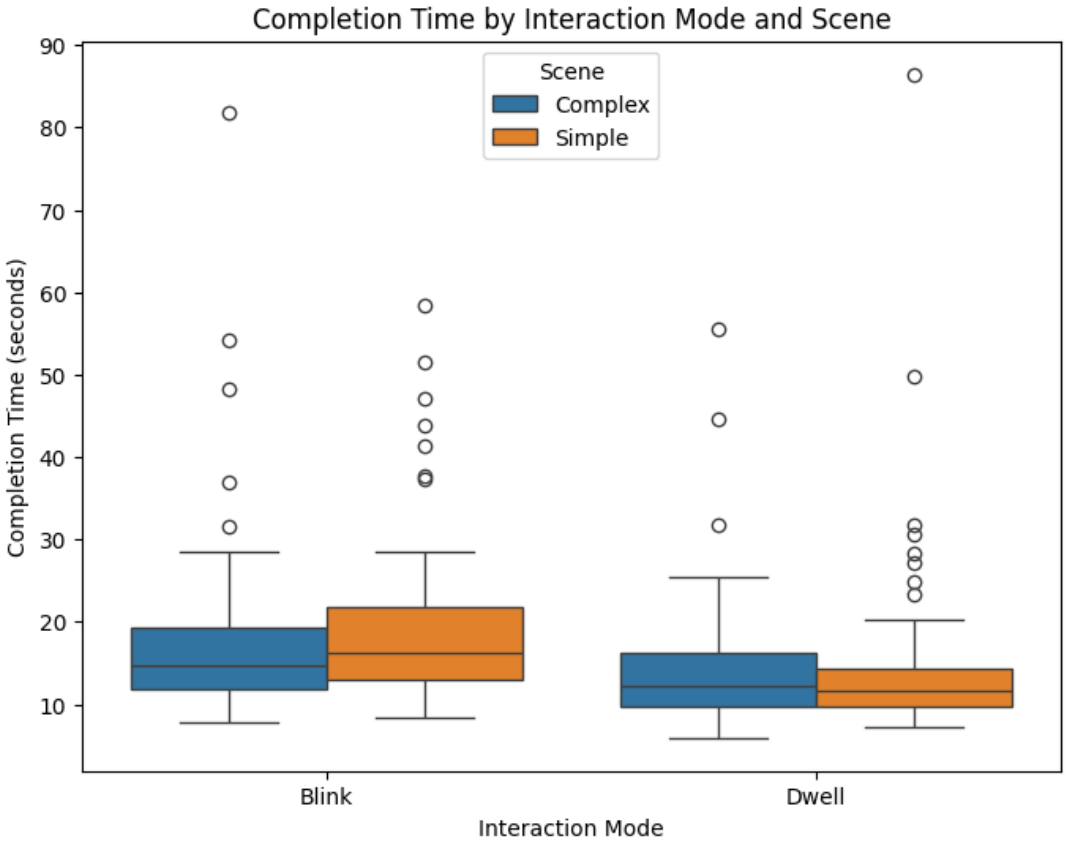


Figure 6.13: Box plot of objective task completion times (in seconds) for the Blink and Dwell interaction methods across Simple and Complex scenes

Objective system logs confirm the users' subjective perceptions. Figure 6.13 illustrates that Dwell consistently resulted in faster completion times. The medians for Dwell were noticeably lower (**11.7 to 12.2 seconds**) compared to Blink (**14.7 to 16.3 seconds**). Furthermore, the interquartile range is generally tighter for Dwell, indicating that user performance was much more consistent between trials. While upper outliers exist across all conditions—often caused by poor eye calibration—the overall data strongly validates that Dwell was the faster modality.

This performance consistency is reflected at the trial level via the **Single Ease Question (SEQ)** scores (Table 6.9). Across all individual trials, Dwell was rated notably easier to use (**Overall Mean = 6.13**) than Blink (**Overall Mean = 5.54**). Breaking the data down by scene complexity reveals that environmental clutter had a negligible impact on perceived ease, as scores remained stable across both Simple and Complex environments. This shows that the difficulty users experienced was caused by the blinking method itself, not by the visual complexity of the scene.

Interaction Mode	Overall Mean	Simple Scene	Complex Scene
Blink	5.54	5.50	5.58
Dwell	6.13	6.15	6.10

Table 6.9: Average Single Ease Question (SEQ) Scores per Interaction Mode and Scene

Despite the differences in speed and effort, objective task accuracy was remarkably high for both methods (Table 6.10). Both achieved a flawless **100%** success rate for object selection, and placement success rates remained robust (**Blink: 96.67%, Dwell: 94.17%**). This demonstrates that gaze-based interaction, regardless of the activation trigger, is a highly accurate modality for 3D manipulation.

Interaction Mode	Selection Success Rate	Placement Success Rate
Blink	100.00%	96.67%
Dwell	100.00%	94.17%

Table 6.10: Task Success Rates for Selection and Placement by Interaction Mode

Finally, to synthesize system performance and user experience, a correlation analysis compared objective completion times against subjective **SEQ** scores for every trial (Figure 6.14). A strong negative correlation was observed, with the steep negative trendline effectively modeling the concept of interaction friction. As the time required to complete a task increased, perceived ease dropped substantially. This successfully bridges the gap between the system logs and user questionnaires, proving that even minor interaction delays in AR heavily penalize the user experience.

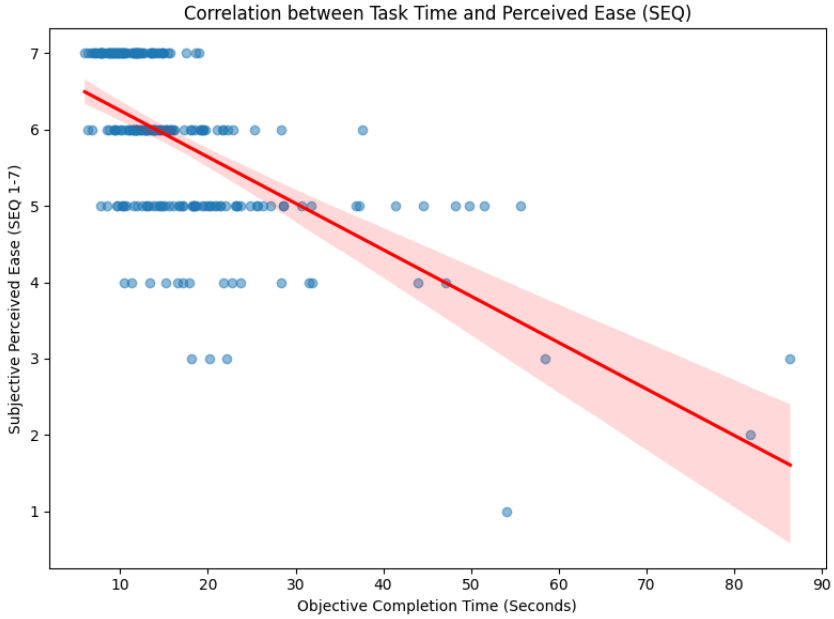


Figure 6.14: Scatter plot illustrating the correlation between objective task completion time and subjective perceived ease (SEQ). The red trendline indicates a strong, statistically significant negative correlation

Overall, qualitative user feedback provided essential context regarding the interaction modes. A frequent point of friction was the placement confirmation button, which appeared after locking an object's position. Many participants found this step redundant. In the Blink condition specifically, attempting to interact with this button often led to errors, as users unintentionally unlocked the placement position instead of confirming it. Because the initial object placement naturally requires sufficient time and intentionality to avoid accidental triggers, users noted that an extra confirmation button added unnecessary cognitive load rather than value. Furthermore, technical limitations—such as poor eye calibration, and inconsistent marker tracking—introduced external frustrations that negatively impacted the overall user experience.

6.3 Evaluation of the Whole System

6.3.1 Experimental Setup

To evaluate the performance of the complete system, an additional user study was conducted in which participants performed pick-and-place tasks using the full gaze-controlled robotic manipulation pipeline. In contrast to the previous evaluation that focused solely on comparing gaze-based interaction techniques, this experiment assessed the usability and effectiveness of the entire system, including gaze-based object selection, task command generation, and robotic arm execution.

Participants interacted with the augmented reality interface using the Magic Leap 2 headset while controlling the robotic arm to manipulate objects in the workspace. During each task, participants selected a target object using gaze interaction and specified the desired placement location within the scene. Once the object and placement location were selected, the robotic arm autonomously executed the pick-and-place operation.

The tasks were performed in the same experimental scenes described earlier in this thesis: a simple scene and a complex scene. Each participant completed a total of six pick-and-place tasks, consisting of three tasks in the simple scene and three tasks in the complex scene. This setup allowed the evaluation of the complete system under different workspace conditions while maintaining the same interaction workflow.

Following the completion of the pick-and-place tasks in each scene, participants were asked to complete two standardized questionnaires: the **Usability Metric for User Experience (UMUX-Lite)** and the **Single Ease Question (SEQ)**. These questionnaires were completed after each scene to evaluate both the system's overall perceived usability (**UMUX-Lite**) and the specific ease of task completion (**SEQ**) under varying levels of workspace complexity.

6.3.2 Participants and Results

Participants were the same as those who took part in the previously analyzed experiment (Figure 6.10). The evaluation results gathered from the **UMUX-Lite** and **SEQ** metrics across both the simple and complex scenes are presented in Figure 6.15.

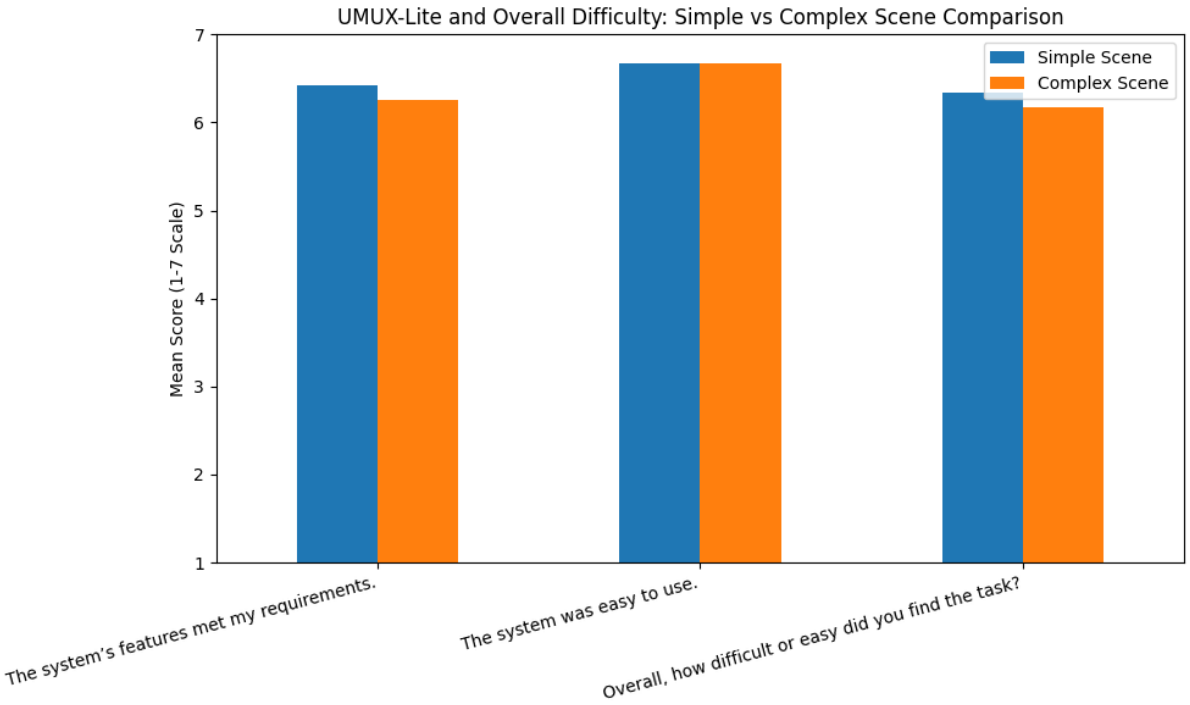


Figure 6.15: Subjective evaluation results for the complete system. The chart presents the Usability Metric for User Experience (UMUX-Lite) and Single Ease Question (SEQ) scores reported by participants following the pick-and-place tasks in both the simple and complex scenes

Questionnaire Item	Simple Scene	Complex Scene
UMUX-Lite		
The system's features met my requirements.	6.42	6.25
The system was easy to use.	6.67	6.67
SEQ		
Overall, how difficult or easy did you find the task?	6.33	6.17

Table 6.11: Mean participant scores for the UMUX-Lite and SEQ items across the simple and complex scenes. All items were evaluated on a 7-point Likert scale

To facilitate comparison with standard usability benchmarks in human-computer interaction, the raw questionnaire scores were converted into a 0 to 100 metric. The **UMUX-Lite** serves as a concise, two-item measure of perceived usefulness and perceived ease-of-use. To benchmark these results against the widely used **System Usability Scale**

(SUS), the two-item interpolation method was applied. For a 7-point Likert scale, this standardized score is calculated using the following formula:

$$Score = \left(\frac{\text{Ease} + \text{Useful}}{2} - 1 \right) \times \frac{100}{6}$$

Applying this transformation, the gaze-controlled system achieved a standardized usability score of **92.36** for the simple scene and **90.97** for the complex scene. In the context of **SUS** evaluations, a score of 68 is generally considered average, while scores above 80 indicate excellent usability. These exceptionally high results demonstrate that users found the system highly intuitive and effective. Furthermore, the minimal difference in overall scores between the two conditions demonstrates that the system's usability remains highly robust, even when operating in a complex and cluttered workspace. However, the slight decrease observed in the "system's features met my requirements" score for the complex scene can be directly attributed to the increased difficulty of robotic grasping in such environments. In cluttered workspaces, it is inherently more common for the perception and planning algorithms to fail to identify a valid grasp pose due to spatial constraints, nearby obstacles, and occlusions. These occasional grasp generation failures likely account for the minor reduction in the perceived fulfillment of system requirements.

In conclusion, the subjective and objective evaluations demonstrate that the complete gaze-controlled robotic manipulation system is highly effective and overall well-received by users. However, it is important to note that the usability scores were marginally impacted by occasional technical instabilities during the experiment. Specifically, intermittent and unprompted disconnections within the Robot Operating System (ROS) framework occurred during some trials, leading to minor usability friction that was reflected in slightly lower evaluations from the affected participants. Despite these technical interruptions, qualitative feedback revealed that users highly valued the system's transparency. Participants found the real-time status messages regarding trajectory planning and specific execution failure reasons to be highly informative, as they successfully interpreted the robot's internal state. Finally, while the system's operational accuracy was acknowledged, users indicated a strong preference for faster overall computational performance. Specifically, reducing the latency involved in the grasp pose detection and trajectory planning algorithms would significantly enhance the responsiveness of the human-robot interaction in future iterations.

Chapter 7

Conclusion & Future Work

7.1 Conclusion

This thesis presented the design, implementation, and evaluation of a simulation-based augmented reality framework for gaze-driven robotic manipulation. The system enables hands-free pick-and-place task specification through a Magic Leap 2 headset, allowing users to select objects via gaze dwell or blink interaction, define placement poses through a virtual object clone, and assign motion execution entirely to the robot. By connecting an AR interface built in Unity to a ROS/MoveIt motion planning backend, and augmenting geometry-based grasp generation with a scene-aware XGBoost classifier, the framework bridges high-level human intent with low-level robotic autonomy in a single closed-loop pipeline.

The evaluation confirmed the framework's effectiveness across all three components. Grasp selection improved dramatically over the geometric baseline demonstrating that scene-aware ML evaluation is essential in both simple and complex environments. Gaze interaction proved both accurate and low-effort, with dwell-time preferred by nearly all participants for its lower cognitive load and faster completion times. End-to-end system usability exceeded 90 on a SUS-equivalent scale and remained stable across both scene complexities, confirming that the interface does not degrade meaningfully under increased workspace clutter.

Together, these results validate the central premise of this work: that gaze-based AR interaction can serve as an accurate, low-effort, and intuitive interface for specifying robotic manipulation tasks, without requiring any physical controller. Although the evaluation was conducted in simulation and on a small participant sample, the consistency of both the quantitative metrics and the qualitative feedback supports the broader applicability of this approach to human–robot collaboration.

7.2 Future Work

Although the proposed framework demonstrates promising results, several directions for future research could further enhance the capabilities and applicability of the system. One important extension involves transitioning from the simulation-based environment to a real-world robotic system. Integrating the proposed AR interface with a physical robotic manipulator would allow evaluation under realistic conditions, including perception noise, calibration errors, and hardware constraints. Such experiments would provide valuable insights into the practical deployment of gaze-driven robotic manipulation systems. This sim-to-real transition can be directly facilitated by the Unity Robotics Hub's ROS communication pipelines and implemented using accessible, open-source hardware like the 3D-printable Niryo One robotic arm.

A second avenue for improvement concerns the perception and grasping pipeline. In the current system, point cloud data is assumed to be available for grasp generation. Future work could integrate real-time perception and object detection techniques to reconstruct object point clouds directly from the environment, enabling the system to operate in dynamic and unstructured scenes. Additionally, more advanced learning-based grasp synthesis approaches could be explored. Deep learning methods trained on large-scale grasp datasets may provide more robust grasp predictions, particularly in cluttered environments or for previously unseen objects.

Further research could also focus on enhancing the interaction and visualization components of the system. The gaze-based interface may be extended through multimodal interaction techniques that combine gaze with modalities such as voice commands, reducing limitations associated with single-modality input and enabling more flexible task specification. Moreover, improved visualization within the augmented reality environment—such as displaying predicted grasp poses, planned robot trajectories, or potential collision risks before execution—could provide users with clearer insight into the robot's decision-making process.

Finally, future work may explore adaptive interaction models that learn from user behavior over time. By dynamically adjusting parameters such as gaze dwell duration, the system could better accommodate individual user preferences.

Taken together, these extensions would move the system from a simulation-based proof of concept toward a deployable framework capable of supporting real-world gaze-driven manipulation, broadening its applicability across domains where hands-free human–robot collaboration is of practical significance, such as assistive robotics and industrial automation.

Bibliography

[1] R. Suzuki, A. Karim, T. Xia, H. Hedayati, *et al.*, "Augmented reality and robotics: A survey and taxonomy for AR-enhanced human-robot interaction and robotic interfaces," in *Proc. CHI Conf. Human Factors in Computing Systems (CHI)*, 2022. doi: 10.1145/3491102.3517719.

[2] Z. Wang, H. Wang, H. Yu, and F. Lu, "Interaction with gaze, gesture, and speech in a flexibly configurable augmented reality system," *IEEE Trans. Human-Mach. Syst.*, vol. 51, no. 5, pp. 524–534, 2021. doi: 10.1109/THMS.2021.3097973.

[3] I. B. Adhanom, P. MacNeilage, and E. Folmer, "Eye Tracking in Virtual Reality: A Broad Review of Applications and Challenges," *Virtual Reality*, vol. 27, pp. 1481–1505, 2023. doi: 10.1007/s10055-022-00738-z.

[4] R. Shadiev and D. Li, "A review study on eye-tracking technology usage in immersive virtual reality learning environments," *Computers & Education*, vol. 196, p. 104681, 2023. doi: 10.1016/j.compedu.2022.104681.

[5] L. Muñoz-Saavedra, L. Miró-Amarante, and M. Domínguez-Morales, "Augmented and Virtual Reality Evolution and Future Tendency," *Applied Sciences*, vol. 10, no. 1, p. 322, 2020. doi: 10.3390/app10010322.

[6] F. Arena, M. Collotta, G. Pau, and F. Termine, "An overview of augmented reality," *Computers*, vol. 11, no. 2, p. 28, 2022. doi: 10.3390/computers11020028.

[7] P. Majaranta and A. Bulling, "Eye Tracking and Eye-Based Human–Computer Interaction," in *Advances in Physiological Computing*, S. H. Fairclough and K. Gilleade, Eds. London, U.K.: Springer, 2014, pp. 39–65. doi: 10.1007/978-1-4471-6392-3_3.

[8] M. A. Goodrich and A. C. Schultz, "Human–Robot Interaction: A Survey," *Foundations and Trends in Human–Computer Interaction*, vol. 1, no. 3, pp. 203–275, 2008. doi: 10.1561/1100000005.

[9] R. J. K. Jacob, "What You Look At is What You Get: Eye Movement-Based Interaction Techniques," in *Proc. SIGCHI Conf. Human Factors in Computing Systems (CHI)*, pp. 11–18, 1990. doi: 10.1145/97243.97246.

[10] L. Pagliarini and H. H. Lund, "The future of robotics technology," *J. Robot. Netw. Artif. Life*, vol. 3, no. 4, pp. 270–273, 2017. doi:10.2991/jrnal.2017.3.4.12.

[11] A. Fischer-Janzen, T. M. Wendt, and K. Van Laerhoven, "A Scoping Review of Gaze and Eye Tracking-Based Control Methods for Assistive Robotic Arms," *Frontiers in Robotics and AI*, vol. 11, 2024, Art. no. 1326670. doi: 10.3389/frobt.2024.1326670.

[12] J. Weisz, P. K. Allen, A. G. Barszap, and S. S. Joshi, "Assistive grasping with an augmented reality user interface," *The International Journal of Robotics Research*, vol. 36, no. 5–7, pp. 520–538, 2017. doi: 10.1177/0278364917707024.

[13] S. Sadeghnejad, V. S. E. Abadi, and B. Jafari, "Rehabilitation Robotics: History, Applications, and Recent Advances," in *Rehabilitation Robotics*, Elsevier, 2023. doi: 10.1016/B978-0-443-18460-4.00008-1.

[14] A. ten Pas, M. Gualtieri, K. Saenko, and R. Platt, "Grasp pose detection in point clouds," *Int. J. Robot. Res.*, vol. 36, no. 13–14, pp. 1455–1473, 2017. doi: 10.48550/arXiv.1706.09911.

[15] G. Ramiotis and K. Mania, "CONTEXT-GAD: A Context-Aware Gaze Adaptive Dwell model for Gaze-based Selections in XR Environments," in *Proc. ACM Symp. Virtual Reality Software and Technology (VRST)*, 2025, Article no. 35, pp. 1–11. doi: 10.1145/3756884.3766048.

[16] L. Alboul, M. Dimitrova, A. Lekova, V. G. Kaburlasos, and P. Mitrouchev, "Editorial: Emerging Technologies for Assistive Robotics: Current Challenges and Perspectives," *Frontiers in Robotics and AI*, vol. 10, 2023, Art. no. 1288360. doi: 10.3389/frobt.2023.1288360.

[17] J. E. Solanes, A. Muñoz, L. Gracia, A. Martí, V. Girbés-Juan, and J. Tornero, "Teleoperation of Industrial Robot Manipulators Based on Augmented Reality," *The International Journal of Advanced Manufacturing Technology*, vol. 111, pp. 1077–1097, 2020. doi: 10.1007/s00170-020-05997-1.

[18] X. Chen, X. Huang, Y. Wang, and X. Gao, "Combination of Augmented Reality Based Brain-Computer Interface and Computer Vision for High-Level Control of a Robotic Arm," *IEEE Transactions on Neural Systems and Rehabilitation Engineering*, vol. 28, no. 12, pp. 3140–3147, 2020. doi: 10.1109/TNSRE.2020.3038209.

[19] J. Lee, T. Lim, and W. Kim, "Investigating the Usability of Collaborative Robot Control through Hands-Free Operation using Eye Gaze and Augmented Reality," in *Proc. IEEE/RSJ Int. Conf. Intelligent Robots and Systems (IROS)*, 2023, accepted. arXiv:2306.13072. doi: 10.48550/arXiv.2306.13072.

[20] F. Di Stefano, A. Giambertone, L. Salamina, M. Melchiorre, and S. Mauro, "Collaborative Robot Control Based on Human Gaze Tracking," *Sensors*, vol. 25, no. 10, p. 3103, 2025. doi: 10.3390/s25103103.

[21] D. Yu, X. Lu, R. Shi, H.-N. Liang, T. Dingler, E. Velloso, and J. Goncalves, "Gaze-Supported 3D Object Manipulation in Virtual Reality," in *Proc. CHI Conf. Human Factors in Computing Systems (CHI)*, 2021, Art. no. 734, pp. 1–13. doi: 10.1145/3411764.3445343.

[22] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A Next-generation Hyperparameter Optimization Framework," in *Proc. 25th ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining (KDD)*, 2019, pp. 2623–2631. doi: 10.1145/3292500.3330701.

[23] F. Sarri and K. Mania, "Effect of Gaze Visualization on Task Efficiency and User Behavior for Guidance Scenarios in Co-Located AR Collaboration," in *2025 IEEE Conference on Virtual Reality and 3D User Interfaces Abstracts and Workshops (VRW)*, 2025. doi: 10.1109/VRW66409.2025.00120.

[24] M. Singh, J. Kapukotuwa, E. L. S. Gouveia, E. Fuenmayor, Y. Qiao, N. Murry, and D. Devine, "Unity and ROS as a Digital and Communication Layer for Digital Twin Application: Case Study of Robotic Arm in a Smart Manufacturing Cell," *Sensors*, vol. 24, no. 17, p. 5680, 2024. doi: 10.3390/s24175680.

[25] J.-W. Hyung, W. Na, K. Won, and D.-J. Kim, "Robotic arm control by augmented reality-assisted object detection," *Sci. Rep.*, vol. 15, art. no. 35678, 2025. doi: 10.1038/s41598-025-19514-5.

[26] R. R. Baptista, N. R. Gerszberg, R. V. Godoy, and G. J. G. Lahr, "MIHRaGe: A Mixed-Reality Interface for Human-Robot Interaction via Gaze-Oriented Control," arXiv preprint arXiv:2505.03929, 2025.

[27] Z. Lin, Y. Sang, and Y. Ye, "RaycastGrasp: Eye-Gaze Interaction with Wearable Devices for Robotic Manipulation," in *Proc. IEEE Int. Conf. Intelligent Reality (ICIR)*, 2025, accepted. arXiv:2510.22113. doi: 10.48550/arXiv.2510.22113.

[28] H. Zeng, Y. Wang, C. Wu, A. Song, J. Liu, P. Ji, B. Xu, L. Zhu, H. Li, and P. Wen, "Closed-Loop Hybrid Gaze Brain-Machine Interface Based Robotic Arm Control with Augmented Reality Feedback," *Frontiers in Neurorobotics*, vol. 11, p. 60, 2017. doi: 10.3389/fnbot.2017.00060.

[29] D. P. McMullen, et al., "Design and Preliminary Evaluation of an Augmented Reality Interface Control System for a Robotic Arm," *Johns Hopkins APL Technical Digest*, vol. 35, no. 3, pp. 220–230, 2020.

[30] A. Fischer-Janzen, T. M. Wendt, D. Görlich, and K. Van Laerhoven, "Eye-tracking-Driven Shared Control for Robotic Arms: Wizard of Oz Studies to Assess Design Choices," arXiv preprint arXiv:2505.23147, 2025.

[31] D. Morrison, P. Corke, and J. Leitner, "Learning robust, real-time, reactive robotic grasping," *The International Journal of Robotics Research*, vol. 39, no. 2–3, pp. 183–201, 2020. doi: 10.1177/0278364919859066.

[32] H.-S. Fang, C. Wang, M. Gou, and C. Lu, "GraspNet: A Large-Scale Clustered and Densely Annotated Dataset for Object Grasping," arXiv preprint arXiv:1912.13470, 2019.

[33] J. Mahler, J. Liang, S. Niyaz, M. Laskey, R. Doan, X. Liu, J. A. Ojea, and K. Goldberg, "Dex-Net 2.0: Deep Learning to Plan Robust Grasps with Synthetic Point Clouds and Analytic Grasp Metrics," arXiv preprint arXiv:1703.09312, 2017.

[34] J. Li, W. Wu, G. Zhang, Y. Han, and W. Lian, "SAGE: Scene Graph-Aware Guidance and Execution for Long-Horizon Manipulation Tasks," arXiv preprint arXiv:2509.21928, 2025.

[35] J. Bohg, A. Morales, T. Asfour, and D. Kragic, "Data-Driven Grasp Synthesis—A Survey," *IEEE Transactions on Robotics*, vol. 30, no. 2, pp. 289–309, 2014. doi: 10.1109/TRO.2013.2289018.

[36] L. Breiman, "Random Forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001. doi: 10.1023/A:1010933404324.

[37] L. Prokhorenkova, G. Gusev, A. Vorobev, A. V. Dorogush, and A. Gulin, "CatBoost: Unbiased Boosting with Categorical Features," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, pp. 6639–6649, 2018.

[38] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, "LightGBM: A Highly Efficient Gradient Boosting Decision Tree," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 31, pp. 3149–3157, 2017.

[39] J. R. Lewis, B. S. Utesch, and D. E. Maher, "UMUX-LITE: When There's No Time for the SUS," in *Proc. SIGCHI Conf. Human Factors in Computing Systems (CHI)*, pp. 2099–2102, 2013. doi: 10.1145/2470654.2481287.

[40] S. G. Hart and L. E. Staveland, "Development of NASA-TLX (Task Load Index): Results of Empirical and Theoretical Research," in *Advances in Psychology*, vol. 52, pp. 139–183, 1988. doi: 10.1016/S0166-4115(08)62386-9.

[41] Magic Leap, "Magic Leap 2 Overview," [Online]. Available: https://developer-docs.magicleap.cloud/docs/guides/ml2-overview.

[42] ArUcoGen, "ArUco Marker Generator," [Online]. Available: https://chev.me/arucogen/.

[43] Unity Technologies, "Unity Robotics Hub," [Online]. Available: https://github.com/Unity-Technologies/Unity-Robotics-Hub.

[44] MoveIt Developers, "MoveIt tutorials," [Online]. Available: https://docs.ros.org/en/melodic/api/moveit_tutorials/html/.

[45] MoveIt Developers, "CollisionObject Message (moveit_msgs)," ROS Documentation, [Online]. Available: `https://docs.ros.org/en/melodic/api/moveit_msgs/html/msg/CollisionObject.html`.

[46] XGBoost Developers, "XGBoost Documentation (v3.2.0)," [Online]. Available: https://xgboost.readthedocs.io/en/release_3.2.0/index.html.

[47] Scikit-learn Developers, "sklearn.metrics.fbeta_score," [Online]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.metrics.fbeta_score.html.

[48] Unity Technologies, "Physics Articulations," Unity Manual, [Online]. Available: https://docs.unity3d.com/6000.3/Documentation/Manual/physics-articulations.html.

[49] Unity Technologies, "XR Interaction Toolkit Documentation," Unity Packages, version 2.6, [Online]. Available: `https://docs.unity3d.com/Packages/com.unity.xr.interaction.toolkit@2.6/manual/index.html`.

[50] Unity Technologies, "ROS-TCP-Connector," GitHub repository, [Online]. Available: https://github.com/Unity-Technologies/ROS-TCP-Connector.

[51] Unity Technologies, "URDF Importer," GitHub repository, [Online]. Available: https://github.com/Unity-Technologies/URDF-Importer.

[52] Unity Technologies, "XR Hands," Unity Packages, version 1.6, [Online]. Available:https://docs.unity3d.com/Packages/com.unity.xr.hands@1.6/manual/index.html.

[53] Magic Leap Developer Forum, "Magic Leap 2 Developer Forums," [Online]. Available: https://forum.magicleap.cloud.